

High Integrity Systems Engineering

Subject Notes for SWEN90010

**School of Computing
and
Information Systems
The University of Melbourne**

Contents

1	Introduction to High Integrity Systems Engineering	7
1.1	Subject information	7
1.2	Introduction to high integrity systems	9
1.3	Subject outline	10
2	Safety-critical and Security-critical systems engineering	12
2.1	An introduction to software safety	12
2.2	Why safety engineering?	13
2.3	Safety engineering	15
2.4	Accidents and safety engineering	16
2.4.1	Case Study: The A320 Airbus	17
2.4.2	Case study: Warsaw, Okęcie – 14 September 1993	20
2.4.3	Case Study: Boeing 737 MAX	22
2.4.4	The role of accident analysis in safety engineering	25
2.5	Tools and techniques for safety engineering	27
2.5.1	Safety standards and safety lifecycles	27
2.5.2	Preliminary hazard analysis	28
2.6	Hazard Analysis Methods	31
2.6.1	HAZOPS	31
2.6.2	Fault Tree Analysis	37
2.6.3	FMEA	39
2.7	Threat Modelling for Security	42
2.7.1	The STRIDE Method	42
2.7.2	Attack Trees	45
3	Model-based specification	49

3.1	What is formal specification?	49
3.2	What is model-based specification?	50
3.3	The costs of formal specification	51
3.4	Logic and set theory	53
3.5	Introduction to Alloy	55
3.5.1	Key characteristics	55
3.6	Alloy Logic	56
3.6.1	Everything is a relation	56
3.6.2	Operators	57
3.6.3	Temporal Logic: specifying systems that change over time	61
3.7	Alloy Language	61
3.7.1	Modules	61
3.7.2	Signatures	62
3.7.3	Facts	62
3.7.4	Predicates	63
3.7.5	Functions	64
3.7.6	Assertions	64
3.7.7	The complete <code>LastPass</code> example	65
3.8	Analysis with the Alloy Analyser	67
3.8.1	Runs	67
3.8.2	Checks	67
3.8.3	Scope	68
3.9	Abstraction and incremental specification of state machines using Alloy operators	69
3.9.1	The specification process	69
3.10	Trace-Based Modelling in Alloy	73
3.11	Verification and validation	75
4	Introduction to Ada	79
4.1	The history of Ada	79
4.2	Ada for programming “in the large”	80
4.3	Hello World	80
4.4	Specifications and program bodies	82
4.5	Comments	84
4.6	Types	84

4.6.1	The predefined Ada type system	84
4.6.2	Arrays	86
4.6.3	Defining new types	87
4.7	Control Structures	88
4.8	Procedural Abstractions and Functional Abstractions	91
4.8.1	Parameter Modes	91
4.8.2	Calling subprograms	91
4.9	<code>access</code> Types: Passing Parameters By Reference	92
4.9.1	Access Types to Read-Only Data	95
4.9.2	Summary	95
4.10	Example: Insertion sort	95
4.11	Concurrency with tasks	96
4.12	Further reading	99
5	Safe programming language subsets	101
5.1	Principles of correctness by construction	101
5.2	Safe programming language subsets	102
5.3	SPARK	104
5.3.1	SPARK Structure	104
5.3.2	What is left out of Ada?	105
5.3.3	Why bother with SPARK or other safe programming language subsets?	107
5.4	SPARK Examiner	108
5.4.1	Annotations	108
5.4.2	SPARK Examiner	109
6	Design by contract	112
6.1	Introduction to Design by Contract	112
6.1.1	History	112
6.1.2	Some Definitions	113
6.2	Specifying ADT interfaces	113
6.2.1	ADT interfaces as natural-language comments	113
6.2.2	ADT interfaces as invariants, preconditions, and postconditions	114
6.2.3	ADT interfaces as contracts	115
6.2.4	Formal contracts	117

6.2.5	Advantages and disadvantages of formal contracts	118
6.3	Contracts in SPARK	118
6.3.1	Syntax	118
6.3.2	Similarity to Alloy	122
6.3.3	Tools	122
6.4	Additional reading	122
7	Reasoning about program correctness	123
7.1	Introduction to reasoning about programs	124
7.1.1	The correctness of binary search	124
7.2	A small programming language	125
7.3	Hoare logic	127
7.3.1	Introduction to Hoare logic	127
7.3.2	An introductory example — Incrementing an integer	128
7.4	The rules of Hoare logic	128
7.4.1	Assignment axiom	128
7.4.2	Consequence rule	129
7.4.3	Weakest Precondition Reasoning	130
7.4.4	Sequential composition rule	130
7.4.5	Empty statement axiom	134
7.4.6	Conditional rule	134
7.4.7	Iteration rule	136
7.4.8	Establishing loop invariants	141
7.4.9	Procedure call rule	143
7.4.10	Other rules	146
7.5	Mechanising Hoare logic	146
7.6	Dijkstra’s weakest precondition calculus	147
7.6.1	Transformers	147
7.6.2	Program proof	148
7.7	Additional reading	148
8	Advanced Verification	150
8.1	Pointers and Aliasing	150
8.2	Safe Programming with Pointers: Avoiding Unsafe Aliasing	151

8.2.1	Unsafe Aliasing	152
8.2.2	Avoiding Unsafe Aliasing in SPARK	152
8.2.3	Anti-Aliasing Assumptions in SPARK	153
8.2.4	SPARK Pointer Checks	153
8.2.5	Other Programming Languages: Rust	154
8.3	Separation Logic	154
8.3.1	Pointers in C	154
8.3.2	Separation Logic (for C)	155
8.3.3	The Meaning of Separation Logic Statements	157
8.3.4	The Rules of Separation Logic	158
8.3.5	An Example Proof	160
8.4	Security: Information Flow	162
9	Fault-tolerant design	163
9.1	Introduction to fault-tolerance	163
9.2	Hardware Redundancy	166
9.2.1	Redundancy	166
9.2.2	Static pairs	167
9.2.3	Redundancy and voting	167
9.2.4	N-Modular Redundancy (NMR)	171
9.3	Software Redundancy	172
9.3.1	N-Version Programming	172
9.3.2	Recovery blocks	174
9.4	Byzantine Failures	177
9.5	Information Redundancy	177
9.5.1	Duplication	177
9.5.2	Parity coding	178
9.5.3	Checksums	180
9.6	Fault tolerance in practice	180
9.6.1	Airbus A310–A380	180
9.6.2	Boeing 777	181
9.6.3	Boeing 737 MAX Flight Control Redesign	181
9.7	Additional reading	182

Chapter 1

Introduction to High Integrity Systems Engineering

1.1 Subject information

Subject aims

The main aim of the subject is to explore the principles, techniques and tools used to analyse, design and implement high integrity systems. *High integrity systems* are systems that must operate with critical levels of security, safety, reliability, or performance. The engineering methods required to develop high integrity systems go well beyond the standards for less critical software.

Topics will include:

- high integrity system definition;
- programming languages for high-integrity systems;
- safety and security analysis;
- modelling and analysis;
- fault tolerance; and
- proving program correctness.

Subject outcomes

On completion of this subject students should be able to:

- Demonstrate an understanding of the issues facing high integrity systems developers.
- Demonstrate the ability to analyse highly dependable systems and to synthesise requirements and operating parameters from their analysis.

- Demonstrate the ability to choose and make trade-offs between different software and systems architectures to achieve multiple objectives.
- Develop algorithms and code to meet high integrity requirements.
- Demonstrate the ability to assure high integrity systems.

Generic Skills

The subject is a technical subject. The aim is to explore the various approaches to building software with specific attributes into a system.

The subject will also aim to develop a number of *generic skills*.

- We encourage *independent research* in order to develop your ability to learn independently, assess what you learn, and apply what you learn.
- You will be developing experience at empirical software engineering, and the interpretation and assessment of quantitative and qualitative data.
- Finally, and perhaps most importantly, you will be encouraged to develop critical analysis skills that will complement and round what you have learnt so far in your degree.

Preconditions

The reason for giving preconditions is that they specify the knowledge assumed at the start of the subject and consequently the knowledge and skills upon which the subject builds. Preconditions also allow students from Universities other than The University of Melbourne and who do not have the formal prerequisites listed above to judge whether or not they have the knowledge required for the subject. Specifically the subject assumes:

1. Knowledge and experience of software engineering processes and practices, the risks associated with software and system development, quality and quality assurance, software process, and software project management.
2. Knowledge and experience of requirements elicitation and analysis techniques, requirements modelling and specification, use-case modelling, and UML or similar modelling notations.
3. Knowledge and experience of software architectures, architectural styles and design patterns, design with UML or similar graphical notation, developing code to meet architectural and detailed designs.
4. Knowledge and experience of discrete mathematics, including predicate logic and formal proof.

Resources

The LMS contains information about the subject staff, mode of delivery, and assessment criteria. It is the primary resource for the subject.

All announcements will be made via the LMS.

It also contains all assignment sheets, notes, lecture recordings, tutorial/workshop sheets and solutions.

<http://www.lms.unimelb.edu.au/>.

1.2 Introduction to high integrity systems

What is a high integrity system?

High integrity systems are software controlled systems in which, in the event they fail, could result in harm to humans (including loss of life), harm on the environment, mass destruction of property, or harm to society in general.

There are four broad areas that are commonly considered as high integrity systems:

- *Safety-critical systems*: Systems whose failure may result in harm to humans or the environment. Examples include aerospace systems, medical devices, automotive systems, power systems, and railway systems.
- *Security-critical systems*: Systems whose failure may result in breach of security. Examples include defence systems, government data stores.
- *Mission-critical systems*: Systems whose failure may result in the failure of some deliberative missions. Examples include navigation systems on autonomous aerospace vehicles, computer-aided dispatch systems (e.g. in emergency services), and command-and-control systems in armed forces.
- *Business-critical systems*: Systems whose failure may result in extreme financial loss to a business or businesses. Examples include stock exchange systems, trade platforms, and even accounting systems (e.g. in a bank).

What is high integrity systems engineering?

The high impact of failure of a high integrity system means that the software controlling the system must be highly *dependable*. The standard software engineering methods used for constructing good software systems typically do not lead to such dependability. As such, specialised methods for engineering software are required for high integrity systems.

Avizienis et al.'s work on dependability attributes help us to understand high integrity systems based on six key attributes, shown in Figure 1.1. For a single high integrity system, one or more of the above attributes will be of high importance.

How do we achieve high-integrity?

High integrity is achieved using a combination of:

- *Rigorous engineering processes*, such as strict configuration management, auditing, and review process. We will not introduce any new theory on these topics in this subject.

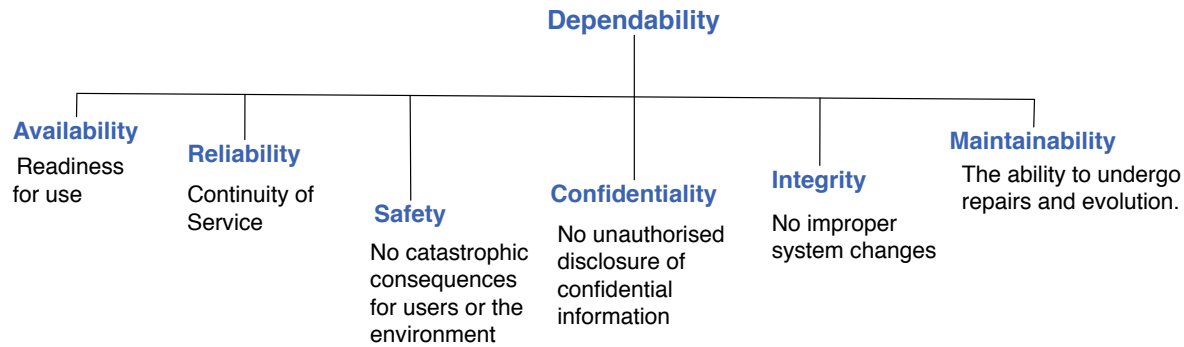


Figure 1.1: Dependability model from “Avizienis, Laprie & Randell: *Fundamental Concepts of Reliability*”

- *Specialised analysis and design methods*, such as safety & hazard analysis and fault prediction methods. These will be covered in Part II of the subject.
- *Specialised modelling and analysis methods/notations*, such as mathematical logic and set theory. These will be covered in Part III of the subject.
- *Fault tolerant design methods*, such as redundancy, design diversity, and fault detection mechanisms. These will be covered in Part IV of the subject.
- *Specialised assurance methods*, such as model checking, proof, and model-based testing. These will be covered in Part V of the subject.

How do we achieve high-integrity?

Methods that are otherwise considered too expensive to apply to “standard” software systems become cost-effective in high integrity domains. This is because the cost of failure far outweighs the effort involved required to engineer a dependable product.

Of importance is that high dependability cannot be demonstrated by just running large set of tests. Although testing still plays an important role in high integrity systems, a really large set of tests is still usually only a fraction of all possible behaviour. As such, testing does not demonstrate dependability.

Perhaps the important lesson to learn from this introduction is:

“Program testing can be used to show the presence of bugs, but never to show their absence!” — Edsger W. Dijkstra [1].

1.3 Subject outline

Now that we have a better idea about high integrity systems engineering, we define the subject outline:

Part I: Introduction to high integrity systems.

Part II: Safety- and security-critical systems.

Topics will be drawn from: safety engineering, and accident & hazard analysis for high-integrity systems; security analysis, cryptography.

Part III: Modelling and analysis.

Topics will be drawn from: discrete maths for software engineering, model-based specification and analysis, proof, and lightweight verification.

Part IV: High integrity systems design.

Topics will be drawn from: reliability, fault detection, fault containment, fault tolerant design, and design by contract.

Part V: Assurance.

Topics will be drawn from: proving program correctness, programming languages for high integrity systems, safe-subset programming languages, programming tools for high integrity verification, and model-based testing.

Bibliography

- [1] E. W. Dijkstra. Notes on structured programming. T. H. Report 70-WSK-03, Technological University Eindhoven, Department of Mathematics, 1970.

Chapter 2

Safety-critical and Security-critical systems engineering

2.1 An introduction to software safety

The trend in systems engineering is toward multi-disciplinary systems. Many of these systems are deployed outside of the standard desktop domain on which programming is typically taught. Such a trend has safety implications: if a piece of software is used to control a hardware system, it may have safety related issues.

For example, consider the following list of applications that have severe safety implications if they fail:

- Computer Aided Dispatch Systems (CAD);
- The A320 Airbus Electronic Flight Control System (EFCS);
- The Boeing 737 MAX 8 MCAS system
- Biomedical technology, such as pacemakers, and surgical robots;
- Train protection systems;
- Space shuttle;
- Automated vehicle braking systems; and
- Chemical plant control systems.

Such systems have influence well beyond the desktop and an individual user. Software itself cannot harm people – it is merely an abstraction. However, software, when used to control hardware or other parts of a system (e.g. to make decisions or send orders on behalf of a human), can be a root cause of accidents and incidents.

The systems described above all rely on an integration of different technologies, and different engineering disciplines. What is perhaps new to most people in this subject is the reliance on computer systems and software to perform safety critical system functions.

Aims

The aim of the module on safety engineering is to develop an understanding of the *safety engineering* approach to developing safety-critical systems. Specifically:

- We will study accidents to get an idea of how they arise, how they are analysed, and what can be done to prevent or mitigate accidents.
- We will study *safety engineering* processes as the pathway to developing and assuring safety critical systems.
- We will study the methods of safety analysis and assurance that can be applied at all stages of the development process.

2.2 Why safety engineering?

In the examples above, software is part of a much larger and much more complex environment. Running a huge set of tests to show that your software behaves as you expected does not really impact safety a great deal.

In safety-critical systems, testing is a necessary but insufficient means of assuring a safe system. Remember that testing is about finding faults – not about showing correctness, and especially not about showing safety.

Safety and security is not just a software issue.

System safety and security involve the hardware that the software is running on, the physical devices to which the software is connected the people which must operate the systems, and operating procedures in force.

Software and its relation to safety

The following are important themes in safety engineering:

- Software is always part of a larger system.
- Design faults can exist at various levels:
 - Written requirements do not conform to the actual requirements.
 - The specification does not conform to the written requirements.
 - Source code does not conform to the specification.
 - Object code does not match the source code.
- Incorrect requirements are a major cause of accidents.
- Simple design faults rarely cause accidents *on their own*.
- Human error is inevitable:

“Components developed over several years are hard to modify. Components which have taken several million years to evolve are almost impossible to modify.”

Is safety equivalent to correctness?

One initial mistake to make is to think that, provided our software is *correct*, it will be *safe*. This is fallacious thinking.

Correctness – Correctness alone does *not* guarantee safety.

Consider a knife. If we assume that it is a good knife and so does not contain any “faults” then is it safe? The answer of course relies on how it interacts with its environment, not on whether or not it contains faults. If I do not get my thumb out of the way while cutting, I can harm my thumb. The knife is not safe. But it can still function correctly.

Now, let’s turn to a systems engineering example. I can design a braking systems for a car that is 100% correct, but 0% safe. My design consists of a braking system with a brake pedal that does nothing. Implementing the design correctly is straightforward: do nothing. But this is surely not safe – unless the wheels are designed to be square. Again, safety results from how our system interacts with its environment.

This does not imply that we should not also strive for correctness. An incorrectly implemented design is more likely to result in safety concerns than a correctly implemented one.

Is safety equivalent to reliability?

Is safety then about reliability?

Reliability – Reliability alone does not guarantee safety.

Consider a box of matches. An unreliable match — that is, one that fails to perform its function according to specification — is far more likely to be “safe” than a reliable match.

A **common misconception** is that if we just get all of the faults out of our system and stop it from failing then all will be well and our system will be “safe”.

Reliability is the probability that a system will work without failure for a specified period of time in a given environment. Thus, reliability is about failure.

Reliability measurements for software are statistical; for example, failure intensity (the number of failures observed per unit time), the Mean Time Between Failure (MTBF), or failure probability gained from testing the computer based part of the system.

The word failure is the key to understanding software reliability.

Reliability and safety

A *failure* is a deviation from the specified system behaviour.

A *failure* in a safety critical system is more commonly a deviation from the intended behaviour – which is to be safe – and especially where the deviation can lead to harm or loss.

In the reliability usage of the word failure, a system may be unreliable but still safe, and worse, it may be completely reliable but totally unsafe.

Example 2.1

Let's consider an example to demonstrate the difference between reliability and safety.

We have already seen that correctness is not the same as safety through the example of the braking system. A similar example can distinguish the difference between reliability and safety. Consider a collection of systems for aircraft.

If I fly on an aircraft, I want a set of systems that always fly (reliable), while also not crashing (safe). However, I can easily implement an unreliable and safe system: never start the engines. The plan will not be able to move, so will be safe. However, it will not be reliable.

Now, consider the converse: something that is reliable but unsafe. We can consider the braking system example above: simply design our aircraft system to have no brakes. I can implement the braking system reliability, but the aircraft will not be safe.

While this example is somewhat extreme, it distinguishes the difference between reliability and safety well. □

2.3 Safety engineering

Safety engineering is the set of processes, methods, and techniques for assuring the system safety. The safety engineering approach is characterised by monitoring safety throughout the product development process – just like other quality attributes.

The safety engineering process ends by ensuring that there is enough evidence to build a *safety case* to demonstrate that the product is safe. Typically, it is not sufficient just to say (or show) that a large set of tests have been run, or that a design has been carefully crafted. In most countries, regulatory authorities require that a safety case demonstrates that safety concerns have been identified, the system has been designed to consider them, and has been implemented to its design.

Definition 2.1

A *safety case* is a high level argument that is presented to regulatory bodies, or customers, to demonstrate that a system is safe. They are generally built up using all of the evidence gathered during the exploratory, deductive, inductive and design processes. □

There are several phases throughout the system development lifecycle in which safety engineering plays an important role:

1. **Requirements Analysis and Product Definition** – Preliminary Hazard Analysis and other methods are used to determine the hazards or potential situations that can cause harm or environmental damage. Their potential causes and impacts are listed.
2. **Design and Implementation Phases** – To guide the design, one typically uses a *deductive* approach: one that starts with potential situation of harm or injury and works back to design or implementation elements. is used. This is typically followed up by an *inductive* approach: one that starts from component or subsystem failures.

During these phases, the system designers and safety engineers work closely to ensure that the design considers the high-risk safety concerns. Systems that must defend against situations of harm or damage must typically be ultra-reliable or must have sufficient redundancy so that a safety related subsystem failure does not lead to a safety related system failure. This is the principle of

“Defending in Depth” against potential accidents. We will discuss more about redundancy later in this subject.

3. **Safety Cases** – Finally, throughout the process, a safety case is constructed, which is used to convince the customer or regulator authority that the system is safe. That is, all non-trivial hazards are identified and the design considers these and deals with them appropriately.

Concepts in safety engineering

The terminology used here is consistent with common usage but beware that terminology differs between standards. Safety as it concerns us in this subject is chiefly concerned with physical products.

- **Platform** — The largest engineered artifact that contains the software, the physical devices on which the software will run (and which connect people to the system), and the operating procedures.

- **Unsafe** — A product is *unsafe* if it causes unacceptable harm; for example, in the loss of life, injury or environmental damage.

Note an important point here. Considering software in isolation from the rest of the system does not tell us anything about its safety or otherwise. When analysing a system for safety the *whole system*, including hardware, other software systems, humans, and the environment, must be considered.

- **Safe** — A product is *safe* if it is not unsafe.
- **Accident** — An accident is an unintended event, or more typically an unintended sequence of events, leading to harm.
- **Incident** — An event that significantly reduces the safety of a platform but does not lead to an accident.
- **Hazard** — a physical situation or state of a platform that can lead to an accident.
- **Failure** — An unintended condition of a system that can lead to a hazard. IEC-61508 states that a *failure* occurs when the delivered service deviates from the specified service.
- **Error** — An unintended state that can lead to a failure.
- **Flaw** – A design defect that can give rise to an error under specific conditions.
- **Fault** — An event that occurs when a flaw is activated resulting in an error.

Note the the definition of *fault* is different in safety engineering than in software testing.

2.4 Accidents and safety engineering

Understanding accidents is key to understanding safety. In this section, we look at accidents and causality.

First, let’s consider some pertinent examples of accidents in which software was a primary contributor:

Therac-25 – Between June 1985 and January 1987, six people died after receiving more than 100 times the intended radiation dose from Therac-25 radiation therapy machine.

A commission into the accident concluded that the primary cause of the radiation overdose was from poor software design, not from coding errors.

A320 Airbus Accidents – The Lufthansa A320 accident at Warsaw airport, and the Mulhouse-Habsheim Airport accident (discussed in more detail below).

London Ambulance – In October 1992, the failure of the London Ambulance Services’ new Computer Aided Dispatch (CAD) system resulted in over 30 deaths and problems in dealing with life-threatening incidents [1].

An investigation concluded that the system was poorly designed and implemented, leading to significant delays in the assigning of ambulances. In some cases, an ambulance took up to 11 hours to arrive.

An upgrade in July 2006 led to repeated system crashes in July and August, and operators reverted to old pen-and-paper methods for ambulance dispatch.

On 8 June 2011, a completely new CAD system, called CommandPoint, was commissioned. On 9 June 2011, the system developed technical problems and operators again had to revert to old pen-and-paper methods. The London Ambulance Service reverted to the previous CAD system on the same day, and CommandPoint was not re-instated until May 2012.

Boeing 737 MAX – Two Boeing 737 MAX 8 aircraft crashed, killing all passengers and crew on board, on October 29 2018 and 10 March 2019. At the time of writing (February 2020) the causes of these accidents are still under investigation. However a flight-control software system called MCAS (Maneuvering Characteristics Augmentation System) of 737 MAX is widely believed to have been a major contributing factor.

2.4.1 Case Study: The A320 Airbus

The Airbus A320 is a *fly by wire* aircraft, in which the pilot’s commands are transmitted to the flight control surfaces as electronic signals rather than mechanically via cables and pulleys.

It has been described as a “*network with an aircraft wrapped around it*”. There are about 150 individual computers on board each of which must communicate with the others.

Fly by wire (and drive by wire) are ubiquitous in modern transport systems. They have many advantages over the old mechanical systems [3]:

- Saving in weight.
- Enhanced control of the aircraft (e.g. anti-lock brakes).
- Simplify retraining.
- Simplify maintenance (i.e. software instead of hardware).
- The flight engineer is made redundant.
- Enhanced safety by creating a protective flight envelope and thereby reducing pilot error.
- Marketing advantage.

A320 On Board Systems

The A320 has the following on-board systems:

- Electronic Flight Control System (**EFCS**).
- Flight Management and Guidance Control system (**FMGC**).
- Full Authority Digital Engine Control system (**FADEC**).
- Electronic Instruments – the “*Glass Cockpit*”.

Habsheim – 26 June 1988

The intention of the flight was to demonstrate the aircraft’s capability at low altitude. It made the first pass slowly at 34ft/10m, flaps extended and landing gear down, successfully.

The second pass was to be done at speed in a *clean* configuration (landing gear raised). The aircraft failed to regain altitude, ploughed into a forest at the end of the runway, and caught fire, killing 3 of the 130 passengers and injuring 34 others.

An official investigation of the accident attributed it to the following causes:

- low fly-over; lower than surrounding obstacles;
- low speed with engines at flight idle; and
- late application of “go around” thrust.

Together with the conclusion that the aircraft was flight worthy and that no malfunction of the on-board systems contributed to the crash, the verdict was *pilot error*.

However, an interesting observation has been made by many safety experts that many more aircraft accidents are attributed to pilot error when the pilot dies than when the pilot lives. This indicates a culture of blaming the pilot first, and then seeking their explanation second.

Further to this, many accidents attributed to “pilot error” are of the form that: “the aircraft did X, but the pilots did not notice Y, therefore, they caused the accident”. For example, in some cases, pilots and co-pilots have failed to notice warning lights. This can be considered quite unfair on the pilots: if neither the pilot nor the co-pilot noticed a warning light, surely it could be considered a failure of the warning light system for not being noticeable? If we agree with the quote in Section 2.2 that human error is inevitable, we should build our systems to consider that humans will make errors, so the system must try to accommodate them.

The Pilot’s Version

The pilot actually survived this crash and so his version has been recorded as well as the official finding. The pilot’s versions stated:

- Company operating procedures were not followed:

- flying lower than the specified minimum runway fly-over altitude (100ft in landing configuration and 300ft in clean configuration according to Air France specifications).
- The flight dossier including Habsheim and its surrounding environment arrived too late for the crew to study it and the crew were never verbally briefed. The first that the crew knew about 30–40ft high trees was when they were 15s flight time away.
- The height chosen for the fly-over was 100ft. The crew thought that they were at 100ft.
 - The captain decided to use the vertical speed indicator and the barometric altimeter to control height (accuracy ± 30 ft). The reason was that the radio altimeter uses a digital display. At low altitude the terrain numbers change so rapidly that it is impossible to read. The barometric altimeter, however, uses an analogue display.
 - The barometric altimeter was checked in the pre-flight checks but was faulty; it displayed 67ft just before take-off but before the wheels had left the ground.
 - The crew were used to a large airport where the control tower is 30m high and the runway is 3km long rather than a control tower 12m high and a runway 900m long as at Habsheim. The crew were also deceived by other visual cues.
- But technical factors contributed to the accident as well;
 - There are two Flight Management Guidance Computers (FMGC), one acting as the primary guidance system and one as a standby. These were not communicating prior to the accident. The pilots had to enter all of the flight data by hand into both the primary and the standby computers. Note: if there is a discrepancy between the two FMGCs and the standby was needed then the navigation screens would go blank (a fault noted a day earlier). The crew decided to enter only one data point to minimise data entry errors.
 - The engines took almost 9s to spool up rather than the 5s that the engines are physically capable of. The pilot's view, however, disagrees with the official report that the engines performed according to specification.
 - Pitch control anomalies in the functioning of the Electronic Flight Control Systems (EFCSs).

What can be concluded?

“L'affaire Habsheim is not over yet” – Mellor [3].

- The spool-up timing of the engines is critical. If it is 5s as claimed then this makes the pilot's story suspect, but if it is 9s as claimed by the pilot then the official findings become seriously questionable.
- Whatever the crew's mistakes were that led to the situation over Habsheim there is evidence that the functioning of the on-board systems led the pilot to a situation (Hazard) that he could not recover from. This is a point of some controversy still.
- The complexity of the A320's systems seems to be part of the cause for the controversy.

However, the most important conclusion that *we* should take from the case study is:

The accident, like many others, was a failure of a system which included operational procedures, crew training and environment as well as the on-board systems.

2.4.2 Case study: Warsaw, Okecie – 14 September 1993

The Kulmbach, a Lufthansa A320-200, was making its final approach on runway 11 at Okecie airport, Warsaw with 64 passengers and a crew of six on board. There was a violent storm and the water was not draining well from the newly resurfaced runway. The runway is 2800m in length but the aircraft touched down 750m along the runway at a speed of 170 knots (see Figure 2.1). This was too late and too fast but a fully laden A320 can stop in 1000m. In this case there was an accident and the aircraft hit an earthen embankment at the end of the runway, killing the co-pilot and one passenger.

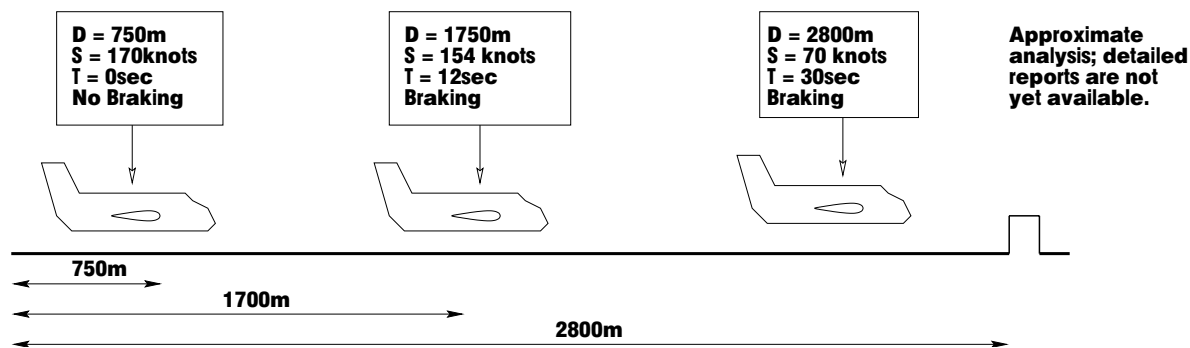


Figure 2.1: The Lufthansa A320-200 at Okecie airport, Warsaw.

Braking logic: Landing an A320 normally

Figure 2.2 presents an architectural view of the A320 braking system. The weight on the wheels (WoW) and the altitude (measure by the altimeter) are the two inputs.

The logic of the braking system is as follows:

brake if and only if:

(WoW > 12 tonnes) *and*

(wheels spinning > 72 km/hr *or*

(wheels spinning *and* radia alt < 10 feet))

The Okecie accident sequence

1. The crew were informed that the wind on the ground was coming from the right and slight in front of the aircraft at 10 knots. The crew were not informed that during the final approach the wind changed to a tail wind of 25 knots.
2. To compensate for the cross wind coming from the front of the aircraft the crew had chosen a landing speed of 152 knots rather than the standard 132 knots which was normal procedure in bad weather of this kind.
3. Expecting a side wind the crew had banked the plane to the right. The result was that only the right landing gear touched the runway. The left gear did not come down for a further 9s.

A/G is indicated by both WOW switches. The LGCIU could not signal A/G and so the SEC did not deploy the ground spoilers and the FADECs did not deploy reverse thrust.

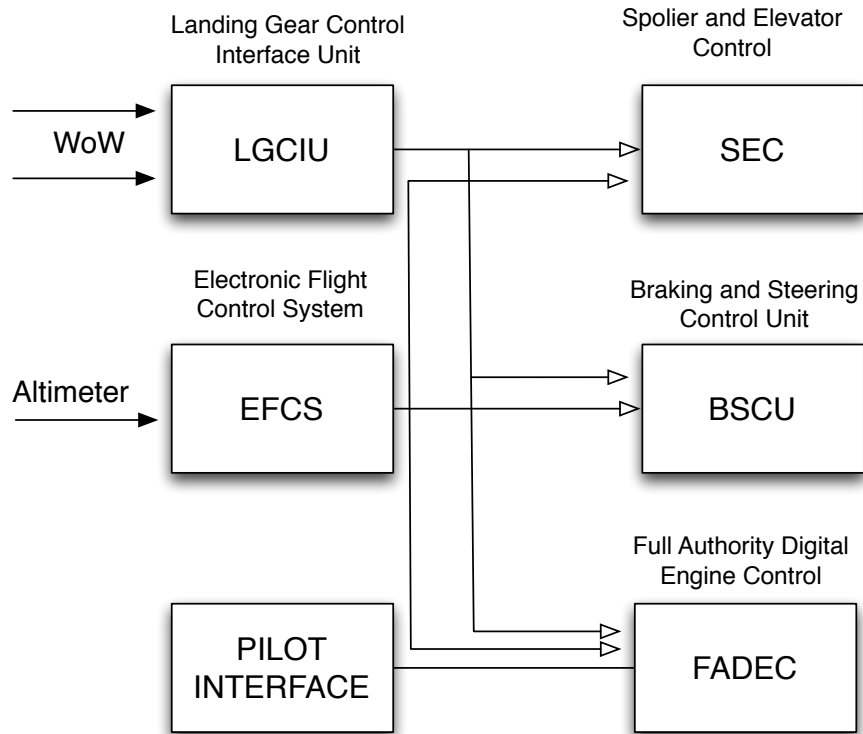


Figure 2.2: Braking system for an A320 landing normally

4. The wheel that had made contact with the runway was aqua-planing and so did not rotate at the required speed corresponding to 72 knots. Thus the other condition for the SEC to deploy spoilers was not true.

Conclusions from the Okecie accident

There is a lack of published evidence in the case of the Kulmbach accident at Okecie. Some conclusions are drawn by Mellor [3]:

- The crash was not due solely to pilot error. The pilot may have contributed to the accident by selection of the landing speed but the Kulmbach should have been able to stop in the distance even at the speed of 170 knots.
- The drainage of water from the runway and the weather conditions were necessary conditions for the accident but probably not sufficient on their own. Several other aircraft landed successfully on runway 11 both before and after the Kulmbach in the similar conditions.
- The A320 systems may have performed to specifications but did not behave as required. In retrospect, the braking logic is restrictive.

Again, it was the failure of the entire system that led to the accident.

2.4.3 Case Study: Boeing 737 MAX

On 29 October 2018 a Lion Air Flight 610 crashed killing all passengers and crew. On 10 March 2019 Ethiopian Airlines Flight 302 also crashed, leaving no survivors. Both aircraft were Boeing 737 MAX 8; 346 people were killed.

The precise causes of each of these incidents is still to be determined, yet it is widely believed that a major contributing factor to both was a software flight-control system of the 737 MAX known as Maneuvering Characteristics Augmentation System (MCAS).

MCAS is a software component of the 737 MAX's flight control computer. It is understood to have been added to the 737 MAX in an attempt to have the 737 MAX mimic the handling of a prior 737 series aircraft, the 737 NG.

The following is a synthesis of the current understanding of MCAS and its role in these accidents, as drawn from a range of news reports available online in February 2020. This synthesis should therefore be treated as informed speculation as gleaned from reputable news media. However it should not be considered authoritative. Because both accidents are still under investigation, authoritative sources on the precise causes of the accidents are not yet available.

Why was MCAS added? The 737 MAX has larger engines than prior 737 series, and those engines are fitted in a different position. Specifically, in order to achieve increased efficiency compared to the 737 NG, the MAX uses a different kind of engine. Those engines are bigger than the ones on the NG. Because of their increased size, they need to be mounted in a different place on the wings, compared to prior 737 series.

Changing the placement of the engines means that the aerodynamics of the aircraft are different. In particular, it means that the aircraft becomes more likely to tilt upwards more steeply than is desired, while in flight.

MCAS's goal is to compensate for the MAX's tendency to tilt upwards. There seems to be some debate about whether MCAS was introduced because (a) the tendency to tilt upwards could lead to the plane stalling (e.g. at low speeds), or (b) whether it was introduced only because the tendency to tilt upwards would mean that the MAX would simply handle differently to the prior NG series 737s. In either case, any deviation in its handling from the prior NG series 737s would have meant that extra training for pilots, who were already trained to fly the NGs, would have been required.

As we understand, avoiding extra training was important for ensuring the MAX was as cheap as possible for airlines to buy and to use. We discuss this more below.

How MCAS Operated The MCAS system attempts to detect when the aircraft's nose tilts up too steeply. Technically, the tilt of the aircraft is referred to as its *angle of attack* (AOA). MCAS attempts to detect when the AOA is too high and, in that case, its job is to automatically push the nose of the plane downwards, lowering the AOA.

The aircraft includes AOA sensors. MCAS determines when the AOA is too high by reading one of the AOA sensors. When it determines that AOA is too high, it adjusts the aircraft's horizontal stabiliser, which causes the nose to pitch downward. When it does this, we will say that MCAS is *activated*.

Immediate Cause of the Accidents It is believed that the immediate cause of the accidents was that MCAS repeatedly activated. That caused it to repeatedly push the nose of the plane downward, each time it repeatedly activated, forcing the plane to fly downwards without the pilots being able to correct its trajectory until it eventually crashed into the Earth.

It is believed that the repeated activation of MCAS was caused by it receiving faulty AOA sensor readings.

Contributing Factors

There are a number of factors that contributed to these accidents that are worth understanding.

Lack of Redundancy The first is that, while the 737 MAX has two AOA sensors, the MCAS system took input from only one of them. Thus there was no redundancy that would have allowed it to detect that it was receiving erroneous AOA data. We will discuss more about the need for redundancy later in the subject. However note for now that the single AOA sensor becomes a single point of failure in the MCAS design. As both accidents appear to indicate, a failure of a single AOA sensor led to the plane crashing without pilots being able to intervene.

Lack of Human Override It is believed that in each accident, the pilots repeatedly attempted to try to bring the nose of the plane upwards to correct its trajectory, by moving the control column. However MCAS was designed so that it could not be overridden by the pilots moving the control column. This was in contrast to previous 737 series.

Lack of Documentation and Crew Training Because MCAS was considered to be a part of the flight control system, it was not described in the flight manual or in training materials. It is believed that this was a deliberate choice to maximise the appearance of similarity to the previous generation 737 NG.

As noted earlier, the goal was to minimise the cost of the aircraft to airlines by ensuring that pilots and crew who were already trained in the NG series would not need much additional training to fly the MAX series.

Thus the pilots were likely unaware of the MCAS system, its potential effects on the plane, nor how to correct for those effects.

Culture: Economics vs Safety

Studying the 737 MAX 8 incidents and their believed causes shows that a range of factors contributed to them. These factors share some common themes. Perhaps the most striking of which is a repeated desire to apparently prioritise lowering cost over redundancy and safety.

For example, the desire to ensure as much commonality between the handling of the MAX and the earlier NG series, while wanting to use larger, more efficient engines, is believed to have been driven by economic concerns. Firstly, more efficient engines mean that the plane is cheaper for airlines to run, because it uses less fuel. Secondly, commonality with the earlier NG series means that little additional training is required for pilots, which further reduces the cost to airlines who purchase the plane. So both

of these choices appear to have been driven by economic concerns: to make the 737 MAX as attractive as possible to potential buyers.

As we have seen, these choices however led to the introduction of the flawed MCAS software system.

We discussed already how MCAS's design was flawed, because it took readings from only one of the AOA sensors, leading to a single point of failure. It is reported that the reason for this design choice was also economic. Specifically, that design ensured that extra flight simulator training would not be required by pilots.

If true, this indicates that the MCAS design violated one of the most fundamental principles of engineering reliable systems—namely, avoiding single points of failure that can lead to catastrophic consequences—for the sake of saving (or, rather, making) money.

So not only did economic factors drive the introduction of MCAS, but they also drove its flawed design.

Since these incidents, whistleblowers have also larger concerns about Boeing's aircraft and its engineering processes. These include concerns about the oxygen system on the 787 Dreamliner, as well as concerns about the operation of certain Boeing factories in the US, in which safety was not given sufficient priority.

While we of course cannot make judgements about the culture at Boeing specifically, together these reports highlight the critical role that culture plays in an organisation. Decisions to prioritise economics over safety (whether made implicitly or explicitly) are ultimately a product of an organisation's culture and values.

Response and Fix

MCAS Fix At the time of writing (February 2020), Boeing is currently working to fix the problems identified with the MCAS system. Its reported fix involves three changes:

1. Having MCAS take AOA readings from both AOA sensors. If the two sensors disagree too much, MCAS will assume one is faulty and will not activate. This guards against a single faulty sensor.
MCAS will also only activate when both sensors agree and move from a low AOA reading to a high one. This guards against a fault in which both sensors get stuck reading a faulty high value.
2. When MCAS activates, it will now activate once only rather than repeatedly.
3. MCAS's ability to move the horizontal stabiliser has been reduced, thereby allowing the pilots to override it by moving the control column.

As part of the ongoing investigation into the 737 MAX, an additional point of vulnerability was identified in the MAX's flight control systems. This was the potential for bit-flip errors to occur in the hardware running the flight control computer. As a result the architecture of the flight control system was overhauled to increase redundancy. We discuss this further in subsection 9.6.3, when we discuss fault tolerant design.

Broader Consequences and Reflections On 10 March 2019, all 737 MAX aircraft were indefinitely grounded, pending investigation of both incidents. As of February 2020, all 737 MAX remain indefinitely grounded. Aside from the 346 deaths, it remains unclear what precise economic cost the grounding

will have on Boeing (remembering of course that the MAX's original design was motivated by a desire to save money).

In December 2019, Boeing removed its CEO over the incidents and their handling. In 2019, airlines reportedly cancelled 183 orders for the 737 MAX. Each MAX reportedly costs at least 100 million US dollars. Those cancelled orders alone therefore represent a potential loss of 18.3 billion US dollars. Of course that cost doesn't account for indirect costs, including those of investigation, remediation and re-certification of the aircraft, nor of those due to lost production (since production was halted in January 2020).

It has been reported that the grounding is projected to lead to losses of 18.4 billion US in future.

The 737 MAX case study neatly illustrates the oft cited engineering maxim that the best (i.e. the cheapest and fastest *over the long term*) way to build a system is to build it right the first time. (As opposed to taking shortcuts that lead to having to re-engineer a system once it is in production.)

Returning to the theme of culture, we may conclude that in safety-critical engineering, a culture that prioritises economics over safety may lead to short-term economic gains, but may ultimately lead not only to catastrophic loss of life but also to *higher* economic costs over the long term.

2.4.4 The role of accident analysis in safety engineering

Accidents, and the study of accidents, are an important part of safety engineering. The most useful way to identify hazards in safety-critical systems is to look at similar systems and study the factors that contributed to accidents (and incidents) in those systems, and to specify safety requirements from both these and the system models (see Figure 2.3). Accident analyses are used during Preliminary Hazard Analysis (for example during HAZOP) to understand potential accidents, causes consequences and risks.

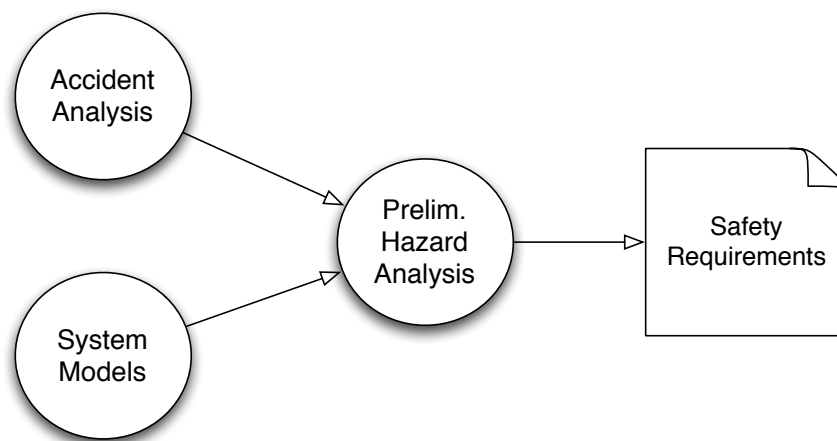


Figure 2.3: The role of accidents in safety engineering

Analysing Accidents: Event Chains

The common method for analysing accidents is to create an *event chain* leading back to a set of accident causes. Figure 2.4 shows an example of an event chain for an accident in a steam boiler.

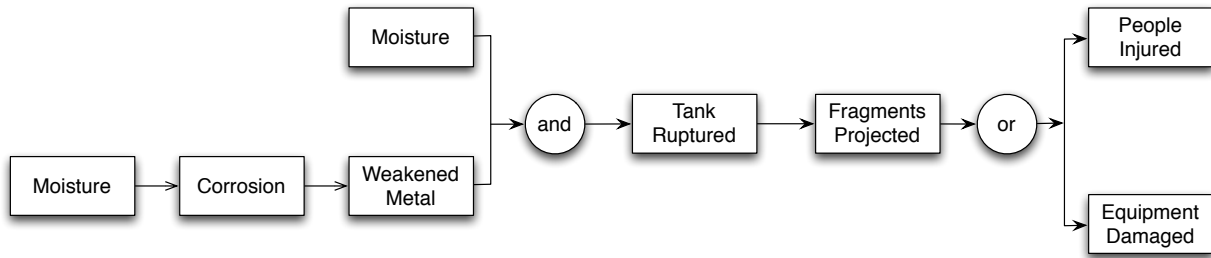


Figure 2.4: An event chain for a steam boiler

Each event in the chain is the cause of the next; for example, moisture is the cause of corrosion, corrosion is the cause of weakened metal in the boiler, moisture *and* weakened metal is the cause of a ruptured tank, etc.

Causality

When can it be said that event A causes event B?

In accident analysis one common way is through the *counterfactual* definition of causality.

Definition 2.2

A *counterfactual* is commonly defined as a conditional expressing what could have happened under different conditions, but did not happen. It is typically expressed in the subjunctive mood: a conditional (or *if-then*) statement indicating what would be the case if its antecedent were true although it is not true. □

As an example, consider the counterfactual: “They would have done better on their assignments, if they had first done the workshops” (take a hint from this).

In accident analysis, “*A causes B*” if we assume that *A* did not occur then *B* would not have occurred. Note that this does not mean that *A* is the sole cause of *B*. As in the accidents described above, often multiple factors contribute in unlikely sequences to cause a single accident.

Counterfactuals and Causality

In the steam boiler example we would reason as follows:

If the metal had not been corroded then the metal would not have been weakened.

The metal was corroded but the counterfactual deals with what *could* have been — in this case, if we remove the corrosion then we can imagine that the metal would not have been weakened (based on our knowledge of corrosion and metal). Consequently the counterfactual is true and we conclude that the corrosion of metal *causes* the weakening.

The essential nature of counterfactuals in accident analysis is the imaged absence of one of the precondition events leading directly to the event being studied. If the consequent is true when we imagine that

the antecedent is true then the counterfactual is true — that is, we imagine the absence of the cause A and if the event B no longer occurs then A causes B .

Counterfactuals and correlation

We must also be careful not to confuse causal relationship with *correlations*.

If it were the case that the barometer falls then a storm would occur.

In this case the barometer falling and the storm occurring are not causally linked — the counterfactual is incidental and *not true*.

2.5 Tools and techniques for safety engineering

The aim of this section is to examine some of the concepts, tools and techniques involved in safety engineering in a bit more depth.

The problem in safety engineering is to develop – with a high level of assurance:

- Systems that do not lead to unintended harm or loss;
- Systems that continue to function in the presence of component faults;
- Systems that perform within hard real-time bounds; and
- Systems that protect data.

2.5.1 Safety standards and safety lifecycles

Software engineering for safety critical systems involves numerous standards each of which requires additional activities in order to guarantee due care with safety critical functions.

For example, in IEC–61508, the safety lifecycle is used to identify:

- key activities where safety analysis techniques are used; and
- the relation between safety critical analysis methods and traditional lifecycle processes.

Engineering for safety, like engineering for other product attributes such as performance, correctness, and usability, is something that must be done over the entire lifecycle. Figure 2.5 shows a possible safety engineering process, and notes some of the techniques and types of methods used in safety engineering.

For each project, or each iteration of a project, the phases of requirements, design, and implementation will have corresponding verification and validation techniques. At the requirements phase, the focus is more on exploratory hazard analysis to determine what could go wrong, while at later phases, it is about designing the system to prevent these.

In the remainder of this chapter, we look at some safety engineering techniques. First, we present a more general overview of preliminary hazard analysis.

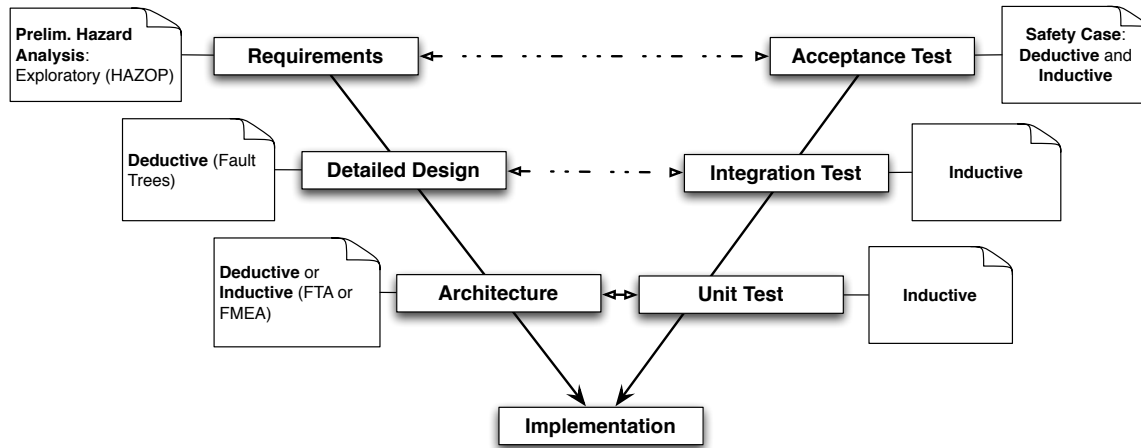


Figure 2.5: A safety engineering process

2.5.2 Preliminary hazard analysis

A *preliminary hazard analysis* is used to identify the hazards and accident scenarios associated with a new system. It is essentially an information collation and filtering exercise, and is usually the first activity carried out in the safety lifecycle. Preliminary hazard analysis methods ask: “What possible accidents or incidents could occur?”

In this section, we look at some of the basic concepts in preliminary hazard analysis.

Phase 1 — Preliminary hazard identification

This is the phase that identifies the hazards and accident scenarios associated with the new system.

- **Inputs – Information sources**

- *Incident and hazard data*: previous in-service incident and accident data of similar systems (usually as checklists).
- *Analyses*: previous dependability analyses for similar systems.

- **Method**

- *Group based*: members experienced in the domain. This is important because people who have domain knowledge and knowledge about accidents are far more likely to think of accidents for the new system.
- *What ifs*: hypothesise safety related difficulties.

- **Output**

- *Preliminary hazard list*: a list of all hazards, their causes.

Phase 2 — Preliminary Hazard Analysis (PHA)

This occurs after the preliminary hazard identification and usually at the later stages of requirements analysis and early stages of design. It has the following properties:

- Possibly included into preliminary design reviews.
- Early identification of hazards and causes in designs.
- It is a poorly defined method:
 - uses brainstorming and questionnaires; and
 - depends on experience and domain knowledge.
- Checklists are commonly used, in an attempt to make the method more systematic, and less reliant on experience and domain knowledge.

Inputs

- design (or perhaps a detailed requirements model).
- preliminary hazard list

Outputs

- list of causes for hazards based on the design.
- risk assessment and risk classes (provisional).
- integrity levels (provisional).
- target failure rates (provisional).

Hazard Logs

The results of the preliminary hazard identification and preliminary hazard analysis form the basis of a *hazard log*.

Definition 2.3

A *hazard log* lists the hazards, their causes and their severity, as well as other data such as target frequencies or hazard types. It is a living document. It evolves during the course of the safety analysis, and can be part of the common literature in special fields such as avionics. □

For example, a preliminary hazard analysis may be summarised in the following hazard log.

Ref.	Hazard	Frequency	Severity	Cause	Integrity Level	Target
...						
...						

Risk

Risk is function of the severity of a hazardous event and the likelihood of it occurring. The *frequency* of the event is the probability that the event occurs in a given period of time.

Assessing Risk

To arrive at a qualitative risk assessment, the idea is to assess consequences against frequencies. Standards usually provide a table for this.

The safety standard IEC – 61508 classifies consequences into:

catastrophic	critical
marginal	negligible

It classifies frequencies into:

frequent	probable	occasional
remote	improbable	incredible

From these two classifications, the standard defines the following table, which classifies each consequence/frequency pair into one of four *risk classes*.

Frequency	Consequence			
	Catastrophic	Critical	Marginal	Negligible
Frequent	I	I	I	II
Probable	I	I	II	III
Occasional	I	II	III	III
Remote	II	III	III	IV
Incredible	IV	IV	IV	IV

The four risks classes, according to IEC – 61508, are:

Class I Intolerable risk.

Class II Undesirable risk and tolerable only if risk reduction is impractical.

Class III Tolerable risk if the cost of risk reduction would exceed the improvement gained.

Class IV Negligible risk.

Using this, we can interpret that frequent events with a catastrophic must not be tolerated, while incredibly rare events in general can be tolerated.

Integrity Levels

Definition 2.4

Safety integrity is the likelihood of a safety related system satisfactorily performing the required safety functions under all stated conditions. □

For example, a system with known failure rates for subsystems:

Integrity Level	Probability of failure to perform system function
4	$\geq 10^{-5}$ to $< 10^{-4}$
3	$\geq 10^{-4}$ to $< 10^{-3}$
2	$\geq 10^{-3}$ to $< 10^{-2}$
1	$\geq 10^{-2}$ to $< 10^{-1}$

Note: the distinction between risk classes and integrity levels. Risk classes are assigned to hazards. Integrity levels are assigned to systems.

Assigning Integrity Levels

It is not always possible to remove hazards entirely. Safety engineers must instead reduce hazard risks to tolerable risk levels. The amount of risk reduction determines the integrity level. The more that the system is required to reduce the risk of hazards the higher the system's integrity level.

There are a number of ways of assigning integrity levels to subsystems, and these can be broken into two categories:

1. Quantitative methods – first determine the uncontrolled risk and then target risk levels numerically, by analysing the failure rates of subsystems from similar situations;
2. Qualitative methods – use risk graphs to determine risk qualitatively.

Some standards, such as IEC-61508, provide for integrity levels which are set according to the severity of the accident.

2.6 Hazard Analysis Methods

Hazard analysis methods fall into three categories:

1. *exploratory*: these methods use expert knowledge and some systematic techniques to imagine potential hazards and their impacts;
2. *causal*: these methods work from system designs and try to deduce what failures are possible from this; and
3. *consequence*: these methods work backwards from possible failures to see how they could occur.

Table 2.1 summarises these three types of methods. In the remainder of this chapter, we will explore the three methods listed as examples in Table 2.1.

2.6.1 HAZOPS

HAZard and OPerability Studies (HAZOPS) is a technique that originated in the chemical processing industry. It has been adapted to many other industries and specifically in software engineering to software HAZOPS and — surprisingly — the study of interfaces in human-computer interaction.

Technique	Example Methods	Good For ...
Exploratory	HAZOP (Hazard and Operability Studies)	Exploring possible hazards, their causes and for completeness, but is labour intensive and hard to change.
Causal	Fault Trees	This is the classic deductive approach and is good for determining what combinations of component failures lead to hazards.
Consequence	FMEA (Failure Modes and Effects Analysis)	This is the classical inductive approach used for working from component failures (failure modes) back to hazards and is good for verifying ideas and designs.

Table 2.1: Types of hazard analysis methods.

A HAZOP study is a well-established technique for preliminary hazard analysis (PHA). The process of identifying hazards requires creativity, lateral thinking, and expert knowledge. The HAZOPS method was introduced into software engineering to provide some repeatable and systematic process into hazard analysis.

The basic technique is straightforward: a system consists of some behaviour. A HAZOP study explores the behaviour by taking a set of generic *guidewords* that describe some alternative to intended behaviour, and exploring what hazards could occur if that alternate behaviour occurred.

Note: this is different to testing where we wish to find deviations from *specified behaviour*.

Guidewords

The guidewords for every HAZOP study are the same¹, and are listed in Table 2.2.

Guide Word	Deviation
NO or NONE	This is the complete negation of the design intention. No part of the intention is achieved and nothing else happens.
MORE	This is a quantitative increase.
LESS	This is a quantitative decrease.
AS WELL AS	All the design intention is achieved together with additions.
PART OF	Only some of the design intention is achieved.
REVERSE	The logical opposite of the intention is achieved.
OTHER THAN	Complete substitution, where no part of the original intention is achieved but something quite different happens.
EARLY	Something happens earlier than expected relative to clock time.
LATE	Something happens later than expected relative to clock time.
BEFORE	Something happens before it is expected, relating to order or sequence.
AFTER	Something happens after it is expected, relating to order or sequence.

Table 2.2: HAZOPS guidewords

¹Although there is nothing to prevent software engineers from adding or removing guidewords.

Roles

There are several roles that can take part in a HAZOP study, including:

1. A Team leader who is responsible for:
 - Planning and Preparation.
 - Chairing the HAZOPS meetings.
 - Signing any documentation.
 - Ensuring follow-up work is completed.
2. Recorder who is responsible for:
 - Documenting the analysis.
 - Participating in the study.
3. Designers
 - Understand and explain the (potential) design.
 - Answer questions about the process.
4. User(s)
 - May be operators, or representatives of other user groups such as maintainers of the system.
 - Supply information about the context in which the system will be used.
 - Help decide issues that may affect safety.
5. Expert(s). The key function of experts is to *explore*:
 - Ask questions.
 - Suggest deviations, causes and effects.

Process

The process for a HAZOP study is outlined in Figure 2.6, taken from [2]. From this figure, we can see that each design item, such as an event in a process, or a transition in a state machine, is inspected. For each item, the item is analysed using the HAZOP guidewords.

For example, consider our earlier idea of brake-by-wire system for a car. One design item contains the intended behaviour that, when the brake pedal is pushed, a signal must be sent to the ECU immediately to let it know that this event has occurred. Using a HAZOP study to analyse what happens if the systems deviates from intention, we can apply the guidewords NONE, EARLY, and LATE, and then reason the following:

- NONE: If no signal is sent, the ECU will not know that the brake has been pushed, so will not send the signal to apply the brakes to the wheels. This will result in the vehicle not decelerating, ultimately resulting in a hazard.

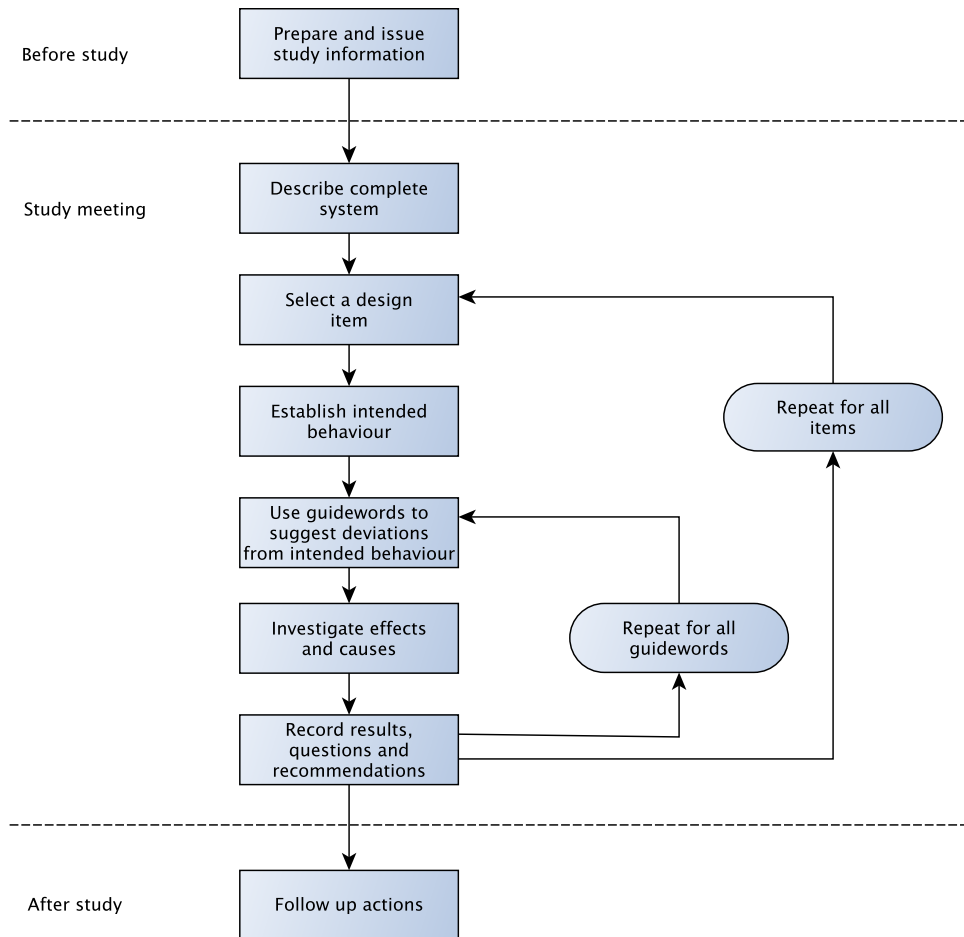


Figure 2.6: A HAZOP study process.

- **EARLY:** We want the signal to be sent immediately, so a signal cannot be sent early, and therefore, no hazard occurs.
- **LATE:** If the signal is sent late, the ECU will process it late and send a signal late to the brakes. This will result in the vehicle decelerating later than anticipated, ultimately resulting in a hazard.

The result is a log of the entire analysis. For each design item + guideword pair that may result in a hazard, the following details are recorded:

1. The deviation from intent (the interpretation of the design item + guideword for this system).
2. The causes of the deviation.
3. The consequences of the deviations.
4. Any safeguards that are in place to prevent the deviation or its cause.
5. A list of recommendations for mitigating the problem.

A template/examples for this is shown in Table 2.3.

ID	Guideword	Deviation	Causes	Consequences	Safeguards	Recommendations
1.1	NONE	...	...	...	...	...
1.2	EARLY	...	...	...	...	...
1.3	LATE	...	...	...	...	...
...						

Table 2.3: A template for a HAZOP log.

Limitations

There are several limitations with using HAZOPS as preliminary a hazard analysis technique:

1. It requires a well-defined description of the intended behaviour. In some projects, this may not be available until quite late.
2. It is extremely time and resource intensive. Several people in a team analysing every design item for every guideword is time consuming, and the people in the team are generally highly experienced.
Despite this, the process is creative and challenging, and many people find it rewarding and interesting. Not me though ...
3. To conform to most safety standards, to devise a safety case, and to enable traceability of decisions, the result of the entire analysis must be recorded. That is, the outcomes of each guideword + design item pair must be recorded, including causes, consequences, and recommendations for mitigating the problem. This results in large amounts of documentation that need to be reviewed and maintained.
4. It focuses on single failures that cause of deviation from intent. However, an accident can occur due to a series of unlikely circumstances along a chain of events in a system. Despite this, the use of guidewords may prompt system designers to consider chains of deviations.

Example 2.2

The following example of a *stepper motor* HAZOP study is taken from Wei and Zhangjin [5].

A stepper motor is a electric motor used for rotation of mechanical parts. The rotation is divided into discrete number of equal size steps. The position of the motor can therefore be determined without the use of sensors for feedback, provided the motor size is known. Figure 2.7 shows the workings of a stepper motor: a single step moves the teeth around until they are locked in, so counting the steps means that the position of the motor is known. The system is real-time, so when the controller of the motor sends a message for it to rotate, power on, etc., it must do it within a specified (and short!) time frame.

The controller uses a C interface to the control the library, shown in Figure 2.8. Motor identifiers can be created, and a specified motor can be powered on/off, moved forward a number of steps, and braked.

First, we need to infer what the deviations from intent are for each guideword:

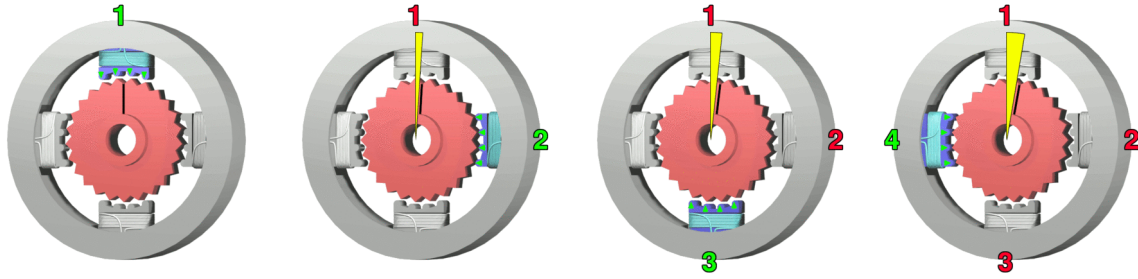


Figure 2.7: An overview of a stepper motor.

```
void write_lpt(struct motor_t *motor);
struct motor_t stepper_motor_setup(int motor);
void stepper_motor_power_on(struct motor_t *motor);
void stepper_motor_power_off(struct motor_t *motor);
void stepper_motor_move_steps(struct motor_t *motor);
void stepper_motor_set_safe(struct motor_t *motor);
void stepper_motor_brake(struct motor_t *motor);
```

Figure 2.8: C interface for a stepper motor.

Item	Guidewords								
	NONE	MORE	LESS	EARLY	LATE	BEFORE	AFTER	REVERSE	...
Power	No voltage	Over voltage	Under voltage	Voltage change	Voltage change	—	—	Backflow	
Step	No steps	Less steps	More steps	—	Late time	—	Wrong order	Wrong direction	
Brake	No brake	Too quickly	Too slowly	Too early	Too late	Wrong order	Wrong order	Accelerate	
...									...

Now that we have the deviations from intent, we need to analyse what the possible hazards that arise as a consequence of these. We do just part of the functionality for moving the stepper motor:

3.0 stepper_motor_move_steps(struct motor_t *motor);						
ID	Guideword	Deviation	Causes	Consequences	Safeguards	Recommendations
3.1	NONE	No steps	No power to motor	Motor does not move		
3.2	MORE	More steps	Power too high	1. Motor goes too fast. 2. Motor may burn out.		Install a voltmeter. Do not apply voltage is over a specified threshold
3.2	LESS	Less steps	Low power due to old wire or large resistor	Motor rotates too slowly		Install a voltmeter. Raise an error is voltage is under a specified threshold
...						

From just these few guidewords, we have noted some potential hazards and provided recommendations for them. Clearly, from the report, some knowledge of stepper motors and their operation is required, even for this small part of the analysis. □

2.6.2 Fault Tree Analysis

Fault trees are one of the classic deductive techniques for hazard analysis and also one of the most common deductive technique in use today. Fault trees are typically drawn by working back from hazards to basic causes. Fault trees are a graphical notation and consist of events and gates.

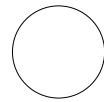
Event and gate symbols

Figures 2.9 and 2.10 show the symbols that are used in fault-tree analysis, dividing into *event* symbols and *gate* symbols respectively.

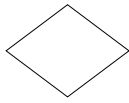
Method

The basic method for working with fault trees is the following:

1. *Select an event*: Initially we select top-level events, which are typically the hazards, or events that can lead to the hazards, identified in the PHA.
2. *Identify immediate, necessary and sufficient causes of this event*:
 - *Immediate*: means that we need to avoid missing out intermediate events and should apply the “*think small*” principle;
 - *Necessary*: means determining those events which must occur for the top-level event to occur and these should be linked with an AND gate;
 - *Sufficient*: means getting those events which alone are sufficient to cause the top-level events on their own and these are linked with an OR gate.

**Basic Event**

An initiating fault requiring no further action

**Undeveloped Event**

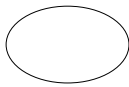
An event which is not developed further, either because it is considered unnecessary or because insufficient information is available.

**Intermediate Event**

An event arising from the combination of other, more basic events.

**Normal Event**

An event which is expected to occur as part of the normal operation of the system.

**Conditioning Event**

Specific restrictions that apply to some types of logic gates, for example the PRIORITY AND or INHIBIT gates.

Figure 2.9: Fault tree symbols

**AND gate**

All of the inputs must occur for the output to occur

**OR gate**

One or more of the input events must occur for the output to occur

**PRIORITY AND gate**

The output occurs if the input events occur in a specific sequence which is described in a conditioning event.

**EXCLUSIVE OR gate**

The output occurs if exactly one of the input events occurs.

**INHIBIT gate**

The output occurs if the single input event occurs in the presence of the enabling condition which is described in the conditioning event.

Figure 2.10: Fault tree gates

3. Classify intermediate events:

- *Basic event* requires no further decomposition.
- *Compound defect*, which may one of (1) a primary failure, which is a simple component

failure and may be classified as a basic event; (2) a secondary failure in which components fail because of external influences and this usually requires further investigation; or (3) command failure in which components receive incorrect signals and this always requires further investigation.

- *System defect*, which is not attributable to a single component and always requires further investigation.

4. *Repeat as required.*

Some heuristics

There are some some heuristics that have evolved over time for the correct construction of fault trees:

1. All inputs to a gate should be defined before any one is examined in more detail.
2. The output of a gate must never be directly linked to the input of another gate – all outputs of gates must go to events.
3. The text in event boxes should be complete in the sense that they should always say what the event is and when it occurs.
4. Causes always chronologically precede consequences.

Example: Chemical mixing plan

Consider the simple chemical mixing plant shown in Figure 2.11 and its accompanying fault tree [2]. The top-level event that leads to a hazard in this case is that the tank may overflow and cause environmental damage.

The immediate, necessary and sufficient causes of this event are that the valve marked A is closed and that the valve marked B is open. Note that these events must occur *before* the tank overflows. The fault tree does not investigate the causes of the event “*valve A closed*” further.

On the other hand, valve B is open if valve B has failed, which is a component failure but is not investigated further, or an incorrect signal has been sent to valve B, which is a command failure and must be investigated further.

2.6.3 FMEA

Failure Modes and Effects Analysis (FMEA) is a tabular technique and is one of the most widely applied inductive (consequence analysis) techniques in industry.

The method specifies that safety engineers should focus on a single component failure, or failure mode, and work forward through subsystems to determine the consequence of the failure mode on the rest of the system. There is a great deal of variation in FMEA practice and, like most hazard analysis methods, it is hard to give a general algorithm for conducting an FMEA.

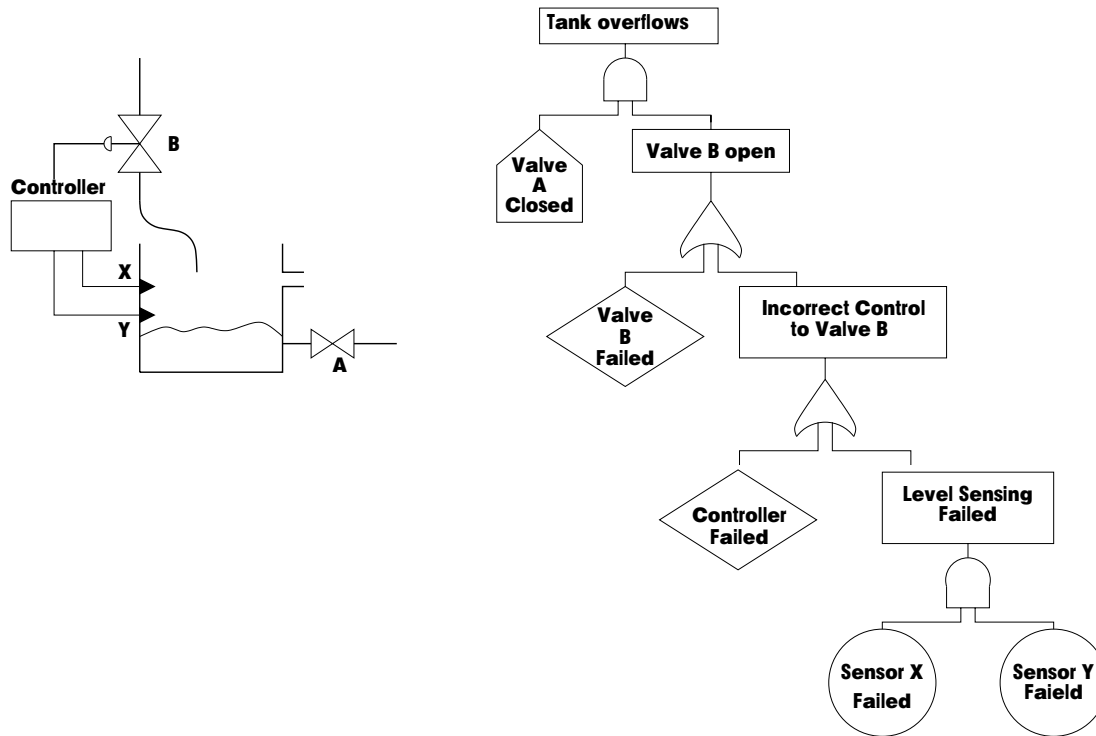


Figure 2.11: A design of a simple chemical mixing plant and its corresponding fault tree.

Applications

FMEA is good at identifying individual elements or operations within a system that render it vulnerable; that is, finding single points of failure. Industries/organisations that frequently use FMEA include: consumer products, automotive, home appliances, aerospace, NASA, Department of Defence, and process industries (e.g. chemical processing).

Failure modes

An FMEA is about identifying the *failure modes* of a system.

Definition 2.5

A *failure mode* is a manner in which a failure can occur, in terms of failure of the item function (such as a function to a sub-system or component) being studied. □

Typical failure modes are:

1. Premature operation.
2. Failure to operate at the required time.
3. Failure to stop operation at the required time.
4. Failure during operation — and this is specific to the equipment.
5. Degraded operational capacity.

6. Excessive operational capacity.

There is a symmetry here between failure modes and the guidewords for HAZOPS; e.g. excessive operational capacity and the guideword “MORE”.

Method

Step 1 Define the system to be analysed and obtain necessary designs, descriptions, diagrams, charts, component lists, etc. The idea is to understand in depth what exactly it is that you are analysing.

- All of a subsystem or just part of it?
- What subsystems or operators must be protected?
- What other subsystems must the subsystem being analysed interact with?

Step 2 FMEA Analysis then uses two questions to guide the analysis.

1. For each analysed element, what are the failure modes?
 - The failure modes can be identified from testing or prior experience with the component; or
 - By appeal to similar components used before; or are
 - Technology specific failure modes; or
 - Can be analysed from damaged components

2. For each failure mode, what are the *failure effects*?

The safety engineer will provide general descriptions of the failure mode and effects, which may also include severity and probability assessments.

Worksheet

The result of an FMEA is an *FMEA worksheet* – a monster of a spreadsheet that documents the failure modes and effects, along with other items, such as causes, probabilities, severities, and recommendations. A worksheet template is shown in Figure 2.12.

Limitations

Like other safety analysis methods, FMEA has several limitations:

- Frequently, human errors and hostile environments are overlooked.
- Because the technique examines individual faults of system elements taken singly, the combined effects of coexisting failures are not considered.
- If the system is at all complex, the process can be tedious and time consuming.

- Failure probabilities can be hard to estimate.
- Sometimes FMEA is done only to satisfy the altruistic urge to “do safety”.

FMEA will find and summarise system vulnerability to *single point* failures, and it will require lots of time, money and effort. An important question to ask before undertaking an FMEA is: *how will the results be used?*

2.7 Threat Modelling for Security

In this section we will discuss common *threat modelling* methods. Similar to hazard analysis, which aims to enumerate hazards, threat modelling aims to enumerate potential security threats to a system.

The *STRIDE* methodology, developed at Microsoft, is a common threat modelling method that has some similarities with the HAZOPS method (see subsection 2.6.1).

Attack Trees are another common method for threat modelling that has similarities to Fault Tree Analysis (see subsection 2.6.2).

We briefly introduce both, drawing heavily on the opening chapters of [4]: *Threat modeling: Designing for security*, by Adam Shostack. Wiley, 2014. You can learn more there, and this book is currently available as an e-book from the University’s library: <http://library.unimelb.edu.au>.

2.7.1 The STRIDE Method

The STRIDE method, akin to HAZOPS, can be seen as a systematic process for brainstorming security threats to a system. While HAZOPS focuses on design items, in which one tends to concentrate on the interactions and communications between system components, STRIDE instead focuses on the interactions that occur between a system’s *trust boundaries*.

Trust Boundaries

Definition 2.6

A *trust boundary* separates one part of a system from another, where the two parts are under the control of different entities who are trusted to differing degrees. □

In any system, there will be parts that we as system designers control, such as the software or the hardware components that we supply ourselves. There will also be other parts that we don’t control: for instance, if we are using an external cloud provider then we likely will not have full control over those parts of our system that run on the cloud platform. A trust boundary exists, therefore, between the parts of the system that we control and those parts that we do not control. For the parts we don’t control, some may be under the control of one entity while others might be under the control of another. If we trust these entities to differing degrees (e.g. perhaps we trust one more than the other because we have a contractual relationship with them) then a trust boundary will also exist between those parts of the system.

Example 2.3

Suppose we are building a website for a business, like the University. The website itself will be hosted

on University servers. However, media like videos and images will be served via a third-party content distribution network (CDN). This scenario is depicted in Figure 2.13.

Each component in this system lives inside a separate trust boundary. In particular, while the web server might be under our control, the user and her web browser is not, and neither is the third-party CDN. However, since we have a contractual relationship with the third-party CDN, we are willing to trust it not to act in a way that would violate that contract. Therefore, we trust it more than we trust the users (and potential attackers who might be on the network). □

The STRIDE Acronym

STRIDE is an acronym: each letter corresponds to a class of threats. The STRIDE approach is about considering each class of threats and how it might apply to the system. This is somewhat analogous to HAZOPS in which each guideword is applied to a system to enumerate hazards.

In a similar way, for each of the classes of threat that STRIDE describes, we consider all of the ways in which that class of threat might apply to our system, i.e. what sorts of attacks might be carried out. STRIDE's purpose is to help systematically enumerate the attacks or threats that our system might be exposed to, and to force us to think about which threats are more serious and should be mitigated in our system's design.

STRIDE therefore forces security engineers to “think like attackers”. The threat classes that each letter of the STRIDE acronym correspond to are:

Spoofing: pretending to be something or someone that you are not.

Tampering: modifying something you are not supposed to.

Repudiation: claiming that you didn't do something (when you actually did do it)

Information Disclosure: revealing or stealing information that you are not supposed to be able to access.

Denial of Service: preventing others from accessing a system, e.g. by crashing it, making it too slow to use, exhausting its storage, etc.

Elevation of Privilege: being able to do something that you, technically, you are not allowed to do (e.g. obtaining Administrator or root access on a server).

We consider each of them briefly, illustrating each using the example system above by giving a few examples. Note that the lists below are not exhaustive.

Also note that the different threat categories are not non-overlapping. Some threats might legitimately belong in multiple categories. For instance, suppose an attacker is able to modify the access logs for a system. This threat could be a repudiation threat, since changing the logs might allow them to claim that they did not do some action that would have otherwise been recorded in the log. However this threat is also a tampering threat.

The goal with STRIDE is not to achieve a perfect categorisation of threats. But rather to aid as a tool to help threat brainstorming.

Spoofing This threat occurs when an attacker pretends to be someone or something that they are not. For instance, an attacker might impersonate a legitimate user of a system (e.g. if they can guess the user's password). Or, in the case of a phishing attack, an attacker might impersonate a legitimate website.

Spoofing threats violate the security goal of **authentication**.

Example 2.4 (Spoofing Threats for Figure 2.13) • One user pretends to be another user of the web-site.

- The attacker impersonates the university's website (via a phishing attack).
- A third party site contains a link to one of the university's pages that causes the user to perform some action on the university site; Cross-Site Request Forgery (CSRF) attack.

□

Tampering This threat occurs when some data is modified that is not allowed to be modified. The modification could occur to data at rest (e.g. stored in a database or on a website) or do data in transit (e.g. as it is being transmitted over the network).

Tampering threats violate the security goal of **integrity**.

Example 2.5 (Tampering Threats for Figure 2.13) • A rogue administrator for the CDN company tampers with the website content hosted on the CDN.

- An attacker tampers with data in transit between the web server and the CDN.
- The attacker modifies the website contents (website defacement).

□

Repudiation This threat occurs when an attacker is able to do something bad, but then later on claim that they did not do it. It usually occurs when a system does not have sufficient logging of actions, or when the logs are not properly protected.

Repudiation threats violate the security goal of **non-repudiation**.

Example 2.6 (Repudiation Threats for Figure 2.13) • The attacker deletes or modifies the access logs for the web server.

- The attacker fills up the disk space on which the log file is being stored.

□

Information Disclosure This threat occurs when an attacker is able to learn information that they should not be able to learn. It also includes attacks in which an attacker is able to cause sensitive information to be publicly released.

Information disclosure threats violate the security goal of **confidentiality**.

Example 2.7 (Information Disclosure Threats for Figure 2.13) • The attacker is able to steal the private key used to secure the users' connections to the university web site.

- A rogue administrator at the CDN accesses sensitive website media.

- An attacker is able to steal the password file for the website.

□

Denial of Service This threat occurs when an attacker is able to prevent the system from providing service to a legitimate user. Denial of Service threats are often much harder to stop than other kinds of threats.

Denial of Service threats violate the security goal of **availability**.

Example 2.8 (Denial of Service Threats for Figure 2.13) • The attacker commands a bot army to flood the website with traffic.

- The offsite CDN server goes down.

□

Elevation of Privilege This threat occurs when an attacker is able perform some action that the system's security policy does not allow. The most common form of this threat is when an attacker is able to gain "superuser" access (i.e. Administrator or root access) to a system, when logged-in as an ordinary user.

Elevation of Privilege threats violate the security goal of **access control**.

Example 2.9 (Elevation of Privilege Threats for Figure 2.13) • Somebody connects directly to the CDN server to directly access website content that would otherwise be password-protected.

- The web server does not perform access control checks, allowing one website user to access another user's content by issuing appropriate HTTP GET requests.

□

2.7.2 Attack Trees

Similar to fault trees, attack trees aim to deductively explore all of the ways in which a threat might be realised. The simple variant of attack trees that we consider here has only one type of node, representing some event that takes place.

Similarly to fault trees, the root node of the tree is the ultimate threat or event whose occurrence we are worried about. The attack tree answers the question of "how could this event occur"?

The children of each node in the tree are the immediate causes or ways in which the event that the node represents might be realised or might occur.

When a node has multiple children, they usually represent different ways in which an event can occur. In comparison to fault trees, they therefore represent an implicit OR node. However different attack tree variants can contain explicit AND and OR nodes, similarly to fault trees.

Given their similarity to fault trees, we do not discuss attack trees in detail here. Consult Shostack [4] for further details or see the various online resources on attack trees for more information.

Bibliography

- [1] Anthony Finkelstein and John Dowell. A comedy of errors: the London Ambulance Service case study. In *Proceedings of the 8th International Workshop on Software Specification and Design*, page 2. IEEE Computer Society, 1996.
- [2] J. McDermid, M. Nicholson, D. Pumfrey, and P. Fenelon. Experience with the application of HAZOP to computer-based systems. In *Proceedings of the Tenth Annual Conference on Computer Assurance*, pages 37–48. IEEE, 1995.
- [3] Peter Mellor. CAD: computer-aided disaster. *High Integrity Systems*, 1(2):101–156, 1994.
- [4] Adam Shostack. *Threat modeling: Designing for security*. John Wiley & Sons, 2014.
- [5] Z. Wei and W. Zhangjin. HAZOP introduction.

Failure modes and effects analysis (FMEA)

Project: 6/6/2008	Date:
FMEA Team: Sushil Cherian	Prepared by:

SEV = How severe is effect on the customer?
 OCC = How frequent is the cause likely to occur?
 DET = How probable is detection of cause?
RPN = Risk priority number in order to rank concerns; calculated as SEV x OCC x DET

Component	Potential failure mode	Potential failure effects	SEV	Potential causes	OCC	Current process controls	DET	RPN	Actions recommended	Responsibility (target date)	Actions taken	SEV	OCC	DET	RPN
What is the component?	In what ways can the component fail?	What is the impact on the customer if the failure is not prevented or corrected?	10	What causes the component to fail? (could the failure mode occur?)	10	What are the existing controls that either prevent the failure mode from occurring or detect it should it occur?	10	1000	What are the actions for reducing the occurrence of the failure mode? What actions should be taken on all high RPNs and on severity ratings of 9 or 10.	Who is responsible for the recommended actions? What should it be completed by?	What were the actions implemented? Include the date when completed (then recalculate resulting RPN).	10	10	10	1000
GPS															
PMU															
POC															
Communication Network															

Figure 2.12: An FMEA worksheet template

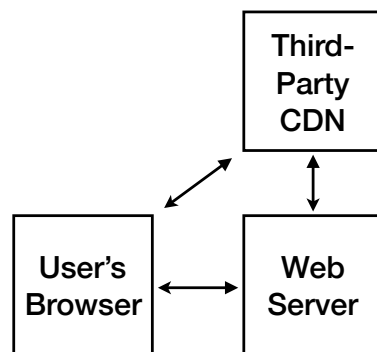


Figure 2.13: STRIDE Example System.

Chapter 3

Model-based specification

In this chapter, we will look at *model-based specification* — one style of formal specification. We will look at the *Alloy* language for specification, and how this can be used to model systems and check properties about the models. In the workshops, we will also look at a tool, called the *Alloy Analyser* for bounded checking of Alloy specifications.

This chapter currently covers Alloy 6, released in late 2021.

3.1 What is formal specification?

Formal specification is the use of mathematical notation to describe properties of a system. The use of mathematical notation provides precision that is not possible with natural language specification, which is subject to ambiguity and incompleteness. Formal specifications provide the specifier with a way to formally define *what* a system must do, without saying *how* it should do it. Thus, they are as formal as a programming language, but they do not constrain the design of the underlying algorithms or data structures.

Because formal specifications do not constrain design detail, and are independent of their implementation, they can be produced early in the project lifecycle; e.g. at the requirements and architectural design phases. At these phases, they are particularly valuable. As well as providing an unambiguous reference for system designers and implementers, the mathematical nature of formal specifications provides software engineers with the opportunity to *prove* properties about the specification, such as correctness, completeness, and consistency, but also other properties, including safety and security.

In this chapter, we will use the *Alloy* specification language [1]. Alloy is based on the Z language [4], which is the most widely used and probably most widely known formal specification language. Alloy sacrifices the expressiveness of Z for computational tractability; that is, some expressive features in Z are removed to allow us to automatically check properties of Alloy specifications.

In this subject, we use Alloy for two reasons:

1. It offers an intuitive notation.
2. The Alloy Analyser tool for checking Alloy specifications is a state-of-the-art research prototype, which has been applied to many industry systems, and is supported by a large community of users.

3.2 What is model-based specification?

Model-based specification is an approach to formal, system specification in which the system is defined as a state machine model. That is, there is a state representing the data in the system, and operations specify the transitions that can take the system from one state to another. This is much like the idea of a module/package in Ada: the variables define the state, and the subprograms define the transitions. The clean mapping from model-based specifications to modules is one reason that model-based specification has become the preferred method for formal specification, as opposed to axiomatic or algebraic approaches, which specify the properties of the systems via sets of axioms.

Model-based specification in the software engineering lifecycle. Model-based specification is not a requirements elicitation technique, nor is it generally a requirements engineering technique. Many model-based specifications are finalised during architectural design, once the components of the system are known. However, the formalisation of the behaviour of system components implicitly leads to a formalisation of those system requirements that the components are implement. As such, model-based specification falls over the phases of requirements engineering and architectural design. We should be comfortable with this, as in modern software engineering, the line between requirements and architectural design is blurred.

Figure 3.1, adopted from Sommerville [3, Ch. 27] shows the relationship between different activities in the early software engineering phases, and how model-based specification is typically carried out in parallel with other activities. Information flow between the activities is two-way — information is always passed between the requirements and design phases.

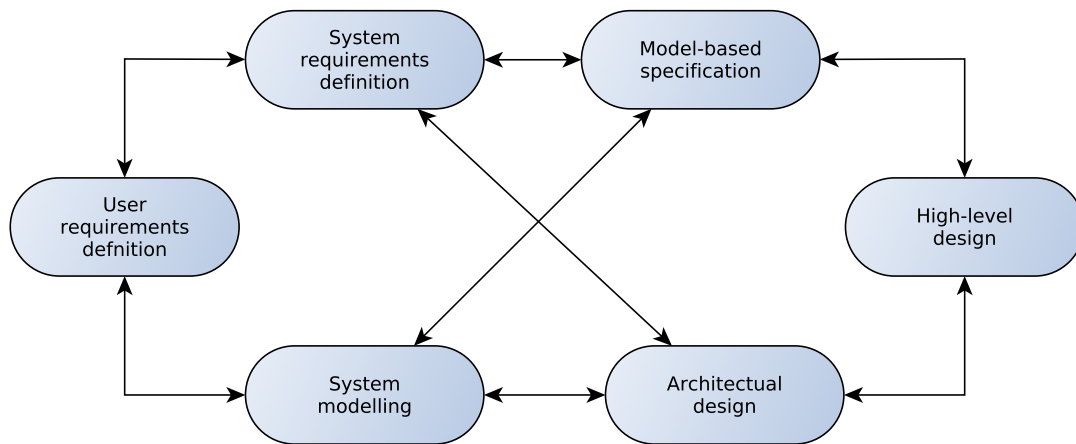


Figure 3.1: Model-based specification in the software process.

Languages for model-based specification. There are several well-known languages for model-based specification, including VDM, B, Z, and Alloy. These languages all have commonalities, especially that data types are based on sets and relations, and operations are specified using predicate logic. UML statecharts are another example of model-based specification, although the level of formality in these is less than in VDM, B, Z, and Alloy, as the intended areas of use are different.

Model-based specification and natural language. Like programming languages, formal, model-based specifications cannot be understood fully by a simple read over. As such, documenting the specification using natural language is important, just like commenting program code. Natural language descriptions not only allow the reader to understand the specification at a superficial level without having to worry about detail, but also allow the reader to understand the formal details, by guiding them through the meaning.

3.3 The costs of formal specification

Using a rigorous process and method for formally specifying system properties can clearly be a costly exercise. The activity of specifying the precise detail of systems and sub-systems using mathematical notation requires careful thought and increased knowledge about the systems. Further, they must come under the same scrutiny of quality assurance using reviews and inspections as other requirements specification techniques.

The question to be asked is: “*Is the cost worth it?*”.

A major limitation to people using formal specification, even in high integrity domains, is that the perceived cost is too high. However, studies repeatedly show the following: the cost of specification is higher, but some of the added costs are recouped later in the process. Figure 3.2, taken from Phillips [2], shows data from a project, comparing the number of errors found per thousand lines of code of using the Z specification language, against not using formal specification at all. The project from which the data comes from is the CICS project undertaken at IBM. The project introduced 268,000 lines of new code into a system already containing 500,000 lines of code. Of the new code, 48,000 lines was engineered from a formal specification written in Z, and Z was used at the product-, component-, and module-level design stages. Others studies show similar results.

Let’s analyse this figure and ask ourselves why formal specification would *reduce* the costs of later phases. There are a number of reasons for this:

1. Using mathematical notation removes ambiguity from specifications, which is a cause of many problems in software. This helps to limit the amount of re-work in later phases, driving down their costs. In high-integrity systems, requirements typically change much less frequently than in say, enterprise systems, so the models do not require major changes every increment.
2. Some cost incurred during design and implementation is simply trying to decide exactly what the behaviour of the system could be. Using natural language to specify requirements means that we can be shorthanded and therefore miss or ignore important detail. Using mathematical notation does not afford this because it is easy to show incompleteness, and many of the issues that generally present themselves during design and implementation must be solved during specification instead. In other words, the people doing the specification do some of the work typically undertaken at the design and implementation phases.
3. Despite opinion to the contrary, designing and implementing a system from a formal specification is *more* straightforward than doing so from natural language requirements. For a novice, this may not be the case, but for someone who is as comfortable with mathematical notation as they are with programming languages, this is most definitely the case.
4. Using formal notation offers the possibility of *proving* that our specification satisfied certain properties. Discovering in later phases that our specification does not satisfy some properties is more

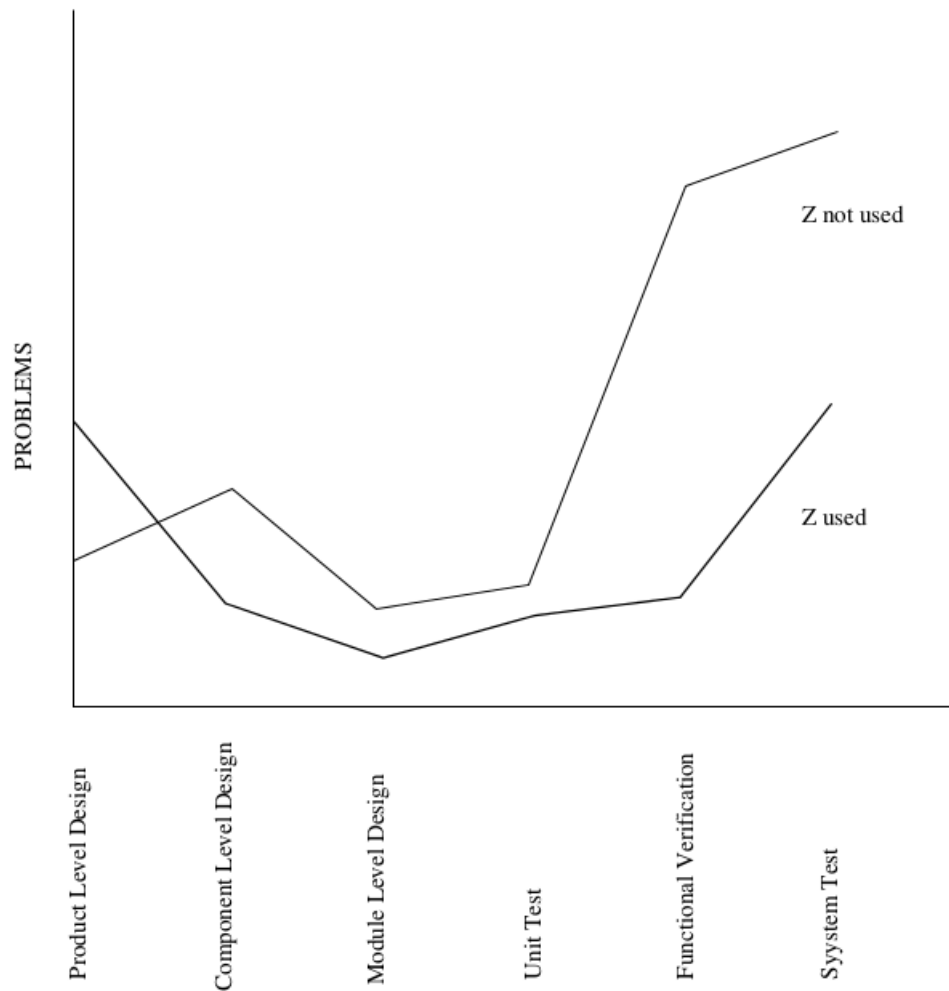


Figure 3.2: The costs of formal specification on the CICS project.

costly than discovering these in early phases.

5. Post-release costs are reduced due to the fact that less mistakes are introduced over the process; or more accurately, the same number of mistakes are made, but more are made *and found* during specification, rather than post release.

A simple (and common) objection to formal specification is that it is just too hard: the formality is too difficult for the average programmer to grasp. I agree that this is case. I would not advocate employing formal specification on a project with programmers who have no experience at it. However, I disagree that this *has to be* the case. The average programmer is already intelligent enough to have learnt a formal language: all programming languages are formal. As such, we know they are equipped to deal with formal languages, even at the specification language.

However, an important extension to this argument is the following: *formal specification is much simpler than programming* — *we only have to specify what a system should do, whereas our program must specify how to do it.*

If programmers were trained in formal specification languages the same way they were trained in pro-

programming languages, reading and using formal specifications would not be an issue. Thus, the cost of formal specification would be *significantly* less than the cost of implementation – although its *value* may not be.

That is not to say that formal specification should be used in every organisation or on every project: it is only cost effective when the cost of releasing software with faults is greater than the cost of preventing them. In environments where the biggest issue is responding to change (e.g. change of customer preferences, change of requirements from a client), ensuring the software is correct is not as high of a priority as getting something out that works for most normal cases.

In high integrity systems, formal specification is generally only applied to those parts of the system that contain high-integrity requirements. Other parts of the system, for example, data storage or user interface, are engineered using more standard techniques.

3.4 Logic and set theory

Most formal model-based specification languages are based on *first-order predicate logic* and *set theory*; both topics with which all students in this subject are expected to be familiar. The set theory part of Alloy contains the standard set theoretic concepts, such as set comprehension, power sets, set operators (union, intersection, subset, etc.), Cartesian products, relations, and functions. The predicate logic part of Alloy contains propositional operators (“and”, “or”, “implies”, etc.) and existential and universal quantifiers (“for all”, “there exists”).

The most important concepts with which you are expected to be familiar are:

- First-order logic: conjunction (“and”), disjunction (“or”), implication, negation, universal quantification (“for all”), and existential quantification (“there exists”).
- Set theory: set membership, set comprehension, set union, set intersection, set difference, and subsets.
- Relations: Cartesian product, domain and range of relations, sequences, partial functions, and total functions.

If you are not familiar these concepts, or the details are hazy, Chapter 3 of Daniel Jackson’s book *Software Abstractions: Logic, Language, and Analysis* [1], provides a detail description of these concepts, and how to express them in Alloy.

If you cannot access this, Chapters 2-9 in Woodcock and Davies book “Using Z” [4] provides a detailed description of basic logic and set theory, which can also be read lightly to pick up the most important aspects. A PDF version of this book is available on the LMS for download.

Why sets and predicates?

Most model-based languages are based on sets and predicates. The reasons are quite simple:

1. Set theory (sets, relations, functions, etc.) provides us with enough notation to specify the data of any system. Just as importantly though, it *restricts* us from specifying design detail, and forces us to think more abstractly — something which is important at the specification level.

2. Predicate logic provides us with sufficient expressive power to specify what changes the data of a system can undergo over time (or the effect of operations on data). Just as importantly, it *restricts* us from specifying *how* those changes should be implemented.

These first two points are important: requirements engineering is already difficult enough without us thinking about design and implementation problems at the same time.

3. Together, proof theory and understanding of set theory and predicate logic is strong enough that we can prove many properties specified using these parts. We can take advantage of hundreds of years of work in these areas and apply this to software engineering — a field that is relatively new in comparison.

Example 3.1

As an example of using logic and set theory to specify software and system behaviour, and how this differs from programming, consider the trivial example of sorting a list of integers.

To specify the behaviour of a procedure that takes a (possibly unsorted) list and sorts it, we have to say that:

1. each element in the input list is present in the output list (that is, the output list is a permutation of the input list); and
2. each element of the output list is less than or equal to the next (that is, the list is sorted).

We can specify what it means for a list of integers to be sorted:

$$\forall list : \text{seq } \mathbb{Z} \bullet \text{sorted}(list) \Leftrightarrow \forall i : 1 \dots \#list - 1 \bullet list(i) \leq list(i + 1)$$

This states that, for all sequences of integers, that sequence is sorted *if and only if*: for each index (except the last) in the list, the integer at that index is less than or equal to the integer at the next index.

We can also specify what it means for one list to be a permutation of the other.

$$\forall a, b : \text{seq } \mathbb{Z} \bullet \text{permutation}(a, b) \Leftrightarrow \forall el : \mathbb{Z} \bullet \text{count}(el, a) = \text{count}(el, b)$$

in which *count* is defined as:

$$\forall el : \mathbb{Z}; list : \text{seq } \mathbb{Z}; c : \mathbb{N} \bullet \text{count}(el, list) = c \Leftrightarrow c = \#(list \triangleright \{el\})$$

That is $\text{count}(el, list) = c$ if and only if c is the number of elements when we take the *range restriction* of *list* on *el* only.

Finally, we can specify the precondition and postcondition of a procedure using these concepts:

procedure *sort*(*in_list* : **in** seq $\mathbb{Z}$; *out_list* : **out** seq $\mathbb{Z}$) **is**
pre $\#in_list > 1$
post $\text{sorted}(out_list) \wedge \text{ispermutation}(in_list, out_list)$

Here, we have added a superficial precondition that the list is not empty.

So, this example specifies the functional behaviour of sorting a list, but does NOT specify *how* the list should be sorted. Any correct algorithm for sorting integers will suffice to fulfil the specification above.

□

Example 3.2

Using the definitions above, we can specify the behaviour of a binary search algorithm in a straightforward manner:

procedure *binary_search*(*list* : **in** seq $\mathbb{Z}$; *target* : **in** $\mathbb{Z}$; *index* : **out** $\mathbb{Z}$) **is**
 pre *sorted*(*list*)
 post *target* = *list*(*index*) $\vee$ (*target* = -1 $\Leftrightarrow$ *target* $\notin$ ran(*list*))

In this example, we have called the procedure “*binary_search*”, but only because we specify in the precondition that the input list must be sorted. Other than this, the algorithm is not constrained by any particular binary search algorithm (e.g. recursive or iterative), and in fact, does not truly constrain it to a binary search algorithm at all.

We can see from this example: specifying binary search behaviour is much neater than specifying the algorithm to achieve it.

□

3.5 Introduction to Alloy

In this chapter, we will present the Alloy language. We will present only a small subset of the language, but it is enough to allow specification of and reasoning about abstract state machines. The Alloy has a rich notation for combining predicates, which allow flexible and incremental building of specifications. We will not go into detail on all of these operators, but we will use some of the basic components to enable clean specification.

An Alloy reference manual is available online at:

<https://alloytools.org/spec.html>

It provides detailed syntax and semantic definitions.

3.5.1 Key characteristics

There are a few key ideas to Alloy that are worth noting.

Alloy = logic + language + analysis

The complete Alloy framework consists of three parts:

1. *Logic*: This consists of first-order predicate logic, plus a simple relational calculus.
2. *Language*: This consists of a handful of operators for structuring specifications, such as specifying types, predicates, functions, and assertions.
3. *Analysis*: This performs an exhaustive search (in bounded domains) for counterexamples using SAT (satisfiability check).

Non-specialised logic

The logic of Alloy is not tied to a particular paradigm, such as concurrent modelling or state machines. However, as of Alloy 6, it has a number of useful features for modelling state machines. In this subject, we will focus primarily on using Alloy as a model-based language for specifying state machines.

Not proof, but counterexamples

The focus of the Alloy Analyser is not to *prove* a model has a particular property, but instead is to find counterexamples that demonstrate the property does not hold. Finding such a counterexample demonstrates that the property does not hold, while failing to find one demonstrates that the property *may* hold.

In particular, Alloy encourages the use of small, finite domains on which to do analysis. That is, rather than search for a counterexample in a large or finite set of objects, it encourages (at least during initial analysis) to use small domains of only a few objects; e.g. only three usernames, URLs, and passwords. This approach has two advantages:

1. Using small domains means the analysis is much faster, as there are less objects to check.
2. Most assertions are (initially) wrong, and small counterexamples to them can be found. Having small counterexamples means understanding the counterexample itself is more straightforward.

The aim of the Alloy Analyser though is not to search a handful of arbitrary “test” cases in a small domain, but search for *all* possible cases in that domain. The real domain may be infinite, but a small, finite set representing that domain will find *almost* all flaws.

3.6 Alloy Logic

In this section, we present the logic used in Alloy, which is based on predicate logic and relational calculus.

We will use the example of a password management system, such as *LastPass*, which remembers the username and password combinations for particular websites (represented by their URL).

3.6.1 Everything is a relation

Alloy uses relations for all data types, including scalars and sets. Sets are just unary relations; e.g.:

```
Username = { (U0), (U1), (U2) }  
URL = { (UR0), (UR1), (UR2) }  
Password = { (P0), (P1), (P2) }
```

scalars are just singleton sets:

```
myUsername = { (U0) }  
myPassword = { (P1) }
```

and relations are just sets of tuples:

```
passwords = {(U0, P1), (U1, P2)}
```

This above represents a mapping of usernames to passwords, in which username U0 maps to password P1. An alternative could be to have a password for a particular URL:

```
urlPasswords = {(U0, UR0, P1), (U0, UR1, P2), (U1, UR0, P2)}
```

Here, user U0 has two passwords for two different URLs (UR0 and UR1), while user U1 has only one password.

An important restriction of relations in Alloy is that *all relations are first-order*; that is, we cannot have sets of sets, or relations of relations. If using a set of sets is a natural way to represent a problem, we instead need to find another representation. This restrictions are is what permits the Alloy language to be (relatively) easily analysed by automated tools.

3.6.2 Operators

Set operators Sets (represented as unary relations) are a commonly-used data type in formal modelling. In Alloy, the main set operators are included in the logic:

Operator	Meaning	Example
+	union	$\{(U0)\} + \{(U1)\}$
&	intersection	$\{(U0)\} \& \{(U1)\}$
-	difference	$\{(U0), (U1)\} - \{(U1)\}$
in	subset	$\{(U1)\} \text{ in } \{(U0), (U1)\}$
=	equality	$\{(U0), (U1)\} = \{(U0), (U1)\}$

Cross product A cross product, represented using the operator $\times$ in set theory, is represented as the following in Alloy:

Operator	Meaning	Example
->	cross product	Username->Password

As an example, consider the cross products taken from URLs, usernames, and passwords:

```
Username = {(U0, U1, U2)}
URL = {(UR0, UR1, UR2)}
Password = {(P0, P1, P2)}
Username->Password = {(U0, P0), (U0, P1), (U0, P2),
                      (U1, P0), (U1, P1), (U1, P2),
                      (U2, P0), (U2, P1), (U2, P2)}
Username->URL->Password = {(U0, UR0, P0), (U0, UR0, P1), (U0, UR0, P2),
                          (U0, UR1, P0), (U0, UR1, P1), (U0, UR1, P2),
                          ...
                          (U2, UR2, P0), (U2, UR2, P1), (U2, UR2, P2)}
```

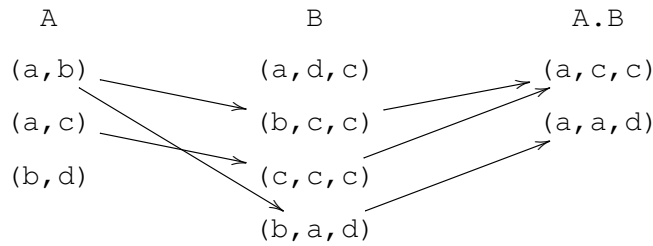
Join operators There are two operators for joining relations in Alloy:

Operator	Meaning	Example
.	dot join	URL.passwords
[]	box join	(Username->Password) [myUsername]

These operators are related via the following properties:

```
a.b = b[a]
a.b.c[d] = d.(a.b.c)
```

A dot join is a relational join, which maps the range of the relation on the left to the domain of the relation on the right. For example:



For the relations:

```
myUsername = { (U0) }
myUrl = { (UR1) }
urlPasswords = { (U0, UR0, P1), (U0, UR1, P2), (U1, UR0, P2) }
```

the following hold:

```
myUsername.urlPasswords = { (UR0, P1), (UR1, P2) }
myUsername.urlPasswords[myUrl] = { (P2) }
```

which matches the U0 in myUsername to the same value in urlPasswords.

Restriction and override The concepts of domain restriction, range restriction, and function override are represented as follows:

Operator	Meaning	Example
<:	domain restriction	urlPasswords <: myUrl
:>	range restriction	urlPasswords :> Password
++	function override	urlPasswords ++ (myUsername->myUrl->newPassword)

Function override is defined as: $A ++ B = (A - (\text{domain}[B] <: A) + B)$.

Informally, domain restriction, $A <: B$ takes the relation A and restricts it to contain only the tuples whose domain is in B, while range restriction, $A :> C$ restricts A to contain only the tuples whose range is in C. For example, if we have:

```
myUsername = { (U0) }
myUrl = { (UR1) }
someUsernames = { (U0), (U1) }
somePasswords = { (P1), (P3) }
newPassword = { (P4) }
```

```
urlPasswords = {(U0, UR0, P1), (U0, UR1, P2), (U1, UR0, P2), (U2, UR3, P3)}
```

then the following hold:

```
urlPasswords <: someUsernames = {(U0, UR0, P1), (U0, UR1, P2), (U1, UR0, P2)}
urlPasswords :> somePasswords = {(U0, UR0, P1), (U2, UR3, P3)}
```

For override, we supply at least one new tuple, which is added to the relation, but with the further constraint that it *overrides* the value of any existing tuple with the same domain; e.g.:

```
urlPasswords ++ (myUserName->myUrl->newPassword) =
  {(U0, UR0, P1), (U0, UR1, P4), (U1, UR0, P2), (U2, UR3, P3)}
```

Propositional logic operators

Operator	Alternative Operator	Meaning
!	not	negation
&&	and	conjunction
	or	disjunction
=>	implies	implication
<=>	iff	if-and-only-if

Predicate logic quantifiers Assuming that x is a variable, e is a set expression, and F is a formula, the following are valid formula:

Operator	Meaning
all $x : e \mid F$	F holds for every x in e
no $x : e \mid F$	F holds for no x in e
some $x : e \mid F$	F holds for at least one x in e
lone $x : e \mid F$	F holds for at most one x in e
one $x : e \mid F$	F holds for exactly one x in e

For example, the following two predicates specify that: (1) there is at least one password for the user corresponding to `myUsername`; and (2) no username has more than one password:

```
(1) some pwd : Password | (myUsername->pwd) in passwords
(2) all user : Username | lone pwd : Password | pwd in user.passwords
```

The last four of these keywords can also be applied to *relations*:

Operator	Meaning
no e	e has no elements/tuples
some e	e has at least one element/tuple
lone e	e has at most one element/tuple
one e	e has exactly one element/tuple

For example, we can re-write the two examples above as:

```
(1) some myUsername.passwords
(2) all user : Username | lone user.passwords
```

Set declarations In predicates and other constructs that use sets, modifiers are used to restrict the elements in declarations:

Operator	Meaning
$x : \mathbf{set} \ e$	x is a subset of e
$x : \mathbf{one} \ e$	x is a singleton subset of e
$x : \mathbf{lone} \ e$	x is an empty or singleton subset of e
$x : \mathbf{some} \ e$	x is a non-empty subset of e
$x : e$	$x : \mathbf{one} \ e$

Set comprehensions New sets can be constructed from expressions:

Operator	Meaning
$\{x : e, y : f \mid F\}$	A set of tuples that satisfy formula F

We can construct a set of usernames that have the password `myPassword`:

```
{user : Username | user in password.myPassword}
```

Cardinalities

Operator	Meaning
$\#e$	The number of tuples in expression e
$0, 1, 2$	Integer literals
$+, -$	Addition, subtraction
$=, <=, <, >=, >$	Arithmetic operators

Note that there is no multiplication or division! Multiplication and division are not included in the language because SAT solvers are typically unable to deal with them in the general case.

let formula and expressions A **let** expression is a shorthand:

Operator	Meaning
let $x = e \mid \text{formula}$	
let $x = e \mid \text{expression}$	

For example:

```
all user : Username | let p = user.passwords | lone p && some p
```

3.6.3 Temporal Logic: specifying systems that change over time

As of Alloy 6, released in late 2021, Alloy's logic now includes *temporal operators* that allow one to talk not only about whether something might be true in a single state but also in future or even past states. These changes add features to Alloy's logic that do not make it less expressive but do make it easier to specify state machines and, in particular, transitions between states and properties that need to hold at certain times.

Next State The `'` operator is used to evaluate an expression in the next state. For instance, we can specify that the value of `user.passwords` does not change between the current state and the next.

```
user.passwords' = user.passwords
```

Temporal Predicate Operators Given predicates P and Q , the temporal operators allow us to talk about whether they hold in certain states before or after the current one. Specifically, a series of states of our model is called a *trace*. Given a trace, suppose the current state is the i th state in the trace. Then we define what it means for each of the temporal operators to hold in state i as follows:

Operator	Meaning
always P	P holds in all states $\geq i$
after P	P holds in the $i + 1$ th state
eventually P	P holds in some j th state where $j \geq i$
P until Q	Q holds in some state $j \geq i$, and P holds for every k th state such that $i \leq k < j$
P releases Q	Q is true in every state $\geq i$ up to and including a state k in which P is true, or there is no such k in which case Q holds in any state $\geq i$
P ; Q	P is true in the i th state and Q is true in the $i + 1$ th state

For instance we can specify that in the next state `user.passwords` is empty:

```
after (no user.passwords)
```

3.7 Alloy Language

The Alloy language provides constructs to structuring specifications using the logic presented in the previous section.

3.7.1 Modules

Alloy modules are analogous to modules in programming languages, so are used to model self-contained specifications that can be imported into other modules and re-used.

3.7.2 Signatures

Signatures are essentially type declarations:

Operator	Meaning
sig A {}	Declare a set of atoms called A
sig B {}	Declare a set of atoms called B, which is disjoint from A
sig A, B {}	As above
sig B extends A {}	Declare sets A and B where B a subset of A ($B \subseteq A$)
abstract sig A {}	An <i>abstract</i> set of atoms
sig B, C extends A {}	B and C are disjoint, and $(B+C) \subseteq A$
sig A {f : e}	f is a binary relation of type $f : A \rightarrow \text{one } e$

The **var** keyword can be applied to signatures or to fields to specify things that can change over time.

For example we can declare a set A of atoms as a variable set: **var sig** A {}. Doing so means that the atoms contained in A in the current state might not be contained in A in future states, and vice-versa, i.e. atoms from A can appear and disappear over time, depending on how we constrain the transitions of our model.

The **var** keyword is more often applied to signature fields, to model fields whose values can change over time.

For example, we can construct types for keeping track of passwords at particular URLs:

```
sig URL {}
sig Username {}
sig Password {}
sig PassBook {var password : Username -> URL -> Password}
```

Here, PassBook models a set of “known” username and URL combinations, and the passwords for these. There is, at most, one password per pair (passwords is a function).

Note that if A is a non-var set, then we cannot write **var sig** B **extends** A {}, i.e. variable signatures cannot extend non-variable ones. However, non-variable sigs can extend variable ones.

3.7.3 Facts

Facts are (possibly named) constraints that are assumed to hold in all initial states. They are introduced using the **fact** keyword.

Operator	Meaning
fact f { F }	Declare a fact called f that introduces a formula F.

For example, we can express that a PassBook can have at most one password for each user/URL pair:

```
fact NoDuplicates
{
  always all pb : PassBook, user : Username, url : URL | lone pb.password[user][url]
```

```
}
```

Note that this fact uses the `always` keyword to say that it holds not only in the initial state, but in all subsequent states too (e.g. after certain operations might be performed on the PassBook).

Such facts can be used to model *invariants* over the data types: constraints that always hold for the data types.

3.7.4 Predicates

Predicates are named formula with declared parameters. Predicates can be accessed from within other predicates.

Operator	Meaning
pred $p \ [x : e] \ \{ F \}$	Declare a predicate called p , which has variable(s) x , and formula F

In this subject, we will primarily be using predicates to introduce *operations* over a state. That is, a set of signatures will define the state space, and predicates will define the possible transitions over that state space.

For example, we can have operations that add and delete passwords to a PassBook:

```
//Add a password for a new user/url pair
pred add [pb : PassBook, url : URL, user: Username, pwd: Password] {
    //no pb.password[user][url]
    pb.password' = pb.password + (user->url->pwd)
}

//Delete an existing password
pred delete [pb : PassBook, url : URL, user: Username] {
    one pb.password[user][url]
    pb.password' = pb.password - (user->url->Password)
}
```

Notice how we use `pb.password'` to refer to the password mapping in the *next* state after these operations have run.

The `add` predicate specifies an operation over a PassBook `pb`, and three input variables `user`, `url`, and `password`.

The first line of the predicate part states that there is not already a password for this user/URL pair (the set formed by `pb.password[user][url]` is empty). This is a *precondition*: the behaviour of the operation is not defined if this doesn't hold. A precondition specifies an assumption made about the initial state from which an operation occurs. We refer to that state as the *pre-state*.

The second line is the *postcondition*, which specifies what transition should happen if the precondition holds. In this case, a union (+) is used with the ' symbol to specify that in the subsequent state `pb.password` is the same as in the pre-state, but with the new details added. The subsequent state reached after an operation is performed is called the *post-state*, so `pb.password'` refers to the value of `pb.password` in the post-state.

The `delete` predicate specifies an operation that deletes a password from the PassBook. The precondition requires that a password exists, and the postcondition uses set difference to remove the tuple from

the relation of passwords.

Predicates can be accessed within other predicates. For example, we could break the precondition of the `add` predicate into its own predicate, and reference this:

```
//True if and only if the user has no password for this URL
pred addPre [pb : PassBook, url : URL, user: Username] {
    no pb.password[user][url]
}

//Add a password for a new user/URL pair
pred add [pb, url : URL, user: Username, pwd: Password] {
    addPre [pb, url, user]
    pb.password' = pb.password + (user->url->pwd)
}
```

The semantics of the `add` operation is exactly as if the body of `addPre` was included where it was referenced, with the appropriate variable renamings.

Note that this is not sequence composition. That is, `addPre` is not “executed”, followed by the line after it. The formulae in `addPre` and `add` are evaluated as one larger conjunction.

3.7.5 Functions

Functions are named expressions with declared parameters and expressions. They can be ‘invoked’ by predicates or other functions by providing an expression for each parameter.

Operator	Meaning
fun <i>f</i> [<i>x</i> : <i>e1</i>] : <i>e2</i> { <i>F</i> }	Declare a function called <i>f</i> , which has variable(s) <i>x</i> , formula <i>F</i> , and is of type <i>e2</i>

For example, we can lookup the password for a given user/URL pair:

```
//Return the password for a given user/URL pair
fun lookup [pb: PassBook, url : URL, user : Username] : lone Password {
    pb.password[user][url]
}
```

Note the expression/type given to the function: (**lone** Password). This function can be used as an expression as part of another formula. For example:

```
//Check if a user's passwords for two urls are the same
pred samePassword [pb : PassBook, url1, url2 : URL, user : Username] {
    lookup [pb, url1, user] = lookup [pb, url2, user]
}
```

3.7.6 Assertions

Assertions are named constraints that are intended to follow from a model. We can express assertions using formula using:

Operator	Meaning
assert a { F }	Declare an assertion called a, with the formula F

For example, we can express the following assertion which says that the add predicate “works”:

```
//If we add a new password, then we get this password when we look it up
assert addWorks {
  all pb : PassBook, url : URL, user : Username, p : Password |
    add [pb, url, user, p] => (after (lookup [pb, url, user]) = p)
}
```

Notice how we use the `after` operator to talk about what should be true in the state after `add` has occurred (i.e. in the post-state).

Assertions can be checked by the Alloy Analyser (see Section 3.8).

Typically, assertions play two roles: (1) to express some properties that we should hold on the model, which we can then check to see if there are flaws in our model (or flaws in our assertions!); and (2) to express essential properties of the model that are higher than facts or properties; e.g. relationship between predicates; which give us a better understanding of the model itself, and possible ways to refine or restructure it.

3.7.7 The complete LastPass example

```
module LastPass

/*
 * LastPass password map
 *
 * A simple example to explain basics of Alloy.
 *
 * The 'PassBook' keeps track of a set of users' passwords for a set of URLs.
 * For each User/URL pair, there is one password. Passwords can be added,
 * deleted, looked up, and changed.
 * A user can also request all of their password for all URLs.
 *
 * author: Tim Miller; updated by Toby Murray for Alloy 6, including simplifications and fixes
 */

sig URL {}
sig Username {}
sig Password {}
sig PassBook {var password : Username -> URL -> Password}

fact NoDuplicates
{
  always all pb : PassBook, user : Username, url : URL | lone pb.password[user][url]
}

//Add a password for a new user/url pair
pred add [pb : PassBook, url : URL, user: Username, pwd: Password] {
  //no pb.password[user][url]
  pb.password' = pb.password + (user->url->pwd)
}
```

```

//Delete an existing password
pred delete [pb : PassBook, url : URL, user: Username] {
    one pb.password[user][url]
    pb.password' = pb.password - (user->url->Password)
}

//Update an existing password
pred update [pb : PassBook, url : URL, user: Username, pwd: Password] {
    one pb.password[user][url]
    pb.password' = pb.password ++ (user->url->pwd)
}

//Return the password for a given user/URL pair
fun lookup [pb: PassBook, url : URL, user : Username] : lone Password {
    pb.password[user][url]
}

//Check if a user's passwords for two urls are the same
pred samePassword [pb : PassBook, url1, url2 : URL, user : Username] {
    lookup [pb, url1, user] = lookup [pb, url2, user]
}

//Retrieve all of the passwords and the url they are for, for a given user
pred retrieveAll [pb: PassBook, user : Username, pwds : URL -> Password] {
    pwds = (pb.password)[user]
}

//Initialise the PassBook to be empty
pred init [pb: PassBook] {
    no pb.password
}

//If we add a new password, then we get this password when we look it up
assert addWorks {
    all pb : PassBook, url : URL, user : Username, p : Password |
        add [pb, url, user, p] => (after (lookup [pb, url, user]) = p)
}

//If we update an existing password, then we get this password when we look it up
assert updateWorks {
    all pb : PassBook, url : URL, user : Username, p2 : Password |
        (update [pb, url, user, p2] => (after (lookup [pb, url, user]) = p2))
}

//If we add and then delete a password, we are back to 'the start'
assert deleteIsUndo {
    all pb : PassBook, url : URL, user : Username, pwd : Password |
        (add [pb, url, user, pwd] ; delete [pb, url, user])
        => pb.password = pb.password''
}

run add for 2 but 1 PassBook
run add for 5 URL, 5 Username, 10 Password, 1 PassBook
check addWorks for 3 but 1 PassBook expect 0
check updateWorks for 7 but 1 PassBook, 3 steps expect 0
check deleteIsUndo for 1 but 1 PassBook

```

3.8 Analysis with the Alloy Analyser

The Alloy language supports two types of construct that are related specifically to the Alloy Analyser. That is, they say nothing about the model itself, but are merely there to tell the analyser what to check, and to provide some parameters for this checking.

In these notes, we briefly discuss the main concepts in analysing Alloy models, and we will explore this more in the workshops.

3.8.1 Runs

The **run** command instructs the analyser to search for instances of values that satisfy a predicate:

Operator	Meaning
run <i>p</i> <i>scope</i>	Find a valid binding of variables for predicate or function <i>p</i> , such that these values satisfy <i>F</i> , using the declared <i>scope</i> .

For example, given the operation `add` defined in the previous section, we can check that the `add` is *satisfiable* in a domain of size three using:

```
run add for 3
```

This instructs the analyser to search for values for the variables, assuming that there are three instances of each signature in the universe; that is, three usernames, three URLs, three passwords, and three passbooks. We can restrict different variables to have different sizes. For instance, `add` talks only about one Passbook `pb`, so it makes sense to restrict the size of this set to just 1 when running the predicate.

```
run add for 3 but 1 PassBook
```

As well as bounding the size of signatures, we can also bound the number of transitions that Alloy explores. For instance, `add` talks only about a pre-state and a post-state, i.e. only one transition. Therefore it makes sense to run it for just a transition between two states:

```
run add for 3 but 1 PassBook, 1..2 steps
```

This specifies that the domain should be of size three, except only one PassBook, and only two states (state 1 and 2), i.e. one transition.

If an instance can be found, this means that the predicate/function is indeed satisfiable. If an instance *cannot* be found, this means predicate/function is *possibly* unsatisfiable. It may be that the scope we used is simply not large enough to find a combination that satisfies the formula.

The idea of scope is discussed in more detail in Section 3.8.3.

3.8.2 Checks

The **check** command instructs the analyser to search for instances of values that *violate* an assertion — a *counterexample* to the assertion:

Operator	Meaning
check a scope	Find a counterexample to the assertion a, such that these values satisfy not F, using the declared scope.

For example, given the assertion `addWorks` defined in the previous section, we can attempt to find a counterexample to the assertion in a domain of size three using:

```
check addWorks for 3
```

If a counterexample can be found, this means that the assertion does not hold on the model; i.e. the assertion is incorrect. If a counterexample cannot be found, then the assertion *may* hold, or there may be a counterexample that exists outside of the specified scope.

3.8.3 Scope

The idea of scope is straightforward. The Alloy Analyser does *bounded* checking, meaning that it can (potentially) try every combination of assignments of instances to variables – although the searching algorithms are much more sophisticated and eliminated many infeasible combinations early on. For models that use the `var` keyword, the scope also includes the number of transitions in the model to search up to.

The scope can be modified to increase or decrease the number of instances that are used in the search. For example, we can use the following:

```
run add
run add for 5
run add for 5 but 3 PassBook
run add for 5 URL, 5 Username, 10 Password, 2 PassBook
run add for 5 but 1..2 steps
```

The first line omits a scope, in which case the default scope of three is used. When the scope doesn't include a bound on the `steps` the default is to check for 1..10 steps. The second command specifies that five instances of all signatures should be used, while the third says the same, except only create three signatures for `PassBook`. The fourth one explicitly lists the number of instances for each signature type.

The final one also specified a bound on the number of steps (transitions to search). Perhaps confusingly, 1 `steps` refers to 0 transitions, i.e. to traces that contain a single state. In general `k steps` means traces containing `k` states and `k-1` transitions. Therefore “1..2 `steps`” means traces with a single transition. One can of course search for instances of `add` within longer traces but doing so doesn't make much sense for this model, since `add` only talks about two states (one transition).

There are several other ways to restrict the scope used in analysis. See Jackson's book on Alloy [1] for information on these.

The small scope hypothesis

By finding instances and counterexamples to predicates, functions, and assertions, we are essentially just doing exhaustive testing using a bounded set of instances. As such, we cannot *prove* that an assertion

holds for all instances in the universe. This is essentially *like* testing, but in actually, it is much more powerful.

The Alloy philosophy relies on the *small scope hypothesis*, which is:

Most faults have small counterexamples.

That is, if an assertion is incorrect, it *often, but not always*, has a counterexample that exists in a small scope of around 3-5 instances per signature and a handful of transitions (around 10). The Alloy Analyser exhaustively analyses all cases within the declared scope, therefore, if a counterexample exists within that scope, it will be found. This is in contrast to testing, which will select a small subset of the scope as test inputs.

Using larger scopes increases the chance of finding a counterexample of course, but also takes more CPU time and memory to analyse. A good strategy is to start with a small scope of three and a few transitions, which is usually sufficient to find flaws in our models early on, eliminate these flaws, and once all of our runs and checks pass, extend the scope to be much larger, and leave the analyser to search for a period while we go for a coffee.

3.9 Abstraction and incremental specification of state machines using Alloy operators

In the previous section, we saw how to create operation predicates from smaller operation predicates; e.g. by specifying the precondition of the `add` operation in its own predicate `addPre` and referencing this. Using predicates as building blocks in this way is an excellent abstraction mechanism that can be used to our advantage in specifying systems.

The formal specification process is not as simple as sitting down and writing it. As with programming, specifications should be put together incrementally. Abstraction mechanisms can be used to perform this top down, allowing us to specify greater detail once we have a better understanding of the problem at hand.

3.9.1 The specification process

Figure 3.3 outlines a process model for defining a formal specification. Clearly, this model is idealised, and at any step, there is a likelihood of returning to any of the earlier steps; however, it serves well as a model. The problem is tackled top down. Given a set of informal requirements, we define the system components and specify them. After this, we bring all components together. To specify a single component in isolation, we first consider the data types, and then the state, init operation, and normal operations. This is a specification of how the components works in an ideal world. After this, we add the exceptional behaviour. The total operations are then composed from the normal and exceptional behaviour operations. Finally, the component specifications are composed into a system specification.

State machines

In these notes, we focus on the use of Alloy to specify and analyse abstract state machines: that is, an abstract model containing a *state* (e.g. `PassBook`) and *transitions* between these states. The `LastPass`

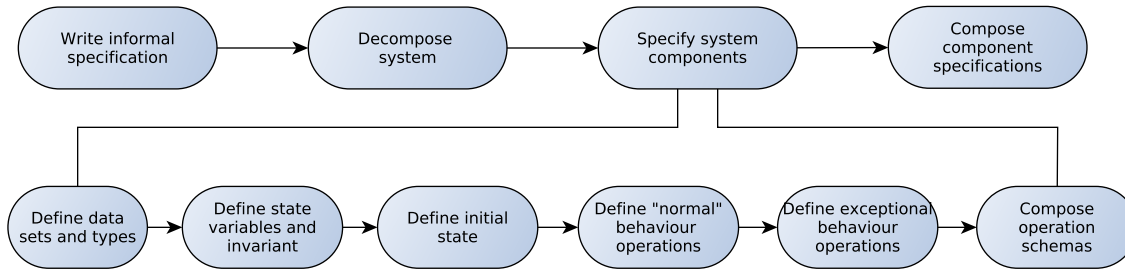


Figure 3.3: A process to develop a model-based specification of a system.

password storage system defined in this chapter is one such example.

To do this, we used the *abstract state machine* pattern. Using this pattern, we use predicates to specify operations on a global state. An abstract state machine in Alloy will have a structure something similar to:

```

sig State { ... }
pred init [s: State] { ... }
pred inv [s: State] { ... }
pred op1 [s: State] { ... }
...
pred opN [s: State] { ... }

```

in which `init` is the *initialisation* operation, `inv` is the *state invariant*, and `Op1 ... OpN` is a set of N operations, such as `add` and `delete` in the password storage system. A state invariant is a formula that all operations in the specification must preserve.

Operation preconditions

We have already seen an example of splitting a precondition from its postcondition in an operation. However, an alternative way, which is preferred by many modellers, is to specify the precondition and postcondition together as separate predicates, and then bring them together in a single operation predicate. For example, we can declare operations `addPre` and `addPost` for the precondition and postcondition respectively, and then conjoin them in `addOk`:

```

//True if and only if the user has no password for this URL
pred addPre [pb: PassBook, url : URL, user: Username] {
    no pb.password[user][url]
}

//Add a password for a user/url pair
pred addPost [pb : PassBook, url : URL, user: Username, pwd: Password] {
    pb.password' = pb.password + (user->url->pwd)
}

//Add a password for a *new* user/url pair
pred addOk [pb : PassBook, url : URL, user: Username, pwd: Password] {
    addPre [pb, url, user]
    addPost [pb, url, user, pwd]
}

```

This has the effect of being more verbose, but providing a clear separation between the preconditions and the postconditions that specify the transitions, which many people find preferable.

Exceptional cases

An important step in Figure 3.3 is the separation of “normal” cases from “exceptional” cases. The idea is to specify normal cases for a model first, which are usually the most difficult parts, and then to enumerate all the relatively-easier exceptional cases after this, many of which just “fall into place”.

To achieve this, we specify the normal and exceptional cases in separate predicates, and then combine them using *disjunction* in one operation predicate:

```
//Add a password for a *new* user/url pair
pred addNormal [pb : PassBook, url : URL, user: Username,
                pwd: Password, report : one Report] {
    addNormalPre [pb, url, user]
    addNormalPost [pb, url, user, pwd]
    report in Success
}

//Fail to add a password that already exists
pred addExceptional [pb : PassBook, url : URL, user: Username,
                    report : one Report] {
    not addNormalPre [pb, url, user]
    pb.password' = pb.password
    report in Failed
}

//Add a password for a *new* user/url, otherwise, add nothing
pred add [pb : PassBook, url : URL, user: Username, pwd: Password,
          report : one Report] {
    addNormal [pb, url, user, pwd, report]
    or
    addExceptional [pb, url, user, report]
}
```

So, the add predicate specifies an operation in which we attempt to add a duplicate entry, in which case nothing changes, *or* the entry is not a duplicate, in which case we add the new entry.

Note the introduction of the variable `report : Report`, which is used as an output variable to specify whether the operation was successful or not. The Report signature is declared as:

```
//Report signatures for informing whether an operation was successful or not
abstract sig Report {}
one sig Failed extends Report {}
one sig Success extends Report {}
```

Thus, `report` must take on a value from either `Success` or `Failed`, which are disjoint sets.

This is useful in assertions where we can specify a case that we expect to e.g. fail, and check that it failed by simply checking that `report in Failed`.

Invariant preservation

When using the abstract state machine pattern, Alloy assertions provide the opportunity to check that the invariant is preserved by an operation, using the following template:

```
assert initEstablishesInv {  
  all s : State | init [s] => inv [s]  
}  
  
check initEstablishesInv  
  
//for each operation  
assert opNPreservesInv {  
  always all s : State, x : E, ... |  
    inv [s] and opN [s, x, ... ] => after (inv [s])  
}  
  
check opNPreservesInv
```

This says that the initial state satisfies the state invariant, and for each operation, if the state invariant holds on an arbitrary pre-state *s*, and we execute the operation, the state invariant holds on the post-state reached after the operation finishes.

In the PassBook example, a natural invariant is that there should be no duplicate passwords for the same username/URL pair.

```
//Each user/url pair maps to exactly one password  
pred inv [pb : PassBook] {  
  all user : Username, url : URL | lone pb.password[user][url]  
}
```

Then we can assert that the initial state satisfies the invariant, and that the add operation preserves it, as follows:

```
assert initEstablishes {  
  all pb : PassBook | init [pb] => inv [pb]  
}  
  
assert addPreservesInv {  
  always all pb : PassBook, user : Username, url : URL,  
    pwd : Password, report : Report |  
    inv [pb] and add [pb, url, user, pwd, report] => after (inv [pb])  
}
```

Of course we would repeat this same pattern for the other operations too.

Such checks can be useful not only for checking state invariants, which may satisfy safety properties, but also to check sanity of operations.

For example, checking this assertion using the Alloy Analyser on the `delete` operation actually revealed a fault that had not been detected before, which was subtle, but which modelled the semantics completely wrong. The line that removed the tuple from the `pb.passbook` relation was:

```
pb.password' = pb.password' - (user->url->Password)
```

Can you see what the fault was?

This predicate states that `pb.password` is equal in the post-state to *itself* with the tuple subtracted. The second occurrence of `pb.password'` should have been `pb.password`. This major semantic problem had not been discovered using check assertions `deleteIsUndo` and `deleteDuplicateDoesNothing`, due to a strange coincidence in how the operation was modelled and how the way these assertions were written.

Bounds When checking that the invariant holds initially, of course only one `State` is needed. Since no operation occurs, a `steps` bound of `1..1` is sufficient.

When checking that each operation preserves the invariant, likewise only one `State` is needed, and a `steps` bound of `1..2` is sufficient (assuming operations only cause a single state transition to occur).

Summary

So, by using the abstract state machine pattern, and structuring the model in a way that separates normal from exceptional behaviour, we can incrementally build up a model using a process such as the one in Figure 3.3.

Further, the abstract state machine pattern then lends itself naturally to some assertions such as checking whether invariants hold.

3.10 Trace-Based Modelling in Alloy

Alloy 6's features for modelling variable signatures and fields, and its temporal logic, are ideal for *trace-based models*, i.e. ones that talk not just about single transitions (involving at most two states) but models that talk about sequences of such transitions. In that case one can specify temporal properties that need to hold over all such sequences. We call those sequences *traces*.

To construct such a model we use `fact` declarations to constrain the set of traces that Alloy considers. Specifically, we need to specify the allowed initial states, from which traces can begin, and the allowed transitions that can occur between states.

Specifying the initial state Recall that we use a predicate `init` to specify the initial state. Recall also that a `fact` declaration specifies what needs to be true of the initial state in a trace. Therefore we can specify that the initial states in the trace must satisfy `init` as follows:

```
fact {
  all s : State | init[s]
}
```

Note that we do not use the `always` keyword here: doing so would specify that not only the initial state satisfies `init` but all subsequent states in the trace do too!

Specifying the allowed transitions To specify the allowed transitions between states we use a second `fact` declaration. Assume we have predicates `op1...opN` to specify the operations (transitions) in our system. Also assume we have a predicate `unchanged[s]` which holds for a `State s` when the post-state is equal to the pre-state, and so specifies what it means for a state to be unmodified by a transition.

Then we can specify that the only allowed transitions are those given by $op_1 \dots op_N$ or unchanged as follows:

```
always {
  all s : State | op1[s] or op2[s] or ... or opN[s] or unchanged[s]
}
```

This fact says that in all states of the trace (not just the initial ones) one of the operation predicates must hold, i.e. there is a transition to a subsequent state given by one of the system's operations or the state remains unchanged (the subsequent state is equal to the current one).

The inclusion of the `unchanged` predicate is important for technical reasons to do with how traces are represented in Alloy. In particular, Alloy traces represent infinite sequences of transitions as *lasso traces*: traces where the final state then transitions to some state already in the trace (possibly the final one). Without including the `unchanged[s]` possibility above, we rule out many such traces that Alloy should consider when analysing our model. Including it means Alloy will consider at least all traces of finite sequences of operations, each drawn from $op_1 \dots op_N$ above, up to the `steps` bound, since the final state is always free to loop back to itself via the `unchanged` transition.

We will see examples of this kind of model in lectures, including use of the temporal operators to search for interesting behaviours of it.

Interaction with Invariant Preservation Earlier we saw that to specify that an operation `op` preserves an invariant `inv` we can write an assertion:

```
assert opPreservesInv {
  always all s : State, x : E, ... |
    inv [s] and op [s, x, ... ] => after (inv [s])
}
```

You might wonder why we included the **always** operator in this assertion. The reason is because of how such assertions interact with the **fact** commands typically used to model traces, specifically the one that states that the first state in a trace is an initial state:

```
fact {
  all s : State | init[s]
}
```

Suppose we didn't include the **always** operator in the assertion above, and had written the following instead:

```
all s : State, x : E, ... |
  inv [s] and op [s, x, ... ] => after (inv [s])
```

Without the **always**, the **all** quantifier in this assertion refers only to the *first* state in the trace. Because the above fact assumes that the first state always satisfies `inv`, it actually *weakens* the above assertion making it equivalent to having written:

```
all s : State, x : E, ... |
  init [s] and inv [s] and op [s, x, ... ] => after (inv [s])
```

In other words, the fact and assertion do not interact well together when the assertion doesn't use **always**, because without **always** the assertion talks only about the first state in a trace and we have assumed the

first state always satisfies `init`. Therefore without **always** the assertion now only checks whether the operation preserves the invariant from the initial state—not from an arbitrary state satisfying the invariant. This is why assertions that operations preserve the invariant should use **always** temporal operator.

Finding Traces that Violate an Invariant Having modelled our system’s traces in Alloy, to get it to search for traces that violate an invariant, we can simply write an assertion that specifies that the invariant always needs to hold throughout all traces.

```
assert always_inv {  
  always all s: State | inv [s]  
}
```

In some system models, we might not wish to constrain the initial state so that it always satisfies the invariant. But we might still wish to search for traces in which the invariant holds in the first state of the trace but then gets violated later on. For such systems we can use an assertion of the following form:

```
assert traces_preserve_inv {  
  all s: State | inv[s] implies always all s2: State | inv[s2]  
}
```

This says that for all traces that begin in states satisfying the invariant, then always throughout the trace the invariant will hold.

3.11 Verification and validation

As with any software engineering artefact, verification and validation (V&V) are important. There are several techniques for V&V of formal specifications:

1. *Animation*: Animation is the process of executing a specification with the intention of finding problems. The formal, unambiguous nature of specifications means that execution is often possible. However, because Alloy (and many other formal specification languages) is built on first-order logic, not all specifications are executable.
2. *Model checking*: This is the process of attempting to execute every operation in every possible state of a system to show that certain properties hold; or more accurately, the find counterexamples of the properties. Model checking for model-based languages is difficult because the sheer number of states may be exponentially large, if not infinite. There is a lot of research around the world looking at how to reduce the number of states for model checking while maintaining its soundness. If you have taken the subject *Modelling Complex Software Systems*, you would used model checking to detect deadlock and to prove temporal logic properties of concurrency models.
3. *Review*: The specification can be subject to peer review, like any other document. This remains one of the most important and useful style of review for formal specification, despite advanced tools that can reason about them automatically.
4. *Proof*: Automated proof is the holy grail of formal specification. Proof is the most valuable of all techniques, because, unlike animation, model checking, reviews, and testing, if a property about

a model can be proved, then we can say with strong conviction that it is true (although in reality, there is always the possibility that the proof itself is wrong!).

In this section, we discuss proof in more detail. This is not a major part of the subject, however, proof is an important motivation as to why practitioners adopt formal methods in software and systems engineering. Woodcock and Davies [4] present a far more thorough and valuable use of proof in specification. In this section, we discuss the implications of proof and present a trivial example.

The importance of proof

If we have a specification of a system, then by reasoning about it using proof, we are likely to detect problems. If we have not started implementing the system, this provides us with an opportunity to find those problems before they become costly to fix. Further to this, the very process of constructing a proof can assist us in finding new requirements. That is, if we notice that we cannot prove a property about a system because some exceptional state has not been considered, we may need to add a requirement for this exceptional state. As Woodcock and Davies state: “*The practice of proof makes for better specifications*”.

There is a commonly-held opinion that proof is not possible on large-scale systems. Industrial application of formal specification and proof shows this opinion to be wrong. Proof can be applied to real systems running in real environments, and can provide cost savings too. There are situations where proof is not relevant or necessary, but to high integrity systems especially, there are situations where it is desirable and almost necessary. There may be some situations where a sketch of a proof is sufficient. The trick in apply proof is to know which proofs are worth attempting, which are worth sketching, and which are worth leaving.

It is worth referencing the following quote:

“Things like even software verification, this has been the Holy Grail of computer science for many decades but now in some very key areas, for example, driver verification we’re building tools that can do actual proof about the software and how it works in order to guarantee the reliability.” — Bill Gates, Chairman of Microsoft, speaking at WinHEC 2002 in Seattle, Washington, USA.

What can be proved?

Proof is used in some high integrity systems for cases in which the outcome of failure is catastrophic; e.g. resulting in death or complete mission failure. The types of properties that can be proved vary, however, some interesting properties that have been proved about formal models include:

- Proofs that certain security protocols cannot be cracked. More importantly, there have been cases of security protocols where proofs have demonstrated that they *can* be cracked.

The *Needham-Schroeder Public-Key Protocol* is a protocol intended to provide mutual authentication between two parties communicating over a network, proposed by Roger Needham and Michael Schroeder in 1978. It was considered to be a failproof way to establish identity between two communicators, however, it was not until Gavin Lowe applied formal proof to the protocol in 1995 that it was discovered that the protocol failed to prevent “man-in-the-middle” attacks.

- Proofs about safety properties. Many safety-critical systems need to ensure certain safety properties hold. For example, railway signalling systems need to ensure that, given an intersection of tracks, at no stage will overlapping tracks both have non-red signals. That is, only one train will be allowed on the intersection at any time. These properties are specified as temporal logic properties, and it can be shown that a formal specification preserves these properties.
- In many life-critical systems, formal proof is applied to minimise the risk of causing harm or death. In many systems employed on NASA space missions, proof is applied to demonstrate that certain cases cannot go wrong. In particular, NASA has been known to reject entire proposals for systems on aerospace systems or life-support systems for astronauts unless model checking or proof have been used to demonstrate mission-critical properties (mainly around safety).

What can't proof do?

It is important not to say that if we “prove” our specification, then it is correct. When we do proofs about a formal specification, we are only proving that certain properties hold. We can say with some conviction that, if our proof can be discharged, then the property holds. However, this is only valuable if the property is correct. Further, we can do many proofs on a system, but omit an important one. Thus, a set of proofs is only as strong as the set of properties they show.

In most cases, proofs are not automated. That is, we cannot provide a specification and a property, and then have it proved. There are a large set of proofs for which this is possible, but there is another set for which it is currently not. The holy grail of theorem proving is exactly to prove things automatically, but for now, it remains a pipedream. For many logics, it has been proved that not all provable properties of the system can be automatically proved.

An example

As a small example of how proof works, we'll use the chemical storage system outlined in Chapter 2.

Example 3.3

In the chemical storage system, the storage tank has a maximum limit, and if that limit is breached, serious safety consequences may result. However, the overfilling problem is resolved using careful specification, and we can prove that the storage tank will not overflow, providing our specification is implemented correctly.

If we consider a `Fill` operation for filling the tank, it will consist of a precondition checking that the amount going into the tank will not overflow, and a postcondition calculating the new amount. The safety property we want to check is that the new amount does not overflow the tank:

- *Precondition:* $contents + amount? \leq capacity$
- *Postcondition:* $contents' = amount? + contents$
- *Safety property:* $contents' \leq capacity$ (which is also the state invariant).

We can then structure our proof as follows:

$$Precondition \Rightarrow (Postcondition \Rightarrow Safety\ property)$$

That is, if the precondition is true, then the postcondition implies that the safety property holds.

For the `Fill` operation, this is straightforward to discharge. We have to prove:

$$\begin{aligned} \text{contents} + \text{amount}? &\leq \text{capacity} \Rightarrow \\ (\text{contents}' = \text{contents} + \text{amount}? &\Rightarrow \text{contents}' \leq \text{capacity}) \end{aligned}$$

Using the *one-point rule*, we can replace all instances of `contents'` with `contents + amount?`:

$$\begin{aligned} \text{contents} + \text{amount}? &\leq \text{capacity} \Rightarrow \\ (\text{contents} + \text{amount}? = \text{contents} + \text{amount}? &\Rightarrow \text{contents} + \text{amount}? \leq \text{capacity}) \end{aligned}$$

The middle proposition $\text{contents} + \text{amount}? = \text{contents} + \text{amount}?$ is trivially true, so can be removed from the entire proposition, leaving:

$$\text{contents} + \text{amount}? \leq \text{capacity} \Rightarrow \text{contents} + \text{amount}? \leq \text{capacity},$$

which is itself trivially true.

So, we have *proved* that, for every possible state of the system, if the precondition holds, the storage container will not be filled beyond capacity. This is much stronger than throwing a million tests at it to see if we can find an input/state pair that violates it.

□

The above example is trivial in comparison to the types of proofs that are discharged on industrial scale systems, however, it illustrates how proof can be applied to a model-based specification.

Despite the small size, the proof required a few steps are several lines of text. However, significant tool support is available for theorem proving. Using automated and interactive theorem provers, many large proofs can be discharged in a fairly straightforward manner, and the documentation of the proof can be generated automatically and stored digitally.

Bibliography

- [1] Daniel Jackson. *Software Abstractions: logic, language, and analysis*. MIT press, 2012.
- [2] Mark Phillips. CICS/ESA 3.1 experiences. In *Z User Workshop, Oxford*, pages 179–185, 1989.
- [3] Ian Sommerville. *Software Engineering*. Pearson Higher Education, USA, 9th edition, 2010.
- [4] Jim Woodcock and Jim Davies. *Using Z: specification, refinement, and proof*, volume 1. Prentice Hall, 1996.

Chapter 4

Introduction to Ada

In this chapter, the programming language called *Ada* is introduced. We will use Ada in this subject for two reasons. First, due to its features, it is heavily used in high integrity domains, especially safety-critical systems. Second, later in the course, we will look at a language called SPARK, which is a safe programming subset of Ada, and has tool support for high integrity systems.

4.1 The history of Ada

Ada is a structured, imperative programming language with static typing, object-orientation, and concurrency built in. It was designed by a company called CII Honeywell Bull, who were contracted by the United States Department of Defence (DoD) in 1977 to provide a new language to replace the *hundreds* of programming languages they were using at the time. The DoD were concerned that many of the languages were specific to particular hardware, and thus not reusable, and were becoming obsolete. Further, many of the languages did *safe, modular* programming, which had emerged as the best way to handle large-scale, complex software systems.

At the time, four contractors were asked to provide proposals for a new language, having been given a set of criteria. The CII Honeywell Bull group's proposal was chosen as the successful proposal, and the Ada language was then designed.

Initially intended to be for embedded and real-time systems, Ada has become highly used in high integrity systems in general, especially in the safety-critical domain. Further, it was used as a general purpose programming language in the 1980s, however, due to its property of being designed to be read rather than written, many people do not consider it suitable for the more “enterprise-y” applications, where time-to-market is considered more important than software integrity.

The language is named after *Augusta Ada King, Countess of Lovelace* (more generally known as *Ada Lovelace*, who is credited as being the first even computer programmer. In 1843, when translating notes a seminar by Charles Babbage on his analytical engine (the first programmable machine) from French to English, Ada Lovelace included an addendum showing how an algorithm for calculating Bernoulli numbers using the analytical engine, which is considered the first program ever written. However, there is significant controversy as to her actual intellectual input into the algorithm, but much of this is inconsistent with Babbage's own recollection.

4.2 Ada for programming “in the large”

A major criteria given by the DoD for a new programming language is that it must provide strong support for good software engineering practices that scale well to very large software systems (10^6 lines of code, and large development teams).

There are several aspects of Ada that contribute to this:

- A strong, static and safe type system. In particular, all type errors can be detected automatically at compile time.
- Modularity. Ada modules (called “packages”) can be compiled separately from all other packages. Interfaces can also be compiled without the need for an implementation, meaning that some design errors can be detected before implementation begins.
- Information hiding. In providing interfaces/packages, Ada separates interfaces from implementation, and provides fine-grained control over visibility.
- Readability. This is an important aspect. Ada favours the *reader* of the program over the *writer*. That is, the aim is to make Ada programs easier to read (compared to other languages), possibly at the expense of making them slightly harder to write. This is because a program is written once but read many times during maintenance.
- Portability. Ada does not include any platform-specific operators or data types, and can thus be compiled on many platforms.
- Standardisation. Ada is an ANSI and ISO standard.

It is these properties, plus others, that makes Ada a great fit for high integrity systems, and is why it is still used in so many high integrity domains today. The language itself continues to evolve, and a new ISO/IEC standard for Ada was published in December 2012.

4.3 Hello World

A great way to introduce a programming language to people with programming experience is to start programs. So, let’s start with a very traditional program to introduce Ada:

Listing 4.1: **hello_world.adb**

```
1 with Ada.Text_IO;  
2  
3 procedure Hello_World is  
4   begin  
5     Ada.Text_IO.Put_Line("Hello, world!");  
6 end Hello_World;
```

Note the following about this program:

- The **with** statement adds the *Ada.Text_IO* package to the program. This is basically a library import, similar to the “import” functionality in Java, or the `#include` keyword in C.

- Block structure is given by **begin...end** pairs. Thus, a procedure begins and ends with these keywords.
- An Ada main procedure does not need to be called “main”, as in some other languages. Any simple name is fine so we have used *Hello_World*.
- The name-space is not flat. Therefore, to access the functions declared in the *Ada.Text_IO* package, we need to prefix them with the package name.

Package renaming

To avoid having to prefix functions with this long name each time, we can use *renaming*:

Listing 4.2: **hello_world_renaming.adb**

```

1 with Ada.Text_IO;
2
3 procedure Hello_World_Renaming is
4   package IO renames Ada.Text_IO;
5 begin
6   IO.Put_Line("Hello, world!");
7   IO.New_Line;
8   IO.Put_Line("...using package renaming!");
9 end Hello_World_Renaming;
```

Compiling Hello World

Ada programs are divided into *compilation units*. An Ada *specification* (or interface) is usually given the extension `.ads` (**ADa** Specification) and the body of the program is given the extension `.adb` (**ADa** Body).

The file name must be the same as the compilation unit name and each compilation unit must hold the name of the procedure to be compiled¹. For example, the hello world program must be placed in the file `hello_world.adb`.

All compilation units must be compiled, bound and linked.

The notes for this subject will use the GNAT Ada compiler, which is downloadable from <http://libre.adacore.com/download/> for Linux and Windows (native, JVM, and .NET)... and Mind-storm! The notes and workshops assume GNAT on Linux. Students are free to use the Windows compiler, but note that the staff cannot give much useful support on using these tools on Windows.

The GNAT Ada compiler can be used on the command line. To compile the hello world program, use the following commands:

1. `gcc -c hello_world.adb` generates the files `hello_world.ali` – the object code – and `hello_world.o` – static analysis information.
2. `gnatbind hello_world` uses `hello_world.ali` to bind the files.
3. `gnatlink hello_world` uses `hello_world.ali` to link the files.

¹Although some Ada compilers are forgiving and just produce a warning in this case

Alternatively, and more conveniently, use the following:

1. `gnatmake hello_world.adb` is a single command that performs all three of the above steps.

The result is an executable program called `hello_world`.

You can use the command `gnatclean -c hello_world` to delete any compiler-generated files.

4.4 Specifications and program bodies

Ada programs consist of *packages*, *procedures*, and *functions*. Packages are synonymous with *modules*, functions are defined as in other languages, and procedures are functions with no return value. More on procedures and functions later.

Specifications show the types, functions, procedures and tasks that are publicly available for an Ada program.

As an example, we can make a simple `Greetings` package with the interface below, defined in the file `greetings.ads`:

Listing 4.3: `greetings.ads`

```
1 package Greetings is
2   procedure Hello;
3   procedure Goodbye;
4   function Talk(Text : in String) return Integer;
5 end Greetings;
```

This declares two procedures, `Hello` and `Goodbye`, but does not define their implementation. The package *body* defines the implementation, possibly including hidden data types (we'll see an example of this later), defined in the file `greetings.adb`:

Listing 4.4: `greetings.adb`

```
1 with Ada.Text_IO; use Ada.Text_IO;
2
3 package body Greetings is
4   procedure Hello is
5     begin
6       Put_Line("Hello, world!");
7     end Hello;
8
9   procedure Goodbye is
10    begin
11      Put_Line("Goodbye, world!");
12    end Goodbye;
13
14   function Talk(Text : in String) return Integer is
15     begin
16       Put_Line(Text);
17       return 1;
18     end Talk;
19 end Greetings;
```

The names, parameter types, and return types must match between the specification and the body.

Because the two procedures are contained in a package, neither of them are directly executable. Instead, they must be executed from another procedure. Below is a code listing that calls the two procedures in succession:

Listing 4.5: `greetings_main.adb`

```
1 with Greetings; use Greetings;
2
3 procedure Greetings_Main is
4   I : Integer;
5 begin
6   Hello;
7   I := Talk(Text => "What's happening?");
8   GoodBye;
9 end Greetings_Main;
```

Note in this example a new keyword: `use`. The statement `use Ada.Text_IO` makes all of the public entities accessible without qualification in the body of the package, meaning that we no longer need to prefix `Put_Line` with the package name. However, use this with caution: two packages can have the public entities with the same name, so importing both results in an ambiguity.

Exercise 4.1

Create two packages that contain a procedure of the same name. Import both of these using the `with` and `use` keywords, into a new program that simply calls the ambiguously declared procedure. Try to compile and run this program to see what happens. □

Following the one-unit-per-file rule the program must be placed in three separate files:

1. `greetings.ads`: contains the specification of the package.
2. `greetings.adb`: contains the implementation of the package.
3. `greetings_main.adb`: contains the main program that makes use of the `Greetings` package.

To compile this program, we can compile the package and the program, and then bind them, as follows:

1. `gcc -c gmain.adb`
2. `gcc -c greetings.adb`
3. `gnatbind greetings_main`
4. `gnatlink greetings_main`

Alternatively, issue the single command:

1. `gnatmake greetings_main`

The `gnatmake` command will locate and compile any packages used in the main program, provided that it can find them on the path. To avoid having to set up paths, we'll only deal with programs in a single directory.

4.5 Comments

Comments can be added to programs using the prefix `--`:

Listing 4.6: Comments

```
1 -- this part ignored
2 Put_Line("Hello, world!"); -- this part too!
```

4.6 Types

The Ada type system is a major part of its success. It is designed for programmers to create abstractions that model real world objects. As such, the type system is not based on a collection of pre-defined types, but instead, allows designers to create new types.

Of particular note in the type system are the following properties:

- *Strong typing*: types are incompatible with one another.
- *Static typing*: type checked while compiling.
- *Abstraction*: types represent the real world data or the problem at hand and computer representations of the data.
- *Name equivalence*: not structural equivalence as used in most other languages. Two types are compatible if and only if they have the same name; not if they just happen to have the same size or bit representation.

4.6.1 The predefined Ada type system

Despite this, there are still a handful of predefined types in Ada. Predefined types are used as interfaces between different packages or libraries. The predefined library uses these types too.

The following predefined types are declared in the *standard package*:

- **Integer**: The standard integer type but must cover at least integers in the range $-2^{15} + 1 \dots 2^{15} - 1$.
- **Float**: The type of floating point numbers with mantissa and exponent but the implementation does not follow the IEEE standard.
- **Duration**: A fixed point type used for timing and representing a period of time in seconds.
- **Character**: A special form of *enumeration* type. There are three predefined kinds of character types: 8-bit characters, 16-bit characters, and 32-bit characters.
- **String**: Three indefinite array types, of 8-bit characters, 16-bit characters, and 32-bit characters respectively.
- **Boolean**: A Boolean in Ada is an enumeration of False and True with special semantics.

The packages *System* and *System.Storage_Elements* predefine some types that are primarily useful for low-level programming and interfacing to hardware:

- **System.Address:** An address in memory.
- **System.Storage_Elements.Storage_Offset:** An offset, which can be added to an address, to obtain a new address. You can also subtract one address from another to get the offset between them. Together, **Address** and **Storage_Offset** and their operators provide address arithmetic.
- **System.Storage_Elements.Storage_Count:** A subtype of **Storage_Offset** which cannot be negative, and represents the memory size of a data structure (similar to C's `size_t`).
- **System.Storage_Elements.Storage_Element:** In most computers, this is a byte. Formally, it is the smallest unit of memory that has an address.
- **System.Storage_Elements.Storage_Array:** An array of **Storage_Elements** without any meaning, useful when doing raw memory access.

Figure 4.1 shows the Ada type hierarchy. A type inherits properties from types above it in the hierarchy.

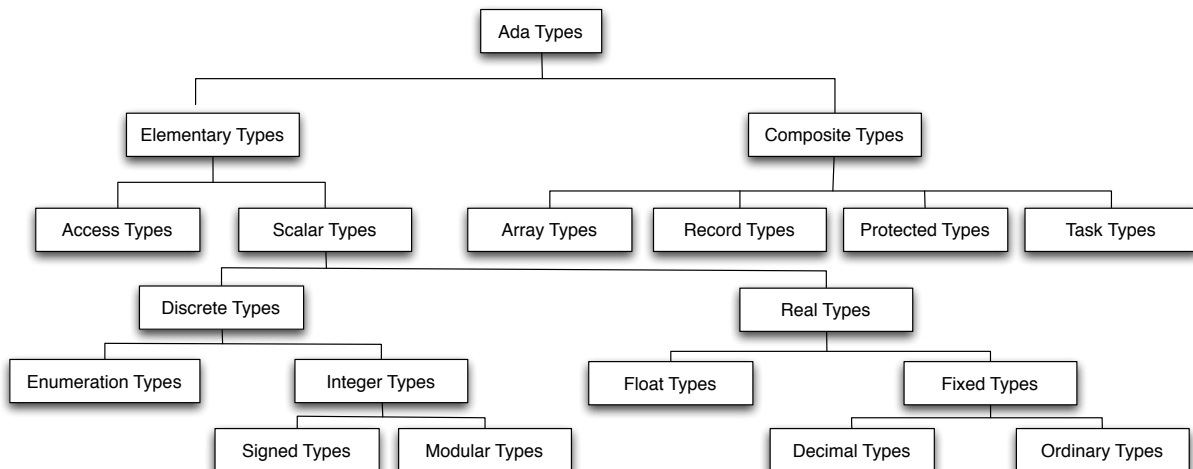


Figure 4.1: The Ada Type hierarchy.

Converting to/from strings

All scalar types (refer to Figure 4.1) have two special attributes called `'Value` and `'Image`, which are used for converting to strings to values, and back. Both are functions that are called on the type, not the individual values (the Java metaphor is a static method, as opposed to an instance method).

The following code listing demonstrates how to convert a string to an integer, and back again:

Listing 4.7: `value_image_attributes.adb`

```
1 with Ada.Text_IO; use Ada.Text_IO;
```

```
2
```

```

3 procedure Value_Image_Attributes is
4   An_Integer : Integer;
5   A_String : String := "5";
6 begin
7   An_Integer := Integer'Value(A_String);
8   Put_Line("Hello, world!" & Integer'Image(An_Integer));
9 end Value_Image_Attributes;

```

Exercise 4.2

Write a program that converts a string containing a non-number to an integer. Compile and run this program to see what happens. □

4.6.2 Arrays

Arrays in Ada are somewhat different to many other high-level programming languages – and in fact, are more powerful as well. One key difference is in the indices of the array. In many languages, the indices of arrays are integers, starting at either 0 or 1. However, in Ada, array indices can be of any discrete type, such as characters, enumerations, etc.

As an example, consider the following program, which declares two arrays of type `Character`, but each with a different type of index: in one case, an range of integers; and in the other, a range of characters. Elements in the array are accessed using integers for the array `Int_Index`, but characters for the array `Ch_Index`.

Listing 4.8: `array_examples.adb`

```

1 with Ada.Text_IO; use Ada.Text_IO;
2 with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;
3
4 procedure Array_Examples is
5   Int_Index : array(Integer range 5..10) of Character :=
6     ('a', 'b', 'c', 'd', 'e', 'f');
7   Ch_Index : array(Character) of Character;
8 begin
9   Put(Int_Index(5)); -- accessing using an integer index
10  New_Line;
11
12  Ch_Index('a') := 'z'; -- setting using a char index
13  Put(Ch_Index('a')); -- accessing using a char index
14  New_Line;
15
16  -- setting and getting an array 'slice'
17  Int_Index(6 .. 8) := ('X', 'Y', 'Z');
18
19  -- array attributes "'First" and "'Last"
20  Put(Ch_Index'First);
21  New_Line;
22  Put(Int_Index'Last);
23  New_Line;
24 end Array_Examples;

```

Arrays have four special attributes: `'First`, `'Last`, `'Length`, and `'Range`. The first two of these do not return the first and last elements of the array, but the first and last elements of the *type* of the array.

Thus, in the above example, `Int_Index'Last` will return 10, and `Ch_Index'First` will return the final ASCII character (null).

The `'Length` attribute returns the length of the array type (and of the array – the entire array is allocated in Ada), while the `'Range` attribute returns the range of indices; e.g. `Int_Index'Range` will return `5..10`.

Array	'First	'Last	'Length	'Range
<code>Int_Index</code>	5	10	6	5..10
<code>Ch_Index</code>	<code><null></code>	<code>ÿ</code>	255	<code><null> .. ÿ</code>

The pre-defined type `String` is a character array. However, the indices need not start at 0 or 1. Consider the following example, taken from the Ada Wikibook²:

Listing 4.9: Attributes of strings

```

1 Hello_World : constant String := "Hello World!";
2 World       : constant String := Hello_World (7 .. 11);
3 Empty_String : constant String := "";

```

In this example, the attribute values are:

Array	'First	'Last	'Length	'Range
<code>Hello_World</code>	1	12	12	1 .. 12
<code>World</code>	7	11	5	7 .. 11
<code>Empty</code>	1	0	0	1..0

Thus, strings can be indexed from any range of integers, and it is not necessarily that case that `'Length` = `'Last`.

4.6.3 Defining new types

A new type is introduced with the syntax:

type `T` **is** ...

Consider the following example of defining a type for dates.

Listing 4.10: A data type for dates

```

1 type Day_type    is range    1 .. 31;
2 type Month_type  is range    1 .. 12;
3 type Year_type   is range 1800 .. 2100;
4 type Hours       is mod 24;
5 type DayOfWeek   is (Monday, Tuesday, Wednesday, Thursday,
6                     Friday, Saturday, Sunday);
7
8 type Date is
9   record
10     Day    : Day_type;
11     Month  : Month_type;
12     Year   : Year_type;

```

²See http://en.wikibooks.org/wiki/Ada_Programming/Types/array

```
13 end record;
```

A *record* is a composite type that groups one or more data type together into a single data type, similar to a `struct` in C.

All types are incompatible with each other. Therefore, the following code snippet is illegal, because `Day_type` and `Month_type` are different and incompatible types, even though the value 8 can be assigned to either:

Listing 4.11:

```
1 A : Day_type := 8;
2 B : Month_type := A; -- illegal!
```

4.7 Control Structures

As a structured language, the control flow of the program is structured into statements. The following control structures are the most pertinent in the Ada language.

Assignment Assignment in Ada is written as

```
X := E
```

where X is a variable and E is an expression.

Assignment is read as “*X becomes equal to E*”.

Conditionals *if-then-else* and *case*:

Listing 4.12: A conditional

```
1 if x < y then
2   Put_Line ("True");
3 else
4   Put_Line ("False");
5 end if;
```

Listing 4.13: A case statement

```
1 case x is
2   when 1 => Put_Line ("First");
3   when 2 => Put_Line ("Second");
4   when 3 => Put_Line ("Third");
5   when others => Put_Line ("None!");
6 end case;
```

Unconditionals *return* and *goto*:

Listing 4.14: Returning a value

```
1 function Use_Return return Integer is
2   X : Integer := 5;
3   Y : Integer := 10;
4 begin
5   return X + Y;
6 end Use_Return;
```

Listing 4.15: Using a goto

```

1 function Use_Goto return Integer is
2   X : Integer := 5;
3   Y : Integer := 10;
4   Z : Integer;
5 begin
6   Z := X + Y;
7
8   goto Label;
9
10  Z := 0;
11
12  <<Label>>
13
14  return Z;
15 end Use_Goto;

```

Loops There are a number of different kinds of loops in Ada:

- Endless loops:

Listing 4.16: endless_loop.adb

```

1 with Ada.Text_IO; use Ada.Text_IO;
2
3 procedure Endless_Loop is
4 begin
5   loop
6     Put_Line("Hello again, world!");
7   end loop;
8 end Endless_Loop;

```

- While loops, which specify the termination condition at the start of the loop:

Listing 4.17: while_loop.adb

```

1 with Ada.Text_IO; use Ada.Text_IO;
2
3 procedure While_Loop is
4   X : Integer := 0;
5 begin
6   while X < 10 loop
7     Put_Line("Hello again, world!");
8     X := X + 1;
9   end loop;
10 end While_Loop;

```

- Exit loops, which specify the termination condition in the middle of the loop:

Listing 4.18: exit_loop.adb

```

1 with Ada.Text_IO; use Ada.Text_IO;
2
3 procedure Exit_Loop is
4   X : Integer := 0;
5   Y : Integer := 0;

```

```

6 begin
7   loop
8     Put_Line("Hello again, world!");
9     X := X + 1;
10    exit when X >= 10;
11    Y := Y + 2;
12  end loop;
13 end Exit_Loop;

```

- Until loops, which specify the termination condition at the end of the loop (which is just an exit loop with the exit condition at the end of the loop):

Listing 4.19: `until_loop.adb`

```

1 with Ada.Text_IO; use Ada.Text_IO;
2
3 procedure Until_Loop is
4   X : Integer := 0;
5 begin
6   loop
7     Put_Line("Hello again, world!");
8     X := X + 1;
9     exit when X >= 10;
10  end loop;
11 end Until_Loop;

```

- For loops, which allow iteration of all values in a range:

Listing 4.20: `for_loop.adb`

```

1 with Ada.Text_IO; use Ada.Text_IO;
2 with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;
3
4 procedure For_Loop is
5 begin
6   for I in Integer range 1 .. 10 loop
7     Put("Hello again, world!");
8     Put(I);
9     New_Line;
10  end loop;
11 end For_Loop;

```

If the range is omitted, the loop will iterate over all possible values of the type.

- Array loops, which allow iteration over all elements in an array (or any other iterable types, for that matter):

Listing 4.21: `array_loop.adb`

```

1 with Ada.Text_IO; use Ada.Text_IO;
2 with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;
3
4 procedure Array_Loop is
5   X_Array : array(Integer range 1..10) of Integer :=
6     (29, 28, 27, 26, 25, 24, 23, 22, 21, 20);
7 begin
8   for I in X_Array'Range loop
9     Put("Hello again, world!");

```

```

10     Put (X_Array(I) );
11     New_Line;
12   end loop;
13 end Array_Loop;

```

4.8 Procedural Abstractions and Functional Abstractions

The Ada language explicitly distinguishes between procedures and functions:

Definition 4.1

A *procedure* does not return any value and is considered a *statement*. □

Definition 4.2

A *function* returns a value and is itself an expression. □

As such, functions can be used as part of larger expressions, but procedures cannot. Earlier in the hello/goodbye example (in the `Greetings` package), we saw an example of how to call a procedure (which contained no parameters).

For succinctness, we use the term *subprogram* to include both functions and procedures.

4.8.1 Parameter Modes

When defining a procedure, each parameter of the procedure must have one of the following modes:

1. `in`: The formal parameter is an input parameter only, and may be read from, but cannot be written to. It is treated as a constant in the subprogram. The actual parameter value is copied into the formal parameter and is not changed by the the call.
2. `out`: The formal parameter is an output parameter only. That is, the calling code will want to read the value of the parameter after the call is finished. The actual parameter's value before the call is irrelevant because it will get a value in the subprogram. The formal parameter may be read and written in the subprogram.
3. `in out`: The formal parameter is both an input and output parameter, and can thus be both read and written. The actual parameter may be redefined in the subprogram body.
4. `access`: The formal parameter is an *access type* (a pointer) to some variable.

If a parameter is declared without one of these modes, then by default, its mode is `in`.

For functions, parameters must be of mode `in` or `access`. To write a subprogram that returns more than one value, the return values must be all `out` parameters.

4.8.2 Calling subprograms

There are three syntax variations for calling subprograms:

1. Calling subprograms that have no parameters – called as just the subprogram name with no brackets.
2. Calling via positional arguments. This is found in most high-level programming languages, in which the position of the arguments in the call must be the same as in the declared subprogram.
3. Calling via named associations. This is where parameter variable names are explicitly specified in the calling code.

The code listing below provides examples of all three call types:

Listing 4.22: `calling_subprograms.adb`

```

1 with Ada.Text_IO; use Ada.Text_IO;
2 with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;
3
4 procedure Calling_Subprograms is
5     An_Int : Integer := 50;
6     Another_Int : Integer := 60;
7 begin
8     Put (An_Int, 20); -- positional parameter
9     New_Line;         -- no parameters
10    Put (Width => 20, Item => An_Int); -- named associations
11 end Calling_Subprograms;

```

Note that the order of the arguments in the final call (using named associations) is different to the declared order.

4.9 access Types: Passing Parameters By Reference

Consider the following program.

Listing 4.23: `pass_by_value.adb`

```

1 with Ada.Text_IO; use Ada.Text_IO;
2 with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;
3
4 procedure Pass_By_Value is
5     type Rec is record
6         X : Integer;
7     end record;
8
9     R : Rec;
10
11    procedure Print_R is
12    begin
13        Put ("R.X: ");
14        Put (R.X);
15        New_Line;
16    end Print_R;
17
18    procedure CheckR(givenR : in out Rec) is
19    begin

```

```

20     Put ("In CheckR"); New_Line;
21     givenR.X := R.X + 1;
22     if GivenR = R then
23         Put ("Equal");
24     else
25         Put ("Not Equal");
26     end if;
27     New_Line;
28     Print_R;
29 end CheckR;
30
31 begin
32
33     R.X := 5;
34     CheckR(R);
35     Put ("CheckR done"); New_Line;
36     Print_R;
37
38 end Pass_By_Value;

```

It declares a new record type `Rec` which in this case has just a single `Integer` field `X`. However in general records can be very large. The program declares a record variable `R` and sets its `X`-field to 5. The `CheckR` procedure is designed to take a `Rec` as argument and if it is equal to the record `R` then it prints out “Equal”; otherwise it prints “Not Equal”. `CheckR` *modifies* the given record first before performing this test.

What happens when we run this program? What does it print?

```

$ ./pass_by_value
In CheckR
Not Equal
R.X:           5
CheckR done
R.X:           6

```

It prints “Not Equal”. The reason is because when `CheckR` is called, the record `R` is passed *by value*. This means that its contents are *copied* into a new record `givenR` that is given to `CheckR`. This is why when `R` is printed during `CheckR`, the value 5 is still printed — the modification to `givenR` has not (at least yet) modified `R`.

When `CheckR` returns, the contents of `givenR` are copied *back* to the original record `R`, which is why at that point the value 6 is then printed.

Programming languages like C support *pointers* to enable data structures to be passed *by reference* rather than by value. A reference to the structure can be passed to a subprogram (function or procedure like `CheckR`), allowing modifications performed during the subprogram to immediately update the original data structure.

The following program is a small variation on **pass_by_value.adb** above showing how `R` can be passed by reference to `CheckR` instead.

Listing 4.24: **pass_by_reference.adb**

```

1 with Ada.Text_IO; use Ada.Text_IO;
2 with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;

```

```

3
4 procedure Pass_By_Reference is
5   type Rec is record
6     X : Integer;
7   end record;
8
9   type RecPtr is access all Rec;
10
11   R : aliased Rec;
12
13   procedure Print_R is
14   begin
15     Put ("R.X: ");
16     Put (R.X);
17     New_Line;
18   end Print_R;
19
20   procedure CheckR(givenR : in RecPtr) is
21   begin
22     Put ("In CheckR"); New_Line;
23     givenR.X := R.X + 1;
24     if GivenR = R'Access then
25       Put ("Equal");
26     else
27       Put ("Not Equal");
28     end if;
29     New_Line;
30     Print_R;
31   end CheckR;
32
33 begin
34
35   R.X := 5;
36   CheckR(R'Access);
37   Put ("CheckR done"); New_Line;
38   Print_R;
39
40 end Pass_By_Reference;

```

Before getting to the details of how this program is implemented, let's first see what it prints when it runs.

```

$ ./pass_by_reference
In CheckR
Equal
R.X:          6
CheckR done
R.X:          6

```

Observe that the modification performed inside `CheckR` directly updates the record `R`. Also observe that `CheckR` now prints out “Equal” since it is now comparing two *pointers* (i.e. two references) that both point to the same data structure, namely to `R`.

This program declares a new type `RecPtr` that holds pointers to `Recs`. Values of this type are not

records of type `Rec` but rather *point to* such records. One can think of variables of type `RecPtr` as storing memory addresses that are the locations in memory of `Rec` objects. This type is an `access` type with the `all` qualifier, meaning that it can point to `Recs` that are declared as local variables like `R` (i.e. those that are allocated on the stack).

`CheckR` now takes a `RecPtr` as its argument. When it is called, rather than passing `R` we pass `R' Access`, which is the expression that evaluates to a pointer to `R`. Ada won't allow us to get a pointer to `R` unless we declare `R` as `aliased`. This is why this qualifier is added to the declaration of `R`. It tells the compiler that more than one reference to the memory occupied by `R` might exist at some point during the execution of the program.

Inside `CheckR`, it compares this given `RecPtr` to see if it is equal to `R' Access`, i.e. whether the given pointer points to the record `R`. Of course it does which is why the program prints out "Equal".

4.9.1 Access Types to Read-Only Data

What if you want to pass something by reference but don't want to allow it to be modified?

This is done by using `access constant` instead of `access all`. For example, if we replace `access all` by `access constant` in **pass_by_reference.adb** and try to compile it, we get a type error caused by the assignment statement that attempts to modify the record pointed to by `givenR`. This is because, with this change, the `RecPtr` type is now declared as pointing to *read only* `Recs`. Therefore any procedure that is passed a `RecPtr` is not allowed to modify the record that the pointer points to.

4.9.2 Summary

To summarise, access types are used to pass data structures by reference, which avoids them having to be copied between procedures. They also allow one to check whether two given references refer to the same data structure in memory.

Access types get more interesting when used in conjunction with dynamic memory allocation, which we don't cover in this course. Here we have only scratched the surface. You should consult the references at the end of this chapter to learn more.

4.10 Example: Insertion sort

Before we go any further, it is time for a complete example. The example we use is *insertion sort*³ – a well-known sorting algorithm with worst-case complexity $O(n^2)$, which works by finding the smallest remaining element in the unsorted part of the array and inserting it at the end of the sorted part of the array. This example uses many of the constructs seen already, declaring the procedure in a package.

The package specification consists of a type declaration and a procedure declaration:

Listing 4.25: **sort.ads**

```
1 package Sort is
2   type Data_Array is array (Natural range <>) of Integer;
3   procedure Insertion_Sort (Item : in out Data_Array);
```

³Taken from Rosetta Code: http://rosettacode.org/wiki/Sorting_algorithms/Insertion_sort#Ada

```
4 end Sort;
```

The package body implements the procedure specification:

Listing 4.26: sort.adb

```
1 package body Sort is
2
3   procedure Insertion_Sort (Item : in out Data_Array) is
4       First : Natural := Item'First;
5       Last  : Natural := Item'Last;
6       Value : Integer;
7       J     : Integer;
8   begin
9       for I in (First + 1)..Last loop
10          Value := Item(I);
11          J := I - 1;
12          while J in Item'range and then Item(J) > Value loop
13              Item(J + 1) := Item(J);
14              J := J - 1;
15          end loop;
16          Item(J + 1) := Value;
17      end loop;
18  end Insertion_Sort;
19 end Sort;
```

The following serves as a small demonstrating of creating an unsorted array, sorting it, and printing the result:

Listing 4.27: sort_main.adb

```
1 with Sort; use Sort;
2 with Ada.Text_IO; use Ada.Text_IO;
3 with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;
4
5 procedure Sort_Main is
6     Int_Array : Data_Array := (5, 3, 9, 1, 5, 4);
7 begin
8     -- sort the array
9     Insertion_Sort (Int_Array);
10
11     -- iterate over the array and print the elements
12     for I in Int_Array'First .. Int_Array'Last loop
13         Put (Int_Array(I), 2);
14         Put (" ", " ");
15     end loop;
16 end Sort_Main;
```

4.11 Concurrency with tasks

The final topic that we cover in this chapter is *tasks*. Tasks are the basic unit of concurrency in Ada. As such, they are much like threads in Java – but have several differences.

Tasks are important in Ada because they help to support real-time systems. Many (perhaps most?) safety- and mission-critical systems are real-time. That is, timing is part of the functional specification of the system, and failure to complete some function before a deadline means that the system has failed.

Tasks start executing the moment they are created. Tasks are not called, unlike functions and procedures. Tasks can communicate with other tasks:

- Tasks can pass messages – message passing;
- Tasks can share variables – shared memory communication.

Ada programs and tasks are scheduled. Unlike programs, the effects of task scheduling are often visible.

The example below⁴ shows the declaration and body of two tasks: one for “backing up” and one for “cleaning the CPU”.

Listing 4.28: **housekeeping.adb**

```
1 with Ada.Text_IO; use Ada.Text_IO;
2 with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;
3
4 procedure Housekeeping is
5
6     task Check_CPU;
7     task Backup_Disk;
8
9     task body Check_CPU is
10    begin
11        for count in 1..50 loop
12            Put (Item => "Check CPU status ");
13            Put (Item => count, Width => 1);
14            New_Line;
15        end loop;
16    end Check_CPU;
17
18    task body Backup_Disk is
19    begin
20        for count in 1..50 loop
21            Put (Item => "Backup disk status ");
22            Put (Item => count, Width => 1);
23            New_Line;
24        end loop;
25    end Backup_Disk;
26
27    -- the two tasks are automatically created and begin execution
28
29 begin -- Housekeeping
30     null;
31     -- Housekeeping waits here for them to terminate
32 end Housekeeping;
```

⁴Taken from https://en.wikibooks.org/wiki/Ada_Programming/Tasking

The master of the tasks is the procedure `Housekeeping`. The procedure itself does nothing except declare the tasks, which then execute automatically. The body of `Housekeeping` is empty and just waits for its child tasks to terminate.

Entry points

Tasks communicate with each other via *entry points* (or *entry calls*). Tasks can have one or more entry points. Explicitly named *entry points* are declared in the task declaration.

The `entry` keyword declares an entry point into a task. Entries are declared in much the same way as procedures. They have identifiers and parameters (which can be `in`, `out`, or `in out` parameters).

An entry is executed by the called task, not the calling task. The calling task is suspended until the call completes. If the called task cannot process the entry call immediately (most commonly because it is already executing on behalf of another call), the call is placed in a FIFO queue associated with that entry.

This interaction between calling task and called task is known as a *rendezvous*. A task accepts rendezvous with any caller for a specific entry by executing an *accept* statement for that entry.

The following code snippet demonstrates how entry and accept statements work:

Listing 4.29: `entry_call.adb`

```
1 with Ada.Text_IO; use Ada.Text_IO;
2 with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;
3
4 procedure Entry_Call is
5
6     task type Encapsulated_Variable_Task_Type is
7         entry Store (An_Integer : in Integer);
8         entry Fetch (An_Integer : out Integer);
9     end Encapsulated_Variable_Task_Type;
10
11     task body Encapsulated_Variable_Task_Type is
12         Datum : Integer;
13     begin
14         loop
15             select
16                 accept Store (An_Integer : in Integer) do
17                     Datum := An_Integer;
18                     Put ("Storing ");
19                     Put (An_Integer);
20                     New_Line;
21                 end Store;
22             or
23                 accept Fetch (An_Integer : out Integer) do
24                     An_Integer := Datum;
25                     Put ("Fetching ");
26                     Put (An_Integer);
27                     New_Line;
28                 end Fetch;
29             or
30                 terminate;
31             end select;
32         end loop;
```

```

33  end Encapsulated_Variable_Task_Type;
34
35  X, Y : Encapsulated_Variable_Task_Type;
36  It : Integer;
37
38  begin
39      It := 5;
40      X.Store(It);
41      X.Fetch(It);
42      It := It + 1;
43      Y.Store(It);
44  end Entry_Call;

```

The above example also demonstrates several other important aspects of tasks:

1. *Task types*: The two tasks used are of the same *task type*, `Encapsulated_Variable_Task_Type`. This allows tasks to be created *dynamically*, and also included into new data structures.
2. *Selective waits*: The `select` keyword is used to prevent a task being held up when it could be doing something. In the above example, the task can choose between storing or fetching an item. If there is only one pending call, that is executed. If there is more than one, *any* of the tasks can be chosen, thus introducing non-determinism into the program.
3. *Termination*: The loop in the tasks above will execute indefinitely. By giving the tasks a `termination` alternative, they will terminate when: (1) they have no pending calls; (2) no other tasks that are children of the task's master have any pending calls; and (3) the task's master has completed.

Exercise 4.3

Remove the `terminate` statement from the `Encapsulated_Variable_Task_Type` task and re-run the example. □

Guards

Entries can have guards that only allow the entry to execute under specific circumstance. For example, in Listing 4.29, we could add a guard to the `Store` entry restricting the input to under certain numbers:

Listing 4.30:

```

1  when An_Integer < 100 =>
2      accept Insert (An_Integer : in Integer) do

```

A guard means that this entry point is not open, another entry point can be executed.

4.12 Further reading

An excellent introduction to Ada, which includes chapters on systems programming, real-time systems, generic types, and distributed systems, is Ben-Ari's book "*Ada for Software Engineers*" [1]. This is available as an e-book from the University of Melbourne library, and is far more comprehensive than these notes.

A excellent online reference is the “Ada programming” Wikibook [2], which offers a substantial overview of the Ada language and many complete examples, some of which are used in this chapter.

Bibliography

[1] Mordechai Ben-Ari. *Ada for software engineers*. Springer, 2009.

[2] Wikibooks. *Ada Programming*. https://en.wikibooks.org/wiki/Ada_Programming.

Chapter 5

Safe programming language subsets

In this chapter, we introduce the concept of *safe programming language subsets*. These are subsets of programming languages in which unsafe parts of the language are not permitted. They are intended to make it impossible to implement certain types of faults in software.

The particular language that we will look at is SPARK – a subset of Ada. SPARK is intended to be a simple language that allows structured programs to be written, while preventing many types of serious faults that can affect the system, and are difficult to detect. It does that by constraining the syntax and the semantics of the language to be straightforward to reason about by both humans and machines.

5.1 Principles of correctness by construction

“There are two ways of constructing a software design. One way is to make it so simple that there are obviously no deficiencies. And the other is to make it so complicated that there are no obvious deficiencies.” — Professor Tony Hoare, 1980 ACM Turing Award Lecture

Correctness by construction is an approach to developing software at Altran Praxis, a small company in the U.K. that specialises in safety- and security-critical software. Altran Praxis are the company that defined SPARK and continue to provide high quality tool support for it.

There are seven key principles adhered to at Praxis:

1. Expect requirements to change.
2. Know why you’re testing: finding faults, not showing correctness.
3. Eliminate errors before testing. Testing is the second most expensive way of finding errors. The most expensive is to let your customers find them for you.
4. Write software that is easy to verify. Testing takes up to 60% of total effort. Implementation typically takes 10%. Doubling implementation effort is cost-effective if it reduces verification cost by as little as 20%.
5. Develop incrementally. Make small changes, incrementally. After each change, verify that the updated system behaves according to its updated specification.

6. Some aspects of software development are just plain hard. The best tools and methods take care of the easy problems, allowing you to focus on the difficult problems.
7. Software is not useful by itself. The executable software is only part of the picture: user manuals, business processes, design documentation, well-commented source code and test cases should not just be added at the end.

Learning outcomes

Throughout the next few chapters, we will explore just *some* of the principles above, and how Praxis (and others) go about adhering to them.

The learning outcomes of the next four chapters are:

1. Motivate the use of safe programming language subsets.
2. To understand the motivation of design by contract.
3. To specify formal contracts for small- and medium-scale packages in Ada.
4. Prove the correctness of a program against its contract.
5. Apply SPARK Ada static analysis tools for semi-automated proof of contract satisfaction.
6. Systematically construct tests from specifications and contracts.

In this chapter, we look at the first item above: safe programming language subsets.

5.2 Safe programming language subsets

The evolutionary nature of programming languages implies that there are many problems with our everyday programming languages. For example, C and Ada, the two most widely used programming languages in embedded software, have no agreed semantics. Furthermore, they both permit programs that have no defined behaviour. These two languages are two of the most well understood and strict languages in common use today. Other languages, such as Python, Ruby, and Java, have far greater problems.

Structured programming is where the procedural logic of a program is a structure composed of sub-structures in a limited number of ways; most commonly, constructed of sequencing, selection, and iteration.

Safe programming language subsets use structured programming to provide a small but powerful programming language that is easier to verify than non-safe languages.

Some programming language subsets

Some well-known safe programming language subsets include:

- *SafeD*, a safe subset of *D*, which is itself related to C++.

- *MISRA C*, a safe subset of C.
- *Joe-E*, a *secure* subset of Java.
- *SPARK*, a safe subset of Ada, but with *annotations* for additional static checking.

In this subject, we will focus on SPARK, as it is the most widely used safe programming language subset, and also has the most mature tools supporting it.

Example 5.1

The following is an example of a safe programming subset of C, defined by a strict coding standard. The coding standard [1] was defined by engineers at NASA's Jet Propulsion Laboratory, for use on NASA's Mars Rover *Curiosity*.

In fact, the standard is a subset of MISRA C, a safe subset of C defined the Motor Industry Software Reliability Association (MISRA), which defines standards for the automotive industry.

For NASA's Curiosity, the following rules had to be abided by:

1. Language Compliance

- Do not stray outside the language definition.
- Compile with all warnings enabled; use static source code analysers.

2. Predictable Execution

- Use verifiable loop bounds for all loops meant to be terminating.
- Do not use direct or indirect recursion.
- Do not use dynamic memory allocation after task initialisation.
- Use IPC messages for task communication.
- Do not use task delays for task synchronisation.
- Explicitly transfer write-permission (ownership) for shared data objects.
- Place restrictions on the use of semaphores and locks.
- Use memory protection, safety margins, barrier patterns.
- Do not use goto, setjmp or longjmp.
- Do not use selective value assignments to elements of an enum list.

3. Defensive Coding

- Declare data objects at smallest possible level of scope.
- Check the return value of non-void functions, or explicitly cast to (void).
- Check the validity of values passed to functions.
- Use static and dynamic assertions as sanity checks.
- Use U32, I16, etc instead of predefined C data types such as int, short, etc.
- Make the order of evaluation in compound expressions explicit.
- Do not use expressions with side effects.

4. Code Clarity

- (a) Make only very limited use of the C pre-processor.
- (b) Do not define macros within a function or a block.
- (c) Do not undefine or redefine macros.
- (d) Place `#else`, `#elif`, and `#endif` in the same file as the matching `#if` or `#ifdef`.
- (e) Place no more than one statement or declaration per line of text.
- (f) Use short functions with a limited number of parameters.
- (g) Use no more than two levels of indirection per declaration.
- (h) Use no more than two levels of dereferencing per object reference.
- (i) Do not hide dereference operations inside macros or typedefs.
- (j) Do not use non-constant function pointers.
- (k) Do not cast function pointers into other types.
- (l) Do not place code or declarations before an `#include` directive.

5. MISRA *shall* compliance

- (a) All MISRA shall rules not already covered at Levels 1-4 (73 rules in total).

6. MISRA *should* compliance

- (a) All MISRA should rules not already covered at Levels 1-4 (16 rules in total).

The rules above are not sufficiently detailed to be understood, but the standard itself [1] (available at http://lars-lab.jpl.nasa.gov/JPL_Coding_Standard_C.pdf) presents a set of well-defined rules, as well as justifications for each rule.

□

5.3 SPARK

SPARK is a subset of Ada, in that any correct SPARK program is also a correct Ada program. However, this is misleading, because SPARK also extends Ada via the use of *annotations*. Annotations are a major part of the language. Thus, $SPARK = safe(Ada) + annotations$.

5.3.1 SPARK Structure

The steps used to construct SPARK is:

- Take Ada, which is a safe language relative to most others.
- Remove unsafe features to get a powerful subset that is still Turing compatible.
- Define a set of annotations to support static checking.

To enable SPARK annotations, we must tell tools that we are using SPARK mode, using syntax such as:

```

1 package IncorrectReassignment with
2     SPARK_Mode => On
3 is

```

SPARK mode can be enabled for packages, individual procedures/functions, or at the project level. By having a standard SPARK mode, this allows any SPARK program to be compiled using a standard Ada compiler. In fact, there is no SPARK compiler — just a set of tools for checking conformance to the SPARK language. If these tools confirm that an Ada program is also a valid SPARK program, then an Ada compiler is used to compile the program.

Chapter 6 presents the use of two types of SPARK annotations: preconditions and postconditions. These are the only annotations we will use in this subject, however, several other useful annotations exist that can be used to cross-check source code.

5.3.2 What is left out of Ada?

SPARK removes many of the features of Ada that are unconsidered either unsafe, or difficult to verify. These include language features such as:

- Dynamic memory allocation
- Tasks
- Gotos
- Exceptions
- Generics
- Access types (similar to references in other languages)
- Recursion

Ada favours the reader heavily over the writer in its design. The reason is that the verification is more difficult and more important than the coding. SPARK carries this principle to further extremes. Why? Because the reader may be a software tool used to verify the program.

Furthermore, writing code takes a fraction of the time it takes to verify, especially in certifiable software.

Why no dynamic memory allocation?

One of the key constraints in SPARK is that memory cannot be allocated dynamically. That is, all arrays etc. must be statically allocated.

The reason for this is for certifiability: if *all* memory is allocated at design time, then we can calculate the maximum amount of memory required to run the program, and ensure that we have enough memory for the program to execute.

If SPARK allowed dynamic memory allocation, there is a possibility that the program could attempt to allocate more memory than is available on the physical platform, which can have catastrophic consequences.

In addition to memory management, static memory allocation makes it straightforward to reason about range correctness; e.g. we can prove that there is no possibility of an array being indexed out of bounds.

Why no tasks?

Tasks are forbidden because tasking programs are inherently more complex than simple sequential programs:

1. The state of a tasking program is a much more complex object than of a non-tasking program.
2. Task interactions are complex. In particular, verifying tasking programs that share memory is exceedingly difficult because we can no longer verify a module as a stand-alone state machine — its internal data may not be hidden.

While SPARK does not allow tasks, *RavenSPARK 95*, a superset of SPARK, does allow tasks (although with strong restrictions), and also supports verification of these.

Why no gotos?

Gotos are not really needed in most programming: at best they are only a minor convenience; but they complicate formal analysis.

When using gotos, the effect of a sequence of code can no longer be represented as the composition of the effects of its sequential components. In other words, they violate the principle of *structured programming*.

Why no exceptions?

Because exceptions make the control flow of a program much more complex. Similar to goto statements, they violate the principle of structured programming.

Further to this, is the issue of what happens when an exception occurs. *Certifiable* programs cannot have unexpected exceptions being thrown. For example, software responsible for controlling part of an aircraft cannot throw an exception and exist.

This implies that all exceptions must be handled using exception handlers. However, exception handlers lead to deactivated code which is problematic for certification.

To work around the idea of not using exceptions, return values from sub-programs should be used to signal exceptional cases, which can then be handled safely.

Why no generics?

Generics are a shorthand for repeated instantiations, so do not provide any fundamental expressive power. But proving properties is more complex because we have to quantify over types. That is, we must prove that the properties we want verified hold for all instantiations, rather than just one.

If we have to prove separate instantiations for certifiable software, it is generally faster and easier to just write all of the instantiations than it is to use generics. So, this creates a more lengthy task at the programming level, but this is clawed back by making verification more straightforward.

Why no access types?

Access types only make sense in connection with dynamic storage allocation, but dynamic allocation is forbidden.

Why no recursion?

Recursive programs are difficult to reason about. In terms of programs, all non-recursive programs can be mapped to *regular* expressions, while some recursive programs cannot. This is somewhat of a violation of the principles of structured programming (but not really).

But more importantly, in the absence of recursion, the maximum stack depth required for a program can be statically determined at design time. Once recursion is introduced, variables can be declared at each call, meaning the memory management must be dynamic – something that is not permitted in SPARK.

Other things left out

Some other language features that are omitted in SPARK:

1. Operation overloading: the number of possible combinations of bindings when operations are overloaded/overridden can be challenging to reason about, and can cause unexpected behaviour.
2. Functions with side effects: any sub-program that needs to modify its input or a global must be a procedure. Functions can only read global variables or parameters.
3. Implicit subtypes; e.g. `X : Integer range 1..10;` are not allowed. We instead must declare separate subtype:

```
subtype XRange is Integer range 1..10;  
X : XRange;
```

This is to prevent designers/programmers from specifying the same implicit subtypes for multiple variables. It is difficult to determine whether these implicit subtypes are intended to represent the same logical concept, or different concepts. Allowing implicit subtypes also presents a maintenance issue: if the range changes, which implicit subtypes need to be changed?

5.3.3 Why bother with SPARK or other safe programming language subsets?

A question to ask is: why bother with safe programming language subsets? Doesn't SPARK make programmers "less productive" by forcing them to declare new subtype ranges, etc.?

The answer is: "yes, *it probably does, but ...*".

It may be true that it takes a bit longer to write one feature of a system, *but*, that time is more than gained back in the verification, and even debugging, of the program. Using the SPARK toolset (which we will use in workshops), we can eliminate a number of possible faults without having to even execute the software.

Testing typically takes about 50% of total effort in a software product. For a certifiable system, verification (including testing) takes up even more. By taking some extra time during programming, we can cut down the verification time significantly. Further to this, we can *certifiably* verify our system.

If we look at the features of SPARK and compare them to the C programming standard used on NASA's Mars Rover *Curiosity*, we can see that many of the Ada features not present in SPARK are those same features not permitted in that coding standard. Thus, SPARK is a formal definition of a language that forbids many programming language features that are considered unsafe or problematic, with a set of mature tools that are able to prove many other problematic faults are not in a program.

5.4 SPARK Examiner

In this section, we present the *SPARK examiner* tool, which uses the annotations to prove that certain properties are (or are not) present in a program.

5.4.1 Annotations

Annotations are Ada comments, starting with `--#`. Like comments, they specify some form of expected/intended behaviour of a program, however, they are *formal* comments and are strict elements of the SPARK syntax. By “formal”, we mean that the comments have a strict syntax and semantics. The syntax and semantics are designed to be both human and machine readable.

As well as making code clearer at the specification level, annotations help with verification by:

1. introducing redundancy (annotations + code), which can be checked for consistency using static analysis; and
2. allowing error checks to be made.

In this chapter, the two types of annotations we will look at are to do with variable declaration and information flow. These are the only two annotations that are compulsory in a SPARK program. These annotations are attached to sub-programs. The two annotations each use their own reserved keywords:

- `global`: this specifies which global variables (those declared outside of the procedure) are used in a function.
- `derives`: specifies which variables are used to derive the value of other variables.

The annotations describe information about the program. Using the program code, we can infer the same information about which variables change and how their value is determined, and cross check this against the annotations. SPARK Examiner is a tool that supports this for SPARK.

5.4.2 SPARK Examiner

SPARK examiner is a tool for statically analysing SPARK programs. It does the following:

- checks syntactic validity of a program;
- checks annotations for consistency and correctness;
- generates useful warnings for debugging; and
- generates auxiliary information helpful in analysis and certification.

One of its primary purposes is to check that a program is a valid SPARK program (as opposed to just an Ada program). SPARK examiner is run prior to compilation. If it produces no warnings or errors, then the program can be compiled by an Ada compiler.

We will investigate SPARK Examiner in greater detail both in later chapters and in workshops. The following provides some examples of how the SPARK Examiner can check internal consistency of a SPARK program.

Example 5.2

The following code listing shows a second attempt at implementing the `Swap` procedure. In this example, a local variable `T` is used as the temporary value holder instead of a global variable.

Listing 5.1: `incorrectassignment.ads`

```
1  procedure Swap (X, Y : in out Float);
```

Listing 5.2: `incorrectassignment.adb`

```
1  procedure Swap (X, Y : in out Float) is  
2      T : Float;  
3  begin  
4      T := X;  
5      X := Y;  
6      Y := T;  
7  end Swap;
```

There is a fault in this program: the final line assigns `X` to `Y`, instead of assigning `T` to `Y`, so the values do not get swapped correctly. Running this through SPARK examiner would result in the following five problems being identified:

1. `T := X` is an “ineffective statement”: the assigned value of `T` is never used.
2. Importation of initial value of `X` ineffective: the initial value of `X` is never used, so declaring it as an `in` parameter is not consistent with the program.
3. The variable `T` is neither referenced nor exported: `T` is declared and is given a value, but that value is never used, and `T` is not an output variable.
4. Imported value of `X` not used in deriving the value of `Y`: SPARK examiner reasons that the value of `X` is not used in deriving `Y`. Despite the final assignment stating doing exactly this, SPARK

examiner is sophisticated enough to reason that the value of *X* is assigned the line before this, so the *imported* value of *X* is not used to derived *Y*, which is inconsistent with the *derives* annotation.

5. Imported value of *Y* may be used in the derivation of *Y*: as a result of the fault, the value of *Y* is derived from itself, which is inconsistent with the *derives* annotation.

□

The above example illustrates several checks that are performed by SPARK examiner. All five errors can be attributed to the same single fault in the final assignment statement.

Exercise 5.1

The following code listing presents a non-sensical example of the “swap” program, in which the assignment of the value of *X* to the variable *X* only occurs if *X* < *Y*. This is not sensible behaviour, but is included just to demonstrate the features of SPARK examiner.

Listing 5.3: weirdswap.ads

```
1  procedure Swap (X, Y : in out Float);
```

Listing 5.4: weirdswap.adb

```
1  procedure Swap (X, Y : in out Float) is
2      T : Float;
3  begin
4      if X < Y then
5          T := X;
6      end if;
7      X := Y;
8      Y := T;
9  end Swap;
```

What is the issue with this procedure?

□

Exercise 5.2

The following example of the *Swap* is more sensible than the last, but the additional code (initialising the variable *T*) will generate a warning for the SPARK examiner.

Listing 5.5: initialiselocal.ads

```
1  procedure Swap (X, Y : in out Float);
```

Listing 5.6: initialiselocal.adb

```
1  procedure Swap (X, Y : in out Float) is
2      T : Float := 0.0;
3  begin
4      T := X;
5      X := Y;
6      Y := T;
7  end Swap;
```

What is the issue with this procedure? □

Exercise 5.3

The two exercises above and Example 5.2 each contain specific (and different) types of faults. Do the three issues on these last three programs seem familiar? HINT: You may need to think back to SWEN90006.

□

Bibliography

- [1] JPL institutional coding standard for the C programming language. JPL DOCID D-60411, March 2009. http://lars-lab.jpl.nasa.gov/JPL_Coding_Standard_C.pdf.

Chapter 6

Design by contract

In this chapter, we present the concept of *Design by Contract* (DbC) – a paradigm for designing software. The paradigm prescribes that designers should specify the interfaces between software components using formal, mathematical statements, such as preconditions, postconditions, and invariants. These statements serve as both documentation, as well as statements that can be reasoned about automatically to provide verification assistance.

SPARK supports DbC using its advanced annotation system, and the tool support for SPARK contracts are particularly mature.

The learning outcomes of this part of the subject are:

1. To understand the motivation, advantages, and disadvantages of design by contract.
2. To specify formal contracts for small- and medium-scale packages in SPARK.

6.1 Introduction to Design by Contract

Design by Contract (DbC) [2] is a design paradigm specifying how sub-systems or components of a system interact with each other. The paradigm is based on the metaphor of “clients” and “suppliers”, who have *obligations* for and receive *benefits* from (respectively) a mutual *contract*.

6.1.1 History

The DbC paradigm was developed by Bertrand Meyer, while a professor University of California, Santa Barbara. In concert with DbC, Meyer and colleagues developed Eiffel — a programming language that embodies the very spirit of DbC.

For his efforts on Eiffel and DbC, Meyer was presented with the 2006 ACM Software System Award:

“For designing and developing the Eiffel programming language, method and environment, embodying the Design by Contract approach to software development and other features that facilitate the construction of reliable, extendable and efficient software.” — 2006 ACM Software System Award Citation.

The Eiffel language and tools to support DbC reasoning are provided by the Eiffel Software company (<http://www.eiffel.com>).

6.1.2 Some Definitions

Before we go further, we cover some standard software engineering definitions that are especially important for DbC:

1. *Information hiding* is a design principle that specifies that data types and implementation should be *abstracted* and hidden behind a publicly available interface.
2. *Encapsulation* is a strategy for information hiding that bundles related entities together behind an interface.
3. An *abstract data type* (ADT) is some package/module/class that encapsulates data and operations on that data, and hides the data types and implementation behind an *interface*.

Exercise 6.1

Explain why these concepts are important in software engineering. □

6.2 Specifying ADT interfaces

In the above definitions, *interfaces* play a key role. In modern programming languages, interfaces are an important part of providing abstraction via information hiding. Most software library interfaces and many components and module interfaces in larger packages document their interfaces, so that a user of the interface can understand what services it provides, and how it must provide inputs to receive correct outputs.

6.2.1 ADT interfaces as natural-language comments

Many ADT interface specifications are documented using comments at the top of modules and operations. These comments, if crafted well, provide users (the developers using the code) with information as to which parts to use and how to use them. They also provide the developers of the module itself with a design against which to implement and verify their code.

Example 6.1

Consider the following example of a Java class interface that implements the behaviour of a bounded stack of integers.

```
1 //Implements a bounded stack data type.
2 //The stack must also be less than a specified size.
3 public class StackImpl implements IStack
4 {
5     //Create a new empty stack, with a maximum size, which
6     // must be greater than 0
7     public StackImpl(int size);
8
9     //Add Object o to the top of the stack
```

```

10 // if the stack is not full.
11 public void push(Object o);
12
13 //Return and remove the object at the top of the stack
14 // if the stack is not empty.
15 public Object pop();
16
17 //Return the size of the stack.
18 public int size();
19 }

```

□

6.2.2 ADT interfaces as invariants, preconditions, and postconditions

While specifying the behaviour of an interface using natural language comments provides users with information to use the class/module, a more structured manner of doing so is using *invariants*, *preconditions*, and *postconditions*.

Recall from Chapter 3, the following definitions:

1. A *precondition* is a statement that must hold for an operation to execute. If the precondition does not hold, the behaviour of the operation is undefined.
2. A *postcondition* is a statement that must hold after a operation executes, provided that the precondition was true before the operation was executed.
3. An *state invariant* (or just *invariant*) is a statement that must hold at all times. It must hold before any operation is executed, and must also hold after that operation is executed. That is, all operations must preserve the invariant.

An invariant can be violated *during* execution of an operation, provided that the invariant is re-established at the end of the operation.

Preconditions, postconditions, and invariants say “what” must hold, but not “how” this must be achieved.

Example 6.1 specifies the behaviour of a Java class, in which the data types and implementation are hidden. However, is it clearer to specify this as preconditions and postconditions?

Example 6.2

The following example specifies the behaviour of the Java class from Example 6.1 using preconditions and postconditions. In this example, the keyword `requires` is used to denote the precondition, and the keyword `ensures` is used to denote the postcondition.

```

1 //Implements a bounded stack data type.
2 public class StackImpl implements IStack
3 {
4     //invariant: The stack must be less than a specified size.
5
6     //ensures: Creates a new empty stack, with a maximum size, which
7     // must be greater than 0
8     public StackImpl(int size);

```

```

9
10 //requires: the stack is not full.
11 //ensures: Object o is added to the top of the stack.
12 public void push(Object o);
13
14 //requires: the stack is not empty
15 //ensures: return and remove the object at
16 //         the top of the stack
17 public Object pop();
18
19 //requires: none (or true)
20 //ensures: return the size of the stack.
21 public int size();

```

□

The above example does not provide any additional information than that of the interface specification from Example 6.1. However, the structured comments are preferable for two reasons:

1. The separation of precondition and postcondition make it clearer to both developers and users of the module what is required from the calling code — that the precondition holds before a call is made to the operation — and what is required from the implementation of the operations — that the postcondition holds after a call is made to the operation.

Establishing the former is the obligation of the user, while establishing the latter is the obligation of the developer.

2. The requirement to provide an invariant, precondition, and postcondition (especially the first two) force the designer to consider these issues — that is, what is invariant about the module and what the preconditions of the operations are? As such, we are more likely to end up with a more complete interface specification.

6.2.3 ADT interfaces as contracts

Software systems are viewed as a set of interacting components. Interaction occurs via calls to operations defined in an interface; in many cases, an ADT interface. When a call is made, the caller is expected to ensure that the input is of the required format, and the callee is expected to ensure that it provides a correct output with respect to the input.

DbC leverages this by using the metaphor of a *contract*.

Contracts

In business, a *contract* is an agreement between two or more parties with mutual obligations and benefits. For example, a contract between a client and supplier of a product has the following properties:

	Obligation	Benefit
Client	Pay for product	Receives product
Supplier	Provide product	Receives income

The DbC paradigm uses this metaphor in software: a module is the supplier and the calling code is the client. The supplying module must perform some computation, and in return the calling code must ensure that the input values sent to the supplying module are correct.

Contracts and software

Using contracts on interfaces ensures that interaction between components is governed by contracts. In total, a contract for an interface should consist of at least the three following items:

1. preconditions for all operations — what the contract expects;
2. postconditions for all operations — what the contract guarantees; and
3. an invariant — what the contract maintains.

Using the metaphor of a contract in software design, we can reflect the above table as follows:

	Obligation	Benefit
Client	Establish precondition	Receives result of computation
Supplier	Provide result of computation	Has precondition established

Expanding the above:

1. The *precondition is the obligation of the client*: before calling any operation, the client must ensure that the precondition of that operation holds. For example, if calling a binary search operation, it must ensure that the input list is sorted in ascending order.
2. The *postcondition is the obligation of the supplier*: the supplier must change the ADT state and/or return the requested result to the client, as specified by the postcondition. The supplier must also establish the invariant.
3. The *client's benefit* is it has the operation executed, as requested.
4. The *supplier's benefit* is that it can assume that precondition of any executed operation, and the invariant, hold before the operation is executed, therefore simplifying the implementation. In practice, the supplier may sometimes throw exceptions or handle violations gracefully, however, this is not always possible or desirable.

Contracts are bi-directional, and assuming all of the contracts are correct, violation of a contract by either party is a *fault*. Further to this, in the case of a fault, we can assign the responsibility of the fault to a particular module: either the calling module or the called module.

The example of the stack in Example 6.2 demonstrates informal contracts via standard comments. However, can we be more precise?

6.2.4 Formal contracts

So far, we have specified interfaces for ADTs using comments, and in one case, comments structured as invariants, preconditions, and postconditions. However, the DbC paradigm advocates that software designers should define *formal*, *precise* and *verifiable* contracts.

Many languages provide constructs, such as assertions, to establish formal properties of their implementations. The DbC paradigm goes further, stating that such properties are so valuable in establishing software correctness, that they should be a routine part of the software design, and that all modules should have contracts.

Example 6.3

The following example provides a formal contract for the running stack example using *Java Modelling Language* [1]. To distinguish JML contract statements from normal Java comments, the @ symbol is used.

```
1 //Implements a bounded stack data type.
2 public class StackImpl implements IStack
3 {
4     private /*@ spec_public @*/ Object [] elements;
5     private /*@ spec_public @*/ int top;
6     private /*@ spec_public @*/ int maxsize
7
8     /*@ public invariant top >= -1 && top < maxsize;
9
10    /*@ requires size > 0;
11    /*@ ensures maxsize == size && top == -1 && elements != null;
12    public StackImpl(int size);
13
14    /*@ requires top < maxsize - 1;
15    /*@ ensures top == \old(top) + 1 && elements[top] == o;
16    public void push(Object o);
17
18    /*@ requires top >= 0;
19    /*@ ensures top == \old(top) - 1 && \result == elements[\old(top)];
20    public Object pop();
21
22    /*@ ensures \result == top + 1;
23    public int size();
```

The `spec_public` keyword denotes that the variable is public for *specification purposes*. That is, it is seen in any generated documentation, but the corresponding variable remains private/protected to other code.

□

The example above is a formalised version of the same interface in Example 6.2, in which JML is used instead of natural language. The same preconditions, postconditions, and invariants are provided, except in a precise manner.

6.2.5 Advantages and disadvantages of formal contracts

Formal contracts, in particular those that are offered as a native support as part of a programming languages, offer several advantages over natural language comments. These include:

1. They document a precise (unambiguous) specification of behaviour. Further, if contracts are enforced as part of a language (or by project/organisational methods), the formal nature offers many of the same benefits offered by formal specification, as outlined in Section 3.3.

Or, to put it better:

“In practice, it is amazing to see how far just stating what a module should do goes towards helping to ensure that it does it. — Bertrand Meyer.

2. They can be used as runtime assertions. That is, during implementation and testing of the system, when an operation is executed, we can check the precondition, postcondition, and invariant all hold, and raise a runtime error if they do not. The failure of a contract to hold implies that either the code or the contract is wrong.
3. A corollary of the above point is that contracts can be used as test oracle. If we are unit testing a module, we can run large amounts of tests, and use the runtime checking to ensure that the contract is satisfied.
4. Contracts can be used for static analysis. That is, using software tools, we can specify certain properties as part of the contract are satisfied, and check these statically.
5. One way to implement operations with contracts is to assume that the precondition of an operation is satisfied; that is, do not use defensive programming. This has the benefit that there is no “double-up” checking of preconditions. That is, in many cases of a precondition, the calling code will ensure that the precondition holds before execution, and then the called operation will also check this. This results in duplicated effort and execution.

The main disadvantage of contracts is that introducing them into a project/organisation requires some learning overhead, but empirical evidence demonstrates that this overhead is recouped quickly.

6.3 Contracts in SPARK

In Chapter 5, the SPARK programming language was introduced, but SPARK annotations were not discussed in detail. SPARK contains several types of annotation, such as variable dependencies, and global variable declaration. In this subject, we’ll concern ourselves mostly with module invariants, operation preconditions, and operation postconditions, which can be used to specify *SPARK contracts*. While SPARK documentation does not use the term “Design by Contract”, these contracts are well within the spirit of DbC.

6.3.1 Syntax

There are several elements that are required in a SPARK contract for a module, with the most important for this subject being:

1. The `Initializes` list: the variables initialised by the module.
2. The `Pre` (for each operation): the precondition.
3. The `Post` (for each operation): the postcondition.

The SPARK annotation language for preconditions and postconditions contains some additional syntax on top of SPARK expressions to specify contracts.

Arrays and records

```

1  x in A'Range           //range membership
2  A'Update(x => y)       //array updates (where 'x' is an array index)
3  A'Update(x => y)       //record updates (where 'x' is a field in the record)

```

For arrays, there are two main types of operation. The first is “range membership”, in which x is a variable and A is a type mark, and $x \text{ in } A'Range$ denotes that the value of x is in the type mark A .

For arrays, as well as records, the operator $A'Update(x \Rightarrow y)$, in which A is an array/record, x is an index/name, and y is an expression, indicates that $A'Update(x \Rightarrow y)$ is the same as array/record A , except with the index/name x overridden with the new value y . This is the same as function override in set theory.

Logic

SPARK offers standard propositional operators and quantification over type marks (i.e. not over arrays).

```

1  P and Q                //conjunction
2  P or Q                 //disjunction
3  if P then Q            //implication
4  not(P)                 //negation
5  for some x in X => (P(x)) //existential quantification
6  for all x in X => (P(x)) //universal quantification

```

Arithmetic

```

1  +, -, /, *             //plus, minus, division, multiplication
2  x mod y                //modulus
3  x**y                   //exponent
4  <, >=, >, <=           //less than (or equal to), greater than (or equal to)
5  =, /=                  //equals, not equals

```

Return

The `Return` operator denotes the value that a function will return:

```

1  FunctionName'Result    //the return value of function 'FunctionName'

```

The first of these is a standard return: just return an expression. The second, called an *implicit return*, allows the declaration of a new variable (V), and then allows the value of that variable to be constrained. The syntax `return V => P(V)` can be read: “return the value of variable V , *such that* predicate P holds on V .”

Some things to note

1. In a postcondition, V' Old refers to the value of the variable *before* execution, and V to the value after. Thus, the syntax is reversed from Alloy’s syntax of labelling the post-state variable.
2. The `=>` and `->` operators are different: the former comes after a quantification (e.g. `for all`), the latter is logical implication.
3. Any valid SPARK expression is a valid expression in a precondition or postcondition. That is, programming-level expressions can be references as part of SPARK predicates.

Example 6.4

This example is taken from the SPARK manual (<http://docs.adacore.com/spark2014-docs/html/lrm/mapping-spec.html>).

The function simply adds two numbers together, but in addition to taking care of the addition, it also prevents integer overflows by specifying, as a precondition, that the inputs cannot lead to an overflow:

```

1  function Add (X, Y : Integer) return Integer
2      with Pre  => (if X >= 0 and Y >= 0 then X <= Integer'Last - Y
3                  elsif X < 0 and Y < 0 then X >= Integer'First - Y),
4      Post => Add'Result = X + Y;
```

The contract specifies that this function must return (that is, the result of the expression `Add'Result`) an integer that adds X and Y . The precondition prevents the overflow. $\square$

Example 6.5

The following is also taken from the SPARK Proof Manual.

Consider a more complex function that swaps two elements in an array:

```

1  procedure Swap_Array_Elements(I, J : Index; A: in out Array_Type)
2      with Post => A = A'Old'Update (I => A'Old (J),
3                                   J => A'Old (I));
```

The precondition states that the *new* value of array A should be the old value (A' Old), but with the elements at indices I and J replaced by the values at indices J and I respectively in the *old* array.

$\square$

Example 6.6

The following illustrates linear search as a contract:

```

1  package Linear_Search
2      with SPARK_Mode
3  is
4      type Opt_Index is new Natural;
5
```

```

6  subtype Index is Opt_Index range 1 .. Opt_Index'Last - 1;
7
8  No_Index : constant Opt_Index := 0;
9
10 type Ar is array (Index range <>) of Integer;
11
12 function Search (A : Ar; I : Integer) return Opt_Index with
13   Post => (if Search'Result in A'Range then A (Search'Result) = I
14           else (for all K in A'Range => A (K) /= I));
15
16 end Linear_Search;

```

In this example, types are used to restrict much of the behaviour (e.g. the index of the array), and the postcondition specifies the functional behaviour: if the result is in the range of the array *A*, then the element at the result must be equal to *I* (the element being searched for); otherwise, it must be that the element is not in the array. □

Example 6.7

As a final example, consider the following contract interface, which includes (among others) the functions for adding and swapping presented earlier:

```

1  package Swap_Add_Max
2    with SPARK_Mode
3  is
4    subtype Index      is Integer range 1..100;
5    type Array_Type is array (Index) of Integer;
6
7    procedure Swap (X, Y : in out Integer)
8      with Post => (X = Y'Old and Y = X'Old);
9
10   function Add (X, Y : Integer) return Integer
11     with Pre  => (if X >= 0 and Y >= 0 then X <= Integer'Last - Y
12               elsif X < 0 and Y < 0 then X >= Integer'First - Y),
13           Post => Add'Result = X + Y;
14
15   function Max (X, Y : Integer) return Integer
16     with Post => Max'Result = (if X >= Y then X else Y);
17
18   function Divide (X, Y : Integer) return Integer
19     with Pre => Y /= 0 and (if X = Integer'First then Y /= -1);
20
21
22   procedure Swap_Array_Elements(I, J : Index; A: in out Array_Type)
23     with Post => A = A'Old'Update (I => A'Old (J),
24                                   J => A'Old (I));
25 end Swap_Add_Max;

```

□

6.3.2 Similarity to Alloy

As a brief note, it is important to note the similarities between the Alloy specification language, and the SPARK contract language. These similarities are not coincidence: the combination of languages such as Alloy and SPARK is common (although to date, most applied work has used the Z language instead of Alloy), and the translation from Alloy or Z to SPARK can be quite straightforward for standard precondition/postconditions. That is, systems engineers specify behaviour in Z/Alloy, prove properties about this using tools based on first-order logic, and then translate the specifications into SPARK contracts. The contracts are then implemented in SPARK code, and this code is verified against the contracts. The similarities between languages like Z/Alloy and SPARK make the translation straightforward in many cases.

6.3.3 Tools

The SPARK toolset comes with tools for supporting contracts in SPARK, including the SPARK Examiner, which can be used to check that a contract is a valid SPARK contract.

In the workshops, we will look at a powerful tool chain (as part of the SPARK tools) that can be used for verifying programs against their contracts.

6.4 Additional reading

1. The following offers an excellent tutorial on SPARK contracts, using a real system developed by Altran-Praxis: <http://www.adacore.com/home/products/sparkpro/tokeneer/discovery/>. The notation used in this tutorial is for SPARK versions prior to 2014, but the notation for precondition and postcondition expressions is the same.
2. The following tutorial on SPARK2014 is a great place to start: http://docs.adacore.com/spark2014-docs/html/ug/getting_started.html
3. The following tutorial on SPARK contracts is another good reference for people who have played with the SPARK examiner: <http://docs.adacore.com/spark2014-docs/html/ug/tutorial.html>
4. Bertrand Meyer's textbook, *Object-Oriented Software Construction* [2], is a detailed presentation of the Eiffel programming language, including Design by Contract.

Bibliography

- [1] G. Leavens, A. Baker, and C. Ruby. JML: A notation for detailed design. In *Behavioral Specifications of Businesses and Systems*, pages 175–188. Springer, 1999.
- [2] B. Meyer. *Object-oriented software construction*. Prentice Hall New York, 2nd edition, 2002.

Chapter 7

Reasoning about program correctness

“How do we convince people that in programming simplicity and clarity — in short: what mathematicians call “elegance” — are not a dispensable luxury, but a crucial matter that decides between success and failure? — Edsger W. Dijkstra [3].

In this chapter, we will look at a very powerful piece of work, known as *Hoare logic*, named after its creator, Tony Hoare. This work dates back as far as 1969, when Hoare proposed the first axiomatic basis for computer programming [4]. Taking only a set of few primitives programming constructs (now considered to be the basis of structured programming), Hoare demonstrated a simple logic that enabled programs to be *proved* correct. Since that time, much research has gone into building on Hoare’s initial work, with considerable success.

We will look at Hoare logic and show how to use it to reason about the correctness of programs, allowing us to *prove* that a program meets its contract/specification. This is much more powerful than running tests, which constitute nothing more than a handful of observations about our program, and also more powerful than using review to check for a larger set of cases. For some systems, testing and review are not enough.

In this chapter you will learn how to prove programs correct *manually*, i.e. by hand, with pen and paper. The techniques you will learn are essentially the same ones that are applied *automatically* by tools like the SPARK Prover when it automatically proves a procedure or function meets its contract (see Section 6.3). The reason that you learn these techniques manually is not only to gain an understanding of how tools like the SPARK Prover work, but also because understanding *how* you prove a program correct changes the way you think about programming.

It is my belief that the (manual) techniques presented in this chapter are the most important of the subject — yet are also the techniques that I can confidently state none of you will ever apply as they are applied in these notes. The importance of this chapter is to change the way that you think about programs, the science of programming, and of software design and verification, in order to make you better programmers and software engineers.

Learning outcomes

The learning outcomes of this chapter are:

1. To critique the difference between program proof and other forms of verification, such as testing.

2. To be able to prove the correctness of simple programs against their contracts.
3. To think about programming in a more systematic and mathematical manner, with the result of improving our programming skills.

7.1 Introduction to reasoning about programs

The first thing to ask is why we would want to reason about programs at all. In previous chapters, we have looked at how to formally specify and design (by contract) our system, and to prove that the specification and design satisfy certain properties. Proof of formal statements are possible because we are operating in space of logic and set theory, which have well-founded proof theories.

However, for high integrity systems, we want to ensure that our designs are implemented correctly. That is, we want to show that our code corresponds to our design.

Quality assurance techniques such as peer review and testing seem to offer very good reliability for software. However, with respect to high-integrity constraints, they do not offer the level of assurance that is required. What if we have missed a few cases that can cause catastrophic consequences?

This chapter covers the theory of how we can *prove* that a program meets its contract. This is a powerful ability that provides us with assurance far above what is possible with testing and review.

7.1.1 The correctness of binary search

Let's motivate the idea of reasoning about programs using the following story from Jon Bentley:

“Binary search solves the problem [of searching within a pre-sorted array] by keeping track of a range within the array in which T [i.e. the sought value] must be if it is anywhere in the array. Initially, the range is the entire array. The range is shrunk by comparing its middle element to T and discarding half the range. The process continues until T is discovered in the array, or until the range in which it must lie is known to be empty. In an N-element table, the search uses roughly $\log(2) N$ comparisons.

Most programmers think that with the above description in hand, writing the code is easy; they're wrong. The only way you'll believe this is by putting down this column right now and writing the code yourself. Try it.

I've assigned this problem in courses at Bell Labs and IBM. Professional programmers had a couple of hours to convert the above description into a program in the language of their choice; a high-level pseudocode was fine. At the end of the specified time, almost all the programmers reported that they had correct code for the task. We would then take thirty minutes to examine their code, which the programmers did with test cases. In several classes and with over a hundred programmers, the results varied little: ninety percent of the programmers found bugs in their programs (and I wasn't always convinced of the correctness of the code in which no bugs were found).

I was amazed: given ample time, only about ten percent of professional programmers were able to get this small program right. But they aren't the only ones to find this task difficult: in the history in Section 6.2.1 of his *Sorting and Searching*, Knuth points out that while the

first binary search was published in 1946, the first published binary search without bugs did not appear until 1962.”

— Jon Bentley, *Programming Pearls* (1st edition), pp. 35-36 [1].

Donald Knuth mentions in *The Art of Computer Programming* [6] that the first version of the binary search algorithm appeared in 1946, but it was not until 1962 that the first *correct* version appeared. All previous versions contained faults.

Further, the most well-known implementation, written and “*proved*” by Jon Bentley, which has been adapted many times, contains a fault. The truth is, very few correct versions of the algorithm exist.

Exercise 7.1

There is a fault in the following widely-tested and widely-used version of binary search. Find this fault. TIP: Converting to SPARK and using the SPARK tools to verify the program will reveal the fault.

```
1  int binarySearch(int [] list, int target)
2  {
3      int low = 0;
4      int high = len(list) - 1;
5      int mid;
6
7      while(low <= high) {
8          mid = (low + high)/2;
9          if (list[mid] < target) {
10             low = mid + 1;
11         }
12         else if (list[mid] > target) {
13             high = mid - 1;
14         }
15         else {
16             return mid;
17         }
18     }
19     return -1;
20 }
```

□

The question then is: if some of the best and brightest minds in computer science cannot get a seemingly simple algorithm correct using testing, and dozens of versions in textbooks can be published with faults that remain undetected for decades, what hope do we have for producing correct software the first time?

In this chapter, we will look at Hoare logic as one approach for proving a program correct with respect to its specification. In workshops, we will look at powerful support provided by the SPARK tools that can assist us in this goal.

7.2 A small programming language

Before we look at Hoare logic, we’ll define a small structured programming language that is Turing compatible, and which will form the basis of the mathematical objects in Hoare’s logic. The language

consists of procedures, variables, expressions (numerical expressions and arrays of expressions), variable assignment, sequencing, conditionals, loops, and procedure calls:

Procedures:

$P \quad := \quad \text{procedure } p(v_1, \dots, v_n) \hat{=} S$

where $v_1, \dots, v_n$ are variable names, p is a procedure name, and S is a program statement (defined below).

Expressions:

$E \quad := \quad NE \mid a[NE]$

$NE \quad := \quad n \mid v \mid E_1 + E_2 \mid E_1 - E_2 \mid E_1 \times E_2 \mid E_1 / E_2 \mid \text{etc.} \dots$

where n is a number, v is a variable, a is an array variable, NE is a numerical expression, and E an expression.

Booleans

$B \quad := \quad \text{true} \mid \text{false} \mid \neg B \mid E_1 = E_2 \mid E_1 \leq E_2 \mid \text{etc.} \dots$

Statements

$S \quad := \quad \text{skip} \mid v := E \mid a[NE] := E \mid p(E_1, \dots, E_n) \mid S_1; S_2 \mid$
 $\quad \text{if } B \text{ then } S_1 \text{ else } S_2 \text{ endif} \mid \text{while } B \text{ do } C \text{ done}$

where v is a variable, a is an array variable, and p is a procedure name.

The **skip** statement is a statement that has no effect and always executes successfully, the same as `null` in Ada.

A *program* in this language consists of a (possibly empty) sequence of procedure definitions, and a final “main” program; e.g.:

procedure $p_1(v_1, \dots, v_m) \hat{=} S_1$
 $\dots$
procedure $p_n(v_1, \dots, v_n) \hat{=} S_n$
 S

where S is the main program, which may reference procedures $p_1, \dots, p_n$.

Using this simple language, we can construct any program that possible with more expressive languages, such as those with references, but the program may be more long-winded.

Example 7.1

Using this language, we can construct a program that calculates the factorial of 10 and assigns it to a variable f (where n is an input variable and f is an output variable):

Algorithm 1 A program for calculating the factorial of a number

procedure FACTORIAL(**in** n , **out** f)

$f := 1$

$i := 0$

while $i \neq n$ **do**

$i := i + 1$

$f := f \times i$

end while

end procedure

FACTORIAL(10, f)

□

7.3 Hoare logic

Hoare logic [4] is an axiomatisation of programming. Hoare's aim was to treat computer programs as mathematical objects, and to provide a sound and complete set of rules for proving things about these objects.

7.3.1 Introduction to Hoare logic

Hoare logic is a set of rules for reasoning about programs. The rules are organised around the program statements in the simple programming language defined above. Any program in the language is a collection of statements, as defined by the grammar S , with a (possibly empty) set of procedures that it references.

The rules are applied to the program in a top down manner, until the final result is the application of rules to atomic statements: **skip** or variable assignments ($V := E$). The rules for compound program statements, such as sequencing and branching, require us to prove properties about the statements that make up the compound statement.

Statements about programs are written using *Hoare triples*, which take the following form:

$$\{P\} S \{Q\}$$

This states that, given a program S with precondition P , if P is true when S is executed, then S will establish the postcondition Q . In other words, a Hoare triple describes how executing a program changes the variables in the program.

Hoare logic is a collection of axioms and rules of this form that can be used to reason about the correctness of programs; similar to using Peano's axioms to reason about numbers. To prove correctness of our program, we need to show that $\{P\}S\{Q\}$ is true, using Hoare's axioms and rules.

The rules are written as inference rules, of the form:

$$\frac{H_1, \dots, H_n}{C}$$

where $H_1, \dots, H_n$ are a collection of hypotheses, and C is the conclusion. An inference rule should be read as: if hypotheses $H_1, \dots, H_n$ can all be proved, then the conclusion C is true. Thus, to prove C , we must prove $H_1, \dots, H_n$.

For example, consider the most famous inference rule of all — *modus ponens*:

$$\frac{A, A \Rightarrow B}{B}$$

This states that, if A is true and $A \Rightarrow B$ is true, then we can infer that B must be true.

An inference rule with no hypotheses is called an *axiom*, and is often just written as the conclusion with no line. For example, the following is an axiom of propositional logic:

$$A \Rightarrow (B \Rightarrow A)$$

That is, this proposition is always true. This could be written as:

$$\frac{true}{A \Rightarrow (B \Rightarrow A)}$$

but the line and premise are often omitted for readability.

In Hoare logic, a hypothesis can be either a predicate or a Hoare triple, and the conclusion is always a Hoare triple.

7.3.2 An introductory example — Incrementing an integer

Consider the following SPARK program used to increment a variable:

```

1  procedure Inc (X: in out Integer)
2  with
3      Pre => (X >= 0) ,
4      Post => (X > 0) ;
5  is begin
6      X := X + 1;
7  end Inc;
```

Can we *prove* that this program establishes its postcondition for all X that satisfy the precondition? It may seem quite easy for us to reason about this in our head to convince ourselves that the program is correct, however, how could we go about automating or mechanising this?

7.4 The rules of Hoare logic

In this section, we present the fundamental rules of Hoare logic, and present several examples.

7.4.1 Assignment axiom

The assignment axiom is specified as a triple:

$$\{P[E/x]\} x := E \{P\}$$

where $P[E/x]$ represents substituting all occurrences of the variable x with expression E in P . Note that this rule has no hypotheses, just a conclusion, so it is an axiom rather than an inference rule.

This rule says that, if we want to prove that P is true after executing the statement $x := E$, then we need to prove that $P[E/x]$ is true before executing $x := E$.

When using this rule on a program proof, we *derive* $P[E/x]$ from the program and the postcondition. $P[E/x]$ is called the *weakest precondition* of S that establishes P . That is, it is the weakest, or more general, predicate for which program S establishes the postcondition P .

Example 7.2

The following are two simple Hoare triples that are true:

$$\begin{aligned} &\{y = 0\} x := y \{x = 0\} \\ &\{5 + y \geq 0\} x := 5 \{x + y \geq 0\} \end{aligned}$$

In both triples above, the variable x has been substituted by the assigned expression to form the precondition.

□

Example 7.3

So, using the assignment axiom, can we prove that our program `INC` establishes its postcondition?

The statement we need to prove is the following:

$$\{x \geq 0\} x := x + 1 \{x > 0\}$$

From Hoare's assignment axiom, we derive the weakest precondition of the program $x := x + 1$ from the postcondition $x > 0$. If we substitute $x + 1$ for all occurrences of x in $x > 0$, we are left with $x + 1 > 0$. Therefore, using the assignment axiom, the following statement is true:

$$\{x + 1 > 0\} x := x + 1 \{x > 0\}$$

However, the precondition of this statement is not the same (syntactically) as the precondition of the original program. To finish the proof, we need to introduce another rule, which we do now.

□

7.4.2 Consequence rule

Hoare's consequence rule is as follows:

$$\frac{P' \Rightarrow P, Q \Rightarrow Q', \{P\} S \{Q\}}{\{P'\} S \{Q'\}}$$

This states that, if we can establish that $\{P\} S \{Q\}$ is true, then any precondition that is at least as *strong* as P , and any postcondition that is at least as *weak* as Q , are also valid preconditions/postconditions for the program S .

Example 7.4

Some examples:

$$\begin{aligned} &\{x \geq 0\} x := x + 1 \{x > 0\} && \text{(because } x \geq 0 \Rightarrow x + 1 > 0\text{)} \\ &\{5 + y \geq 0 \wedge z = 0\} x := 5 \{x + y \geq 0\} && \text{(a strengthened postcondition)} \\ &\{5 + y \geq 0\} x := 5 \{true\} && \text{(a weakened postcondition)} \end{aligned}$$

□

7.4.3 Weakest Precondition Reasoning

Suppose we have to prove an arbitrary Hoare triple about some assignment statement: $\{P'\} x := E \{Q\}$. We can prove this by first applying the consequence rule to *generalise* the precondition P' . The consequence rule says that we can prove this triple if we can find some (possibly weaker) precondition $?P$ for which

$$\{?P\} x := E \{Q\}$$

and $P' \Rightarrow ?P$, i.e. the original precondition P' is no weaker than the generalised one $?P$. In general we should choose $?P$ to be the weakest precondition that guarantees Q (to ensure that P' won't be weaker than $?P$).

In fact, the assignment axiom tells us how to compute $?P$, since it computes the weakest precondition $Q[E/x]$ for the postcondition Q . Therefore after applying the consequence rule to generalise the precondition, we then apply the assignment axiom to compute $Q[E/x]$. Finally we check that $P' \Rightarrow Q[E/x]$ and if so we have proved $\{P'\} x := E \{Q\}$ as required.

This is a general structure that we can use to reason about our programs, and is the approach we will follow in this chapter. In particular, the rules we will see in the remainder of the chapter allow us to *compute* a precondition $?W$, given a program statement S and postcondition Q , such that $\{?W\} S \{Q\}$ holds. To prove an arbitrary Hoare triple $\{P\} S \{Q\}$, we simply apply the rules to compute a precondition $?W$ such that $\{?W\} S \{Q\}$ and then check whether $P \Rightarrow ?W$ for the $?W$ we computed.

For programs without loops, $?W$ can be computed entirely *automatically*, and the check whether $P \Rightarrow ?W$ can also be performed automatically via constraint solving (specifically, SMT solving). This is precisely the approach that tools like the SPARK Prover apply to automatically prove that a program satisfies its specification.

For programs with loops, we will see that to derive the precondition $?W$ we need to be given some extra information in the form of a *loop invariant*. So in general we cannot fully automate Hoare logic reasoning. However, we're getting ahead of ourselves. We will come to the rule for loops, and to loop invariants, later in this chapter.

The assignment axiom gives us a clue about *how* we compute the precondition $?P$: by *working backwards* over the program S .

7.4.4 Sequential composition rule

The sequential composition rule allows us to reason about programs of the form $S_1; S_2$, i.e. the program that first does S_1 , and then does S_2 , where S_1 and S_2 are program statements.

Suppose we are given a postcondition Q . How can we compute a precondition such that executing $S_1; S_2$ guarantees that Q will hold afterwards? As we said before, the trick is to work backwards. First start with S_2 and then look at S_1 afterwards.

In particular, given S_2 and Q , we can apply the rules of Hoare logic (most of which we haven't seen yet) to compute a condition R such that $\{R\} S_2 \{Q\}$ holds. This tells us that if we begin executing S_2 in a state in which R holds, then afterwards we know Q will hold. Then, having got R , all that remains is to use the rules of Hoare logic to compute a precondition P such that $\{P\} S_1 \{R\}$ holds.

This works because, if we have $\{P\} S_1 \{R\}$ we know that if we execute S_1 in a state satisfying P , then afterwards R will hold. We also know that $\{R\} S_2 \{Q\}$, i.e. that S_2 will give us Q when executed

from a state satisfying R . Therefore we can conclude that if we execute S_2 after executing S_1 in a state satisfying P , then Q will hold at the end, because we know that R will hold in-between.

This gives us the rule for reasoning about sequential composition.

$$\frac{\{P\} S_1 \{R\}, \{R\} S_2 \{Q\}}{\{P\} S_1; S_2 \{Q\}}$$

Example 7.5

Let's apply this rule to reason about a simple program that swaps two integers x and y , using a temporary variable t :

```
t := x;
x := y;
y := t
```

The statement we wish to prove is that this program correctly swaps the values of x and y . Let us assume that the initial value of x is some arbitrary value X ; and the initial value of y is Y . Here we are using the *logical* variables X and Y to represent the initial values of the *program variables* x and y respectively. Then one way to specify that the swap program works correctly is to prove this Hoare triple:

Program Specification (PS)

```
{x = X ∧ y = Y}
t := x;
x := y;
y := t
{x = Y ∧ y = X}
```

As stated above, we begin by applying the consequence rule to generalise the precondition to some (as yet unknown) precondition $?W$, giving us two goals to prove:

$$x = X \wedge y = Y \Rightarrow ?W$$

and

```
{?W}
t := x;
x := y;
y := t
{x = Y ∧ y = X}
```

We abbreviate these two goals by writing simply:

```
{x = X ∧ y = Y}
{?W}
t := x;
x := y;
y := t
{x = Y ∧ y = X}
```

The first two lines should be read as saying that the first one needs to imply the second one. The second line to the sixth line should be read as saying the triple above needs to hold also.

We work backwards.

We begin by applying the Hoare logic rules to compute $?W$ such that the bottom triple holds. After computing $?W$ we then check the first implication.

This program is the sequential composition of 3 assignment statements. Because sequential composition (“;”) is associative we can think of this 3-statement program as a program with two statements, the first of which is the statement $t := x$ and the second of which is $x := y; y := t$. Thinking about it this way, we first apply the sequential composition rule which tells us that we can prove this Hoare triple above by finding an intermediate condition $?R$ such that:

$$\begin{array}{l} \{?W\} \\ t := x \\ \{?R\} \\ x := y; y := t \\ \{x = Y \wedge y = X\} \end{array}$$

We write the intermediate condition as “ $?R$ ” with a question mark to remind ourselves that we haven’t actually figured out what it is yet.

Note here that we are concatenating two Hoare triples as shorthand, such that for all P, R, Q , and S_1 and S_2 ,

$$\{P\} S_1 \{R\}, \{R\} S_2 \{Q\}$$

is equivalent to

$$\{P\} S_1 \{R\} S_2 \{Q\}.$$

Therefore we really have to find a condition $?R$ that makes the following two Hoare triples true:

$$\begin{array}{l} \{?W\} \\ t := x \\ \{?R\} \end{array}$$

and

$$\begin{array}{l} \{?R\} \\ x := y; y := t \\ \{x = Y \wedge y = X\} \end{array}$$

We work backwards, i.e. we begin by working on the *second* of these two triples. Specifically, we apply sequential composition rule. This tells we need to find some intermediate condition $?R_2$ such that:

$$\begin{array}{l} \{?R\} \\ x := y \\ \{?R_2\} \\ y := t \\ \{x = Y \wedge y = X\} \end{array}$$

Before we go any further, let’s put together all of the goals we have to prove. Using the abbreviations, it looks like this:

$$\begin{array}{l} \{x = X \wedge y = Y\} \\ \{?W\} \\ t := x; \\ \{?R\} \end{array}$$

$$\begin{array}{l}
x := y; \\
\{?R_2\} \\
y := t \\
\{x = Y \wedge y = X\}
\end{array}$$

Compare this to the original Hoare triple **(PS)** about this program. Notice that by applying the consequence rule and then the sequential composition rule we have essentially inserted the unknown intermediate conditions between each of the top-level statements.

In future when proving programs we will do this automatically, without explicitly applying these rules.

What we now need to do is work backwards: first compute $?R_2$, then from that compute $?R$, and then $?W$ and finally check the first implication.

To compute $?R_2$ we again work backwards and apply the assignment axiom to the final assignment statement here, giving us:

$$\begin{array}{l}
\{x = X \wedge y = Y\} \\
\{?W\} \\
t := x; \\
\{?R\} \\
x := y; \\
\{x = Y \wedge t = X\} \\
y := t \\
\{x = Y \wedge y = X\},
\end{array}$$

Having found $?R_2$ we continue to work backwards by again applying the assignment axiom to compute $?R$:

$$\begin{array}{l}
\{x = X \wedge y = Y\} \\
\{?W\} \\
t := x; \\
\{y = Y \wedge t = X\} \\
x := y; \\
\{x = Y \wedge t = X\} \\
y := t \\
\{x = Y \wedge y = X\},
\end{array}$$

We continue again applying the assignment rule to compute $?W$:

$$\begin{array}{l}
\{x = X \wedge y = Y\} \\
\{y = Y \wedge x = X\} \\
t := x; \\
\{y = Y \wedge t = X\} \\
x := y; \\
\{x = Y \wedge t = X\} \\
y := t \\
\{x = Y \wedge y = X\},
\end{array}$$

Finally we need to check the initial implication, i.e. whether $x = X \wedge y = Y \Rightarrow y = Y \wedge x = X$? Of course this holds trivially since the left- and right-hand sides of the implication are equivalent (because $\wedge$ is commutative).

This completes the proof.

□

7.4.5 Empty statement axiom

The empty statement axiom asserts that the **skip** statement changes nothing, so whatever holds before **skip** is executed will hold afterwards:

$$\{P\} \text{ skip } \{P\}$$

This axiom is trivial, but is important for completeness of the axioms.

Another way to read this axiom is that the weakest precondition that establishes the postcondition P after **skip** executes is P itself.

7.4.6 Conditional rule

The next rule we introduce is for proving programs containing conditional statements. We assume that if-then-else statements are the only conditionals, but other conditionals are just shorthand for if-then-else, and even then, the reasoning rules are much the same. For instance, a one-sided if condition (without an else branch) can be modelled by using **skip** as the else branch of an if-then-else.

The conditional rule tells us how to compute the weakest precondition for **if B then S_1 else S_2 endif** to guarantee that a postcondition Q holds afterwards. It says that we first compute a weakest precondition P_1 for S_1 such that $\{P_1\} S_1 \{Q\}$ holds. We do likewise for S_2 to find a precondition P_2 such that $\{P_2\} S_2 \{Q\}$ holds. Then, having found P_1 and P_2 the weakest precondition for the entire if-then-else statement is $(B \Rightarrow P_1) \wedge (\neg B \Rightarrow P_2)$, i.e. to guarantee that Q will hold afterwards, we need to know that when B is true (and so S_1 will be executed), the weakest precondition P_1 for S_1 will hold, and that when B is false (and so S_2 will be executed), the weakest precondition P_2 for S_2 will hold.

$$\frac{\{P_1\} S_1 \{Q\}, \{P_2\} S_2 \{Q\}}{\{(B \Rightarrow P_1) \wedge (\neg B \Rightarrow P_2)\} \text{ if } B \text{ then } S_1 \text{ else } S_2 \text{ endif } \{Q\}}$$

Example 7.6

Conditional swapping

To illustrate this, consider our swap program again, except that it only swaps the values of x and y if $x < y$. In other words, it establishes the postcondition $x \geq y$:

```

{true}
if  $x < y$  then
     $t := x;$ 
     $x := y;$ 
     $y := t$ 
else
    skip
end if
 $\{x \geq y\}$ 

```

As a side note, the statement **if B then S else skip endif** is equivalent to just **if B then S endif** (that is, removing the **else** branch — a one-armed branch statement), so there is no need to a rule covering this case.

To prove the triple above, we use the conditional rule to derive a weakest precondition $?W$ and then check that $true \Rightarrow ?W$.

```

{true}
{?W}
if  $x < y$  then
     $t := x;$ 
     $x := y;$ 
     $y := t$ 
else
    skip
end if
 $\{x \geq y\}$ 

```

We use the conditional rule to derive $?W$. It says that $?W$ is equal to $(x < y \Rightarrow ?P_1) \wedge (x \geq y \Rightarrow ?P_2)$ where we have to find $?P_1$ and $?P_2$ such that the following two triples hold:

```

{?P1}
 $t := x;$ 
 $x := y;$ 
 $y := t$ 
 $\{x \geq y\}$ 

```

and

```

{?P2}
skip
 $\{x \geq y\}$ 

```

We can write this more succinctly by annotating the original program as follows:

```

{true}
 $\{(x < y \Rightarrow ?P_1) \wedge (x \geq y \Rightarrow ?P_2)\}$ 
if  $x < y$  then
    {?P1}
     $t := x;$ 
     $x := y;$ 
     $y := t$ 
     $\{x \geq y\}$ 
else
    {?P2}
    skip
     $\{x \geq y\}$ 
end if
 $\{x \geq y\}$ 

```

The rule for skip tells us immediately that $?P_2$ is simply $x \geq y$ to make the second triple true.

```

{true}
 $\{(x < y \Rightarrow ?P_1) \wedge (x \geq y \Rightarrow x \geq y)\}$ 
if  $x < y$  then
    {?P1}
     $t := x;$ 
     $x := y;$ 

```

```

      y := t
      {x ≥ y}
    else
      {x ≥ y}
      skip
      {x ≥ y}
    end if
  {x ≥ y}

```

To compute $?P_1$ we have to apply the sequencing rule and the assignment axiom similarly to the previous example. Doing so gives

```

  {y ≥ x}
  t := x;
  {y ≥ t}
  x := y;
  {x ≥ t}
  y := t
  {x ≥ y}

```

and so tell us that $?P_1$ is $y \geq x$. Therefore we have:

```

  {true}
  {(x < y ⇒ y ≥ x) ∧ (x ≥ y ⇒ x ≥ y)}
  if x < y then
    {y ≥ x}
    t := x;
    {y ≥ t}
    x := y;
    {x ≥ t}
    y := t
    {x ≥ y}
  else
    {x ≥ y}
    skip
    {x ≥ y}
  end if
  {x ≥ y}

```

Finally we need to check that *true* implies $(x < y \Rightarrow y \geq x) \wedge (x \geq y \Rightarrow x \geq y)$ which of course holds trivially.

□

7.4.7 Iteration rule

Now, we get to the most difficult-to-apply rule in Hoare logic: the *iteration* rule. As its name says, it is used to reason about iteration in programs. As with conditionals, we assume only one basic iteration operator, the *while* loop, but other type of loops, such as *for* and *do-while* loops, are straightforward extensions from this.

The iteration rule is:

$$\frac{\{P \wedge B\} S \{P\}, \quad P \wedge \neg B \Rightarrow Q}{\{P\} \textbf{while } B \textbf{ do } S \textbf{ done } \{Q\}}$$

To understand this rule, it helps to remember that its job is to reason about iteration in a way that is independent of the number of times a loop executes. A while loop might execute zero times, if its condition B is false when the loop is first encountered. It might execute once if B is initially true but becomes false during the first execution of the loop body S . Or it might execute twice, three times, etc.

When doing a proof about a loop, we want to reason about the loop body only once, even though it might execute zero or many times. To accomplish this, the rule makes use of a piece of information provided by the person who is using the rule (i.e. the person who is proving the program correct), called a *loop invariant*. This is the predicate P in the rule above. Notice that the rule says that P is the precondition for the loop as a whole. So this rule technically doesn't compute the weakest precondition for a loop (in fact doing so in general is impossible). Instead, the person applying the rule has to supply the precondition P and this rule checks that P is a valid precondition for ensuring that the postcondition Q when the loop finishes executing.

The rule checks two things: firstly that if the loop never executes, then Q will hold right away, i.e. that if the loop precondition P holds but the loop condition B is false, then Q is guaranteed to hold: $P \wedge \neg B \Rightarrow Q$.

The second thing it checks is that the loop body S *preserves* P , i.e. that if P holds and the loop body S is executed (because the condition B is true) then P still holds afterwards: $\{P \wedge B\} S \{P\}$. This tells us that no matter how many times the loop executes, P will always continue to hold in between each loop iteration and, therefore, by the time that the condition B becomes false, P will still hold and (from knowing that $P \wedge \neg B \Rightarrow Q$) therefore the postcondition Q is guaranteed to hold after any number of loop iterations.

Reasoning about loops

To reason about a while loop, we must actually calculate the loop invariant. It turns out that this is by far the hardest part about applying Hoare logic.

The loop body change the variables of the program. Our job is to find a relationship between the values of the variables that are preserved by the execution of the loop body. That is, the individual values of the variables may change, but the relationship between them does not.

In general, a loop invariant must do three things:

1. Hold at the beginning of the loop
2. Be preserved by each loop iteration (when the loop guard is true)
3. Imply the loop's postcondition (when the loop guard is false)

Because the loop invariant needs to imply the postcondition, and it needs to relate the variables that the loop body mentions, invariants usually talk about the loop body variables in a way that resembles the loop postcondition.

Example 7.7

To demonstrate the rule as well as the concept of loop invariants, we'll consider the following SPARK program for calculating the factorial of a number:

```
1  procedure Factorial (N : in Integer,  
2                        F : out Integer)  
3  with  
4    Pre => (N >= 0),  
5    Post => (F = Fact(N));  
6  is  
7    I : Integer;  
8  begin  
9    F := 1;  
10   I := 0;  
11   while I /= N do  
12     I := I + 1;  
13     F := F * I;  
14   done;  
15 end Factorial;
```

In this procedure, the expression `Fact (N)` refers to the factorial mathematical function.

To prove this program establishes its postcondition under its precondition, we need to prove the following triple:

```
{n ≥ 0}  
f := 1;  
i := 0;  
while i ≠ n do  
  i := i + 1;  
  f := f * i;  
end while  
{f = n!}
```

where $n!$ is defined as the factorial function:

$$\begin{aligned} 0! &= 1 \\ n! &= n \times (n - 1)! \quad \text{where } n > 0 \end{aligned}$$

First, we need to establish the loop invariant.

The factorial loop invariant

The loop invariant for the factorial program is:

$$f = i!$$

Notice how this invariant satisfies the conditions mentioned above: it is true when we reach the loop (when $i = 0$ and $f = 1$); it is preserved by each loop iteration; and, when the loop guard is false ($i = n$), it implies the loop's postcondition: $f = n!$. It also mentions the loop body variables, i and f , and resembles the loop postcondition.

Back to the proof

The first step is to apply the consequence rule to generalise the precondition, and sequential composition rule to split the program into its constituent parts:

```
1:  $\{n \geq 0\}$ 
2:  $\{?Q\}$ 
3:  $f := 1;$ 
4:  $\{?P\}$ 
5:  $i := 0;$ 
6:  $\{?I\}$ 
7: while  $i \neq n$  do
8:    $i := i + 1;$ 
9:    $f := f * i;$ 
10: end while
11:  $\{f = n!\}$ 
```

Notice that we have called the as yet unknown loop precondition $?I$ because it will also be the loop invariant.

We work backwards, as usual, to first work out what $?I$ should be, by applying the Hoare logic rules. The only rule besides the consequence rule that applies to the final program statement (the while loop) is the iteration rule, in which $?I$ is the loop invariant. Of course we will use the invariant $f = i!$ for $?I$. Therefore we apply that rule, which requires showing that the loop body preserves the invariant and the invariant implies the postcondition when the loop finishes. We add these conditions as annotations on the program:

```
1:  $\{n \geq 0\}$ 
2:  $\{?Q\}$ 
3:  $f := 1;$ 
4:  $\{?P\}$ 
5:  $i := 0;$ 
6:  $\{f = i!\}$ 
7: while  $i \neq n$  do
8:    $\{f = i! \wedge i \neq n\}$ 
9:    $i := i + 1;$ 
10:   $f := f * i;$ 
11:   $\{f = i!\}$ 
12: end while
13:  $\{f = i! \wedge i = n\}$ 
14:  $\{f = n!\}$ 
```

Of course the final implication $f = i! \wedge i = n \Rightarrow f = n!$ holds straightforwardly.

We therefore now need to prove the loop body satisfies its specification.

```
 $\{f = i! \wedge i \neq n\}$ 
 $i := i + 1;$ 
 $f := f * i;$ 
 $\{f = i!\}$ 
```

This is like proving any other Hoare triple: we start by applying the consequence rule to generalise the precondition and the sequence rule to insert intermediate conditions, then work backwards to calculate

those conditions.

$$\begin{aligned} & \{f = i! \wedge i \neq n\} \\ & \{?W_1\} \\ & i := i + 1; \\ & \{?W_2\} \\ & f := f * i; \\ & \{f = i!\} \end{aligned}$$

Working backwards we calculate $?W_2$ and then $?W_1$ using the assignment axiom:

$$\begin{aligned} & \{f = i! \wedge i \neq n\} \\ & \{f * (i + 1) = (i + 1)!\} \\ & i := i + 1; \\ & \{f * i = i!\} \\ & f := f * i; \\ & \{f = i!\} \end{aligned}$$

We need to check the first implication: $f = i! \wedge i \neq n \Rightarrow f * (i + 1) = (i + 1)!$ From the left-hand side, we can rewrite the right-hand side to $i! * (i + 1) = (i + 1)!$ which holds from the definition of factorial. Therefore we have proved the loop body, giving us overall:

```

1:  {n ≥ 0}
2:  {?Q}
3:  f := 1;
4:  {?P}
5:  i := 0;
6:  {f = i!}
7:  while i ≠ n do
8:      {f = i! ∧ i ≠ n}
9:      {f * (i + 1) = (i + 1)!}
10:     i := i + 1;
11:     {f * i = i!}
12:     f := f * i;
13:     {f = i!}
14: end while
15: {f = i! ∧ i = n}
16: {f = n!}

```

We now need to reason about the program before the loop, continuing to work backwards to calculate $?P$ and then $?Q$. Applying the assignment axiom gives first $?P$ as $f = 0!$ and then $?Q$ as $1 = 0!$:

```

1:  {n ≥ 0}
2:  {1 = 0!}
3:  f := 1;
4:  {f = 0!}
5:  i := 0;
6:  {f = i!}
7:  while i ≠ n do
8:      {f = i! ∧ i ≠ n}
9:      {f * (i + 1) = (i + 1)!}
10:     i := i + 1;

```

```

11:    {f * i = i!}
12:    f := f * i;
13:    {f = i!}
14:  end while
15:  {f = i! ∧ i = n}
16:  {f = n!}

```

Finally it remains to check $n \geq 0 \Rightarrow 1 = 0!$ which holds from the definition of factorial, completing the proof.

□

Loops and termination

The iteration rule presented in this section is not valid for all while loops: it does not consider the case in which the loop does not terminate. This means that the iteration rule only allows us to prove *partial correctness*. The correctness is partial because we have not proved that the program terminates, and thus, it may not even achieve its postcondition. What we have proved is that, *if the program does terminate*, then it will satisfy the postcondition.

The solution is a modified rule that must include a proof that the loop terminates. Including the modified rule gives us the ability to prove *total correctness*. We can define total correctness as:

$$\text{total correctness} := \text{partial correctness} + \text{termination}$$

We will not consider total correctness in these notes.

7.4.8 Establishing loop invariants

Establishing loop invariants automatically is an open and challenging research question. This is good news for researchers in the area, but bad news for those of us that want to prove programs correct.

Loop invariants are important because knowing the loop invariant provides you with a stronger property to work with when showing that the loop establishes the required postcondition. We already talked about the three properties that we require in a loop invariant, I :

1. It should hold at the point right before the loop executes for the first time.
2. With the negated branch condition, $\neg B$, it should establish the loop's postcondition Q : $I \wedge \neg B \Rightarrow Q$.
3. With the branch condition, B , the body must re-establish the invariant. That is, $\{B \wedge I\} S \{I\}$.

Heuristics for finding loop invariants

The above properties are not enough to find loop invariants. While finding loop invariants requires some creativity, there are heuristics to help with this. The following heuristics can help us to infer loop invariants:

1. Loop invariants generally contain most of the variables from the loop condition, loop body, the precondition, and the postcondition (where we mean the precondition/postcondition of the loop, not necessarily the entire program).
2. They generally state a relationship between the variables.
3. They generally contain a predicate that is closely related to the postcondition, but is not the same.
4. They hold even if the loop condition is false.

Example 7.8

In the factorial program, we have the following:

1. Initially, $f = 1$ and $i = 0$.
2. After termination, we want that $f = n!$ and $i = n$.
3. At each loop, i and f are both increased.

From this, we can infer that a good loop invariant is $f = i!$.

□

Example 7.9

As an example of loop invariants and the iteration rule, let's consider the following program that sums the values in an array of length N :

```

1  procedure Summation (N : in Integer; A : in IntArray; Sum : out Integer)
2  with
3      Pre => (N >= 0),
4      Post => (Sum =  $\sum_{j=0}^{N-1} A(j)$ );
5  is begin
6      I := 0;
7      Sum := 0;
8      while I /= N do
9          Sum := Sum + A(I);
10         I := I + 1;
11     done;
12 end Summation;

```

What is the loop invariant for this program? We can eyeball the program to see that it will need to support precondition of $i = 0 \wedge \text{sum} = 0$, and the postcondition $\text{sum} = \sum_{j=0}^{N-1} a[j]$.

What we need to look for is a predicate that is close to the postcondition, but describes a relationship between sum and i . We can use what we know about the program to help us. The loop body is summing up elements from an array, adding each element, $a[i]$, one at a time, and using the variable sum to record the result. We know that i is 0 initially and is incremented until it reaches N . The loop exits when $i = N$, so a good guess is to replace i with N in the postcondition to get:

$$\text{sum} = \sum_{j=0}^{i-1} a[j]$$

This becomes our loop invariant. Now, we can annotate our program with the loop invariant in the right places (before/after the loop, and before/after the loop body), and work backwards to get the following annotations:

```

 $\{N \geq 0\}$ 
 $\{0 = \sum_{j=0}^{-1} a[j]\}$ 
 $i := 0;$ 
 $\{0 = \sum_{j=0}^{i-1} a[j]\}$ 
 $sum := 0;$ 
 $\{sum = \sum_{j=0}^{i-1} a[j]\}$ 
while  $i \neq N$  do
     $\{i \neq N \wedge sum = \sum_{j=0}^{i-1} a[j]\}$ 
     $\{sum + a[i] = \sum_{j=0}^i a[j]\}$ 
     $sum := sum + a[i];$ 
     $\{sum = \sum_{j=0}^i a[j]\}$ 
     $i := i + 1;$ 
     $\{sum = \sum_{j=0}^{i-1} a[j]\}$ 
end while
 $\{i = N \wedge sum = \sum_{j=0}^{i-1} a[j]\}$ 
 $\{sum = \sum_{j=0}^{N-1} a[j]\}$ 

```

Using these, we need to check the implications:

1. The loop postcondition: $\{i = N \wedge sum = \sum_{j=0}^{i-1} a[j]\}$ implies $\{sum = \sum_{j=0}^{N-1} a[j]\}$, which holds trivially.
2. The top of the loop body: $\{i \neq N \wedge sum = \sum_{j=0}^{i-1} a[j]\}$ implies $\{sum + a[i] = \sum_{j=0}^i a[j]\}$ which holds since $(\sum_{j=0}^{i-1} a[j]) + a[i] = \sum_{j=0}^i a[j]$.
3. The initial precondition: $\{N \geq 0\}$ implies $\{0 = \sum_{j=0}^{-1} a[j]\}$, which holds since the sum is defined over an empty list (from 0 to -1) which is 0 by definition.

Since these three implications hold it means that our summation algorithm conforms to its contract.

□

7.4.9 Procedure call rule

Non-Examinable Content Follows: The remaining content in this chapter is optional reading and is not examinable.

In the first instance, we will consider only calls to non-recursive procedures — either direct or indirect. The rule for this is straightforward, but comes with some restrictions.

The procedure call rule is:

$$\frac{\{P\} S \{Q\}}{\{P[E_1/v_1, \dots, E_n/v_n]\} p(E_1, \dots, E_n) \{Q[E_1/v_1, \dots, E_n/v_n]\}} \quad \text{where } p(v_1, \dots, v_n) \hat{=} S$$

This states that, if the body, S , of a procedure p , establishes a postcondition, Q , under the precondition P , then a call to that procedure will establish the same postcondition, but with all parameters replaced by the supplied arguments.

To prove a Hoare triple containing a procedure call, we first prove the correctness of the procedure call in terms of its formal parameters, and then replace the formal parameters with the actual arguments in the precondition and postcondition.

Unfortunately, this rule is not valid for all procedure calls. If our language only had procedures with no parameters, the simplified version of this rule (with no renaming) would be sufficient. However, the rule is not valid for cases in which the same variable is used as an argument to the procedure. The following example, taken from Hoare [5], demonstrates a counter example to the rule.

Example 7.10

Assume:	$p(x, y) \hat{=} x := y + 1$	(1)
	$\{y + 1 = y + 1\} x := y + 1 \{x = y + 1\}$	[Assignment axiom] (2)
	$true \Rightarrow y + 1 = y + 1$	[Logical theorem] (3)
From 2, 3:	$\{true\} x := y + 1 \{x = y + 1\}$	[Consequence rule] (4)
From 1, 4:	$\{true\} p(x, y) \{x = y + 1\}$	[Procedure rule] (5)
From 5:	$\{true\} p(z, z) \{z = z + 1\}$	[Procedure rule] (6)

Clearly, the final line (6) contains a contradiction because $z = z + 1$ is not satisfiable, showing that the procedure rule is not sound for all programs.

□

Parameters and arguments

In many languages, procedures take two types of arguments: *value* variables, which are values passed to the procedure that cannot be changed; and *value-result* variables, which are *variables* passed to the procedure that can be changed. In Ada, these are labelled using the keywords `in` and `out`.

From the counterexample above, it is clear that certain combinations of parameters and arguments can cause problems in some programs with respect to Hoare logic. To ensure that the procedure call rule is valid, the set of programs accepted by Hoare logic is restricted to the following:

1. The number of arguments in a procedure call must be equal to the number of parameters in the procedure.
2. The formal parameter names must be distinct from each other.
3. The arguments for value-result variables must be distinct from each other. It is this rule that prevents the problem outlined in the counterexample above — the procedure call would be illegal.
4. Value parameters cannot be changed in the procedure body.
5. No recursion.

In languages such as SPARK, these rules apply, and the SPARK toolset will raise errors when typechecking programs that violate the above rules.

Example 7.11

As an example, consider the factorial program from Example 7.7. In that example, we established that the factorial program conforms to its contract.

Now, we want to prove the following *call* to the FACTORIAL procedure:

$$\{N = 10\} \text{FACTORIAL}(10, F) \{F = 10!\}$$

We have already proved the triple $\{N \geq 0\} \text{FACTORIAL}(N, F) \{F = N!\}$, which is the hypothesis of the procedure call rule. So, applying the procedure call rule, we substitute in 10 for N to get:

$$\{10 \geq 0\} \text{FACTORIAL}(10, F) \{F = 10!\}$$

The postcondition is achieved, but not the precondition. We use the consequence rule to show that the precondition of our program implies the weakest precondition above: $N = 10 \Rightarrow 10 \geq 0$. This is trivially true because $10 \geq 0$ is true.

□

Recursive procedures — the Induction rule

Reasoning about recursive procedures is not possible using the procedure call rule above. This is because the procedure call rule “unfolds” a procedure call and proves its body. If the procedure is recursive, then the body contains the same procedure call, so the unfolding will continue *ad infinitum*.

Reasoning about recursive procedures is not so difficult, although it is not straightforward to automate. As it turns out, the solution is simple yet extremely powerful: in the proof of the procedure body, we assume that recursive calls to the procedure already hold. Formally, this can be stated:

$$\frac{\{P\} p() \{Q\} \vdash \{P\} S \{Q\}}{\{P\} p() \{Q\}} \quad \text{where } p() \hat{=} S$$

in which we have omitted parameters to improve readability. In the above, the symbol $\vdash$ means “entails” or “proves”; so $P \vdash Q$ means that Q is provable if we assume that P holds.

This rule is called the *induction* rule. It says: if we can prove that the procedure body satisfies the precondition and postcondition under assumption that all recursive calls to p also do so, then p satisfies the postcondition under the precondition.

How can this be?! Assuming the conclusion as part of the program body proof seems strange, but the following example provides a concrete case on which it holds.

Example 7.12

The following program shows a recursive procedure:

```
procedure  $p(\text{in } x, \text{out } y)$ 
  if  $x \leq 5$  then
     $y := 1$ 
  else
     $x := x - 1;$ 
     $p(x, y)$ 
  end if
```

end procedure

It is clear to reason that if $x > 5$, it will be decremented until it reaches 5, and the program will terminate. The program will always terminate with $y = 1$, so the fact that the second branch contains a recursive call is inconsequential: all that matters are the terminating states.

□

Recursive procedure calls can never be terminating statements themselves: the statements within the call must be executed. The terminating statements in the procedure are the only statements with end states, we need to only prove that these statements establish the postcondition. For the recursive statements, we assume that these are true for the purposes of establishing the proof.

This is analogous to the use of induction in logic: we prove the base case, and then the inductive case. In the recursive procedure rule above, we offer only the inductive step, because the base case corresponds to the zero-th unfolding of the procedure, which is achieved by default: the zero-th unfolding is not executing the procedure at all, which results in a terminating state of **false**, and **false** implies any postcondition.

7.4.10 Other rules

Many extensions of the basic Hoare logic have been proposed, including extensions that consider pointers and references, concurrency, goto statements, object-oriented programs, and real-time programs. However, the basic Hoare logic here is sufficient to reason about a wide range of programs – including all programs written in the SPARK language.

7.5 Mechanising Hoare logic

It is clear that proofs about Hoare triple are long and tedious, even for small programs like the factorial program, with many small details that need to be done correctly. There are many opportunities to make mistakes, which may lead us to “prove” a correct program is incorrect, or even worse, that an incorrect program is correct.

However, with the exception of finding loop invariants, many of the steps can be mechanised, improving both the speed at which we can prove programs, and the correctness of the proofs themselves.

The procedure for mechanising proofs is straightforward, and it mimics that of how we tackle the proof manually. If we consider that the input to the process is a Hoare triple, then the following steps are undertaken:

1. Annotate the program with the necessary predicates; for example, the annotations between sequentially composed statements, and the loop invariants. Many of these can be done automatically using the idea of weakest preconditions, but loop invariants are difficult. This is an active research area, even here at the University of Melbourne in the Department of Computing and Information Systems.
2. Use a program to generate *verification conditions*. The verification conditions can be generated automatically by applying the Hoare logic rules. The parts of the Hoare logic proofs that remain

are the verification conditions; for example, that a program precondition implies the weakest precondition.

3. Prove as many verification conditions correct as possible using a theorem prover.
4. Leave the remainder of the verification conditions for the user, who can try to prove manually, or more likely, with a proof assistant tool.
5. If all verification conditions can be proved, the program is correct. If at least one is proved incorrect, the program is incorrect. If some cannot be proved or disproved, we cannot be certain of the program's correctness.

In workshops, we will look at the SPARK tools, which use logics quite similar to Hoare logic and processes such as the one described above to prove that SPARK programs conform to their contracts.

7.6 Dijkstra's weakest precondition calculus

In this section, we briefly present the *denotational* semantics of programming languages, known as *Dijkstra's weakest precondition calculus*, or *Dijkstra's predicate transformer semantics* [2]. Unlike Hoare logic, which uses axioms to define the semantics of a programming language, the weakest precondition calculus assigns each program to a corresponding *predicate transformer*. These transformers are functions that map a postcondition of a program statement to the weakest precondition that achieves that postcondition.

We have already seen weakest preconditions in Hoare logic, but the concept has a first-class importance in Dijkstra's definitions.

Definition 7.1 (Weakest precondition)

Given a statement S and a postcondition R , the weakest precondition of S under R , written $wp(S, R)$, is the weakest predicate that as a precondition, establishes R if S is executed and terminates. $\square$

Thus, the following holds:

$$\{wp(S, Q)\} S \{Q\}.$$

Dijkstra's weakest precondition calculus is a set of predicate transformers for determining the weakest precondition of a program given a postcondition.

7.6.1 Transformers

The transformers are similar to that of the Hoare logic rules:

Rule	Input	Output
Skip	$wp(\mathbf{skip}, R)$	R
Assignment	$wp(x := E, R)$	$R[E/x]$
Sequence	$wp(S_1; S_2, R)$	$wp(S_1, wp(S_2, R))$
Conditional	$wp(\mathbf{if } B \mathbf{ then } S_1 \mathbf{ else } S_2 \mathbf{ endif}, R)$	$B \rightarrow wp(S_1, R) \wedge \neg B \rightarrow wp(S_2, R)$
While loop	$wp(\mathbf{while } B \mathbf{ do } S \mathbf{ done})$	$\exists k. (k \geq 0 \wedge P_k)$ where $P_0 \equiv \neg B \wedge Q$ $P_{k+1} \equiv B \wedge wp(S, P_k)$

For the while loop, the rule states that the loop body S executes k times, and terminates with Q holding. Note that P_k is an infinitely long assertion, for which induction is the most suitable approach to prove in most cases.

7.6.2 Program proof

To prove that a terminating program S establishes a postcondition Q under a precondition P — that is, the Hoare triple $\{P\} S \{Q\}$ is provable for terminating S — we need to just prove that P satisfies the weakest precondition:

$$P \rightarrow wp(S, Q).$$

7.7 Additional reading

1. Hoare's original 1969 paper "An axiomatic basis for computer programming" [4] is a truly brilliant article that I highly recommend reading. It contains only a subset of the rules presented in this chapter.
A PDF copy is available on the LMS and in the subject repository.
2. Mike Gordon from Cambridge University has written an excellent article on Hoare logic titled "Background reading on Hoare Logic", which provides additional material and examples. This can be downloaded from <http://www.cl.cam.ac.uk/~mjc/Teaching/2011/Hoare/Notes/Notes.pdf>
3. Dijkstra's seminal work on *predicate transformer semantics* [2].

Bibliography

- [1] J. Bentley. *Programming pearls*. Addison-Wesley Professional, 2000.
- [2] E. W. Dijkstra. Guarded commands, nondeterminacy, and formal derivation of programs. In *Programming Methodology*, pages 166–175. Springer, 1978.
- [3] E. W. Dijkstra. "Why is software so expensive?": An explanation to the hardware designer. In *Selected Writings on Computing: A Personal Perspective*, pages 338–348. Springer, 1982.

- [4] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- [5] C. A. R. Hoare. Procedures and parameters: An axiomatic approach. In *Symposium on Semantics of Algorithmic Languages*, pages 102–116. Springer, 1971.
- [6] Donald E Knuth. *The Art of Computer Programming, 3: Sorting and Searching*. Addison Wesley, Reading, MA, 2nd edition, 1998.

Chapter 8

Advanced Verification

8.1 Pointers and Aliasing

Pointers are ubiquitous in programming. If you have programmed in Java, you have programmed with pointers (even if you haven't been aware of it). Pointers are the source of many bugs in C programs. Ada also supports pointers, via **access** types (see section 4.9).

Just as programming with pointers can often be tricky, *reasoning* about programs with pointers presents difficulties.

To understand why, consider the following Ada program, which is a small variation on an Ada program we met when discussing pointers in section 4.9.

Listing 8.1: `pass_by_reference.adb`

```
1 with Ada.Text_IO; use Ada.Text_IO;
2 with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;
3
4 procedure Pass_By_Reference is
5   type Rec is record
6     X : Integer;
7   end record;
8
9   type RecPtr is access all Rec;
10
11   R : aliased Rec;
12
13   procedure Print_R is
14   begin
15     Put ("R.X: ");
16     Put (R.X);
17     New_Line;
18   end Print_R;
19
20   procedure CheckR(givenR : in RecPtr) is
21     X : Integer := R.X;
22   begin
23     Put ("In CheckR"); New_Line;
24     -- here we know that { X = R.X }
```

```

25     givenR.X := R.X + 1;
26     -- but here, R.X has changed, even though we did not
27     -- explicitly assign to R! So { X = R.X } is no longer true
28     if GivenR = R'Access then
29         Put("Equal");
30     else
31         Put("Not Equal");
32     end if;
33     New_Line;
34     Print_R;
35 end CheckR;
36
37 begin
38
39     R.X := 5;
40     CheckR(R'Access);
41     Put("CheckR done"); New_Line;
42     Print_R;
43
44 end Pass_By_Reference;

```

We have added a local variable `X` to `CheckR` procedure, which is assigned the initial value of `R.X`. Therefore, before we assign to `givenR`, we know that `X = R.X`, as reflected in the first comment.

However, since `givenR` refers to the same record as `R`, when `CheckR` executes the assignment `givenR.X := R.X + 1`, this statement modifies the record `R`. Hence, after that assignment, `X = R.X` is no longer true, *even though the assignment did not explicitly modify the record `R`!*

This highlights the fact that when two variables like `R` and `givenR` refer to the same region of memory, reasoning about programs becomes very difficult. The essence of the problem is that updates to one variable (e.g. `givenR`) can modify a separate variable (e.g. `R`). This problem is known as *aliasing*.

Definition 8.1 (Alias)

We say that two variables *alias* when they both refer to the same region of memory. For instance, both might be pointers that refer to the same record or the same integer, or one might refer to a record and the other might refer to a field within that record.

When this happens, we also say that one variable is an *alias* for the other. $\square$

Because pointers make it very easy to (inadvertently) introduce aliasing into a program, reasoning about programs with pointers can be challenging.

8.2 Safe Programming with Pointers: Avoiding Unsafe Aliasing

How can we prove things about programs with pointers? Since aliasing is what makes reasoning about pointers difficult, one approach is to limit aliasing. This is the approach taken by recent versions of SPARK, and we will explain it in this section.

Another approach is to develop new extensions to Hoare logic that force one to prove, when reasoning about a program, that no unsafe aliasing occurs. We will discuss this approach later in section 8.3.

8.2.1 Unsafe Aliasing

Aliasing is almost impossible to avoid. But luckily not *all* kinds of aliasing is bad for reasoning about programs. Consider the following example Swap procedure.

Listing 8.2: A Procedure to Swap two Pointers

```
1  procedure Swap(X : in out IntPtr; Y : in out IntPtr)
2    with Post => (X = Y'Old and Y = X'Old)
3    is
4      Temp : IntPtr := X;  -- Temp and X alias each other
5    begin
6      X := Y;
7      Y := Temp;
8    end Swap;
```

This procedure *intentionally* creates an alias Temp for X. That is, after the assignment Temp := X (when Temp is initialised), those two variables alias each other. However, this alias is *not* unsafe. The reason it is not a problem is because, unlike in **pass_by_reference.adb** in section 8.1, we never use both Temp and X at the same time to access the region of memory that they both refer to.

Definition 8.2 (Safe Aliasing)

We say that, for two variables X and Y that alias, the aliasing is *safe* if at no point during the program while the two variables alias, do we ever use both variables at the same time. □

Under this definition, the aliasing in **pass_by_reference.adb** in section 8.1 is *not* safe, because in the assignment givenR.X := R.X + 1, we are using both givenR.X and R.X at the same time.

For two variables to be used “at the same time” the two uses don’t need to be part of the same statement. As another example of potentially unsafe aliasing, suppose we decided to extend the Swap procedure above so that, before doing the swap, it printed out the two integers that X and Y refer to, as follows.

Listing 8.3: A Procedure to Swap two integer Pointers while Printing Out the integers

```
1  procedure Swap_And_Print(X : in out IntPtr; Y : in out IntPtr)
2    with Pre => (X /= null and Y /= null),
3    with Post => (X = Y'Old and Y = X'Old)
4    is
5      Temp : IntPtr := X;  -- Temp and X alias each other
6    begin
7      -- print out the integers that X and Y each point to
8      Ada.Integer_Text_IO.Put(X.all);
9      Ada.Integer_Text_IO.Put(Y.all);
10     X := Y;
11     Y := Temp;
12  end Swap;
```

Now the alias between Temp and X is potentially unsafe. Temp and X both alias. We access X to print out the integer it refers to, and then later we also access Temp to assign it to Y (on the final line).

8.2.2 Avoiding Unsafe Aliasing in SPARK

To avoid unsafe aliasing, we need to make sure that at all times in the program, even if two variables alias, we only ever use one of them.

SPARK enforces this discipline by enforcing an *anti-aliasing rule*.

Definition 8.3 (Anti-Aliasing Rule)

This rule says that as soon as an alias is created, by assigning to some variable Y that now aliases an existing variable X , X is not allowed to be used anymore. $\square$

Note that the `Swap` procedure above follows this rule because, as soon as we create the alias `Temp` for X , we don't use X anymore. Of course, updating X 's value, in the assignment $X := Y$, does not count because that assignment means that `Temp` and X no longer alias.

SPARK allows this rule to be weakened in some cases. In particular, when it can be sure that the aliasing is being used only to read the memory being referred to (but not to modify it). Therefore one can have limited forms of aliasing when using access-to-**constant** types. You can read more about SPARK's restrictions on pointer usage in the SPARK Reference manual [1, Section 6.4.2. Anti-Aliasing].

Another situation in which SPARK allows aliasing is when passing arguments to **functions**. Recall that in SPARK, functions need to be free of side-effects. This means that SPARK functions cannot modify memory that is referred to by a pointer. Therefore it is always safe to pass two pointers to a SPARK function, even if those two pointers alias.

8.2.3 Anti-Aliasing Assumptions in SPARK

Because SPARK enforces the anti-aliasing rule, it means that it is free to assume that any two pointers that are both used at the same time cannot alias. To see how this works, consider the following procedure:

Listing 8.4: A procedure that writes to one pointer but not the other.

```
1  procedure Zero_First_Arg( $X$  : in IntPtr;  $Y$  : in IntPtr)
2      with Pre => ( $X \neq \text{null}$  and  $Y \neq \text{null}$ ),
3          Post => ( $Y.\text{all} = Y.\text{all}'\text{Old}$  and  $X.\text{all} = 0$ ) is
4      begin
5           $X.\text{all} := 0$ ;
6          Ada.Text_IO.New_Line;
7      end Zero_First_Arg;
```

This procedure takes two integer pointer arguments X and Y and updates the value pointed to by X (making it point to 0) but not that of Y . The SPARK Prover is able to prove that it satisfies its postcondition, which says that the value pointed to by Y is not changed. This is true, of course, only when X and Y do not alias. The SPARK Prover is free to assume that they do not alias, since both are used (in particular in the postcondition), because it enforces the anti-aliasing rule.

8.2.4 SPARK Pointer Checks

Just as the SPARK Prover proves the absence of runtime errors like integer overflow and division by zero, it also proves that each pointer dereference is valid. A pointer is *dereferenced* when it is used to access the memory that it points to. A pointer is *invalid* when it is the null pointer (i.e. one that doesn't refer to valid memory). The SPARK prover will prove that all pointer accesses are valid and will give a warning when it cannot prove this.

As an example, take the `Zero_First_Arg` procedure above, which accesses the memory that X refers to, by storing 0 to that memory: $X.\text{all} := 0$. This dereference is valid only if X is not the null

pointer. This is the reason that the procedure has the precondition requiring that `X != null`. Since its postcondition also dereferences `Y` (in order to state that the memory referred to by `Y` does not change), it also requires the precondition that `Y != null`.

8.2.5 Other Programming Languages: Rust

SPARK has only recently begun to support programs with pointers. Without its anti-aliasing rules it would not be feasible for SPARK to reason about Ada programs with access (pointer) types.

SPARK's anti-aliasing rules were heavily inspired by the Rust programming language, from Mozilla. Rust is a programming language that is designed to be a safe alternative to “systems” programming languages like C. One of Rust's main differences from traditional languages is that it enforces strong anti-aliasing rules in all programs.

Rust's rules are more permissive than the ones described above, and they exist not primarily to allow one to reason about Rust programs but instead to avoid common kinds of programming errors that can occur when using pointers. These include errors that might occur when two pieces of code share access to some common memory, that is deallocated by one of them but subsequently used by the other. Rust prevents this by preventing both pieces of code holding a (de-allocatable) reference to the memory at the same time. The same restrictions also allow one to program safe concurrent programs in Rust, that are free of data race errors.

You can learn more about Rust by consulting the various online resources available on that language.

8.3 Separation Logic

Another way to reason about programs with pointers, which avoids the restrictions imposed by SPARK, Rust and similar languages, is to extend Hoare Logic to support explicit reasoning about pointers. The most widely used extension to Hoare Logic for reasoning about pointers is called *Separation Logic*.

8.3.1 Pointers in C

Before describing separation logic, we give a brief reminder of how pointers work in the C programming language. We saw earlier how pointers can be used to implement pass-by-reference (i.e. how they can be used to avoid having to pass large data structures around in programs, to improve efficiency). Another common use for pointers is for dynamically allocated memory. The following C program gives a small example. It allocates a very small amount of memory dynamically—just enough to hold a single signed integer—using the `malloc()` function from the standard C library. The `malloc()` function returns a pointer that refers to the newly allocated memory, if the allocation was successful. Otherwise, if allocation fails, `malloc()` returns `NULL`: the null pointer.

The result from `malloc()` is stored into the local variable `p`: this variable is of type `int *`, which means that it is a pointer to a signed integer. The program checks to see if allocation was successful. If so, it stores the integer 5 into the memory that was allocated. Specifically, in C a pointer like the variable `p` in the program below stores the *address* of the memory that it refers to. The line of code `*p = 5;` means “write the number 5 into the memory location that is stored in the variable `p`”.

The program then loops until the value stored in the location that `p` points to is no longer greater than

zero. It repeatedly decrements the value that p points to, by first reading this value into the local variable v and then storing back into this location the result of $v - 1$.

Listing 8.5: A tiny C pointer program.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(void){
5     printf("Hello, world!\n");
6     int *p = malloc(sizeof(int));
7     if (p != NULL){
8         *p = 5;
9
10        while(*p > 0){
11            int v = *p;
12            *p = v - 1;
13        }
14        printf("All done!\n");
15    }else{
16        printf("Allocation failed!\n");
17    }
18    return 0;
19 }
```

Recall that in C programs, local variables live in a region of memory called the *stack*. As explained above, pointers store memory addresses. These addresses often refer to memory in the *heap*, which is the region of memory used to hold dynamically allocated structures (like the dynamically allocated integer in the program above).

8.3.2 Separation Logic (for C)

Separation Logic is an extension of Hoare logic. That is, it adds just a few extra ingredients to the Hoare Logic that we learned about in chapter 7.

Specifically, the memory of programs now contains not only local variables but also the heap. An assignment like $x := x + 1$ modifies the local variable x , by incrementing its value. In general we can write assignments like $x := E$ where E is an expression that mentions local variables (but cannot not refer to the heap). Local variables do not live in the heap, and modifying a local variable does not alter any of the memory in the heap.

To access the heap, there are two new kinds of statements: the **load** and **store** statements.

The Load Statement

The load statement is used to read data from the heap and copy it into local variables.

The load statement looks like this:

$$x := *E$$

where E is an expression like $x + y$, $z - 1$, etc. The expression E is evaluated and its result is interpreted as a memory address. This memory address is looked-up in the heap and whatever value is stored at that

heap location is copied into the local variable x .

For example, suppose variable p holds a pointer to an integer (that, necessarily, lives in the heap). For instance, p might store the address 120. In the heap at location 120 suppose there is the number 5. Then after executing $x := *p$, x would hold the value 5.

The Store Statement

The store statement is used to write data into the heap. It looks like this:

$$*E_1 := E_2$$

where E_1 and E_2 are both expressions that can mention local variables. The expression E_2 is evaluated to produce a value v . The expression E_1 is evaluated to produce a memory address a . Then the value v is written into the heap at the memory location a . For example, if p is a local variable holding the address 120 and x holds the number 5, then $*p := x - 1$ evaluates $x - 1$ to produce the value 4 and stores this value into the heap at location 120.

The Points-To Assertion: $\mapsto$

To reason about these two new commands, separation logic adds some new, special kinds of assertions that can be used in pre- and post-conditions. The first of these is called the *points-to* assertion and is written as follows, for expressions E_p and E_v :

$$E_p \mapsto E_v$$

It means that in the heap at the location identified by the expression E_p is the value E_v . For instance, in the example above in which the pointer p holds the address 120 and the heap at address 120 holds the value 4, we would write $p \mapsto 4$. As another example, suppose the local variable a held the address of an array of integers that was allocated on the heap. Suppose that this array lives at address 160. Suppose the second element of the array was the number 2 (i.e. we might normally write $a[1] = 2$, assuming the array was indexed from 0). In separation logic, we write $a + 1 \mapsto 2$, which literally means that at address 161 in the heap is the number 2.

If p is the null pointer, then it does not refer to any value in the heap. Therefore there is no value v for which $p \mapsto v$ is true.

The Problem of Aliasing

The points-to assertion is separation logic's first weapon in the struggle to reason about pointers. Its second weapon we will introduce shortly. To understand why it is needed, however, consider the following scenario. Suppose we have two pointers x and y and we know that they both refer to the value 2 in the heap, i.e. we know $x \mapsto 2 \wedge y \mapsto 2$. Then suppose we store 3 into the heap using the x pointer, i.e. the program does $*x := 3$. Is it the case that afterwards $x \mapsto 3 \wedge y \mapsto 2$? i.e. is the following Hoare logic statement true?

$$\begin{array}{c} \{x \mapsto 2 \wedge y \mapsto 2\} \\ \quad *x := 3 \\ \{x \mapsto 3 \wedge y \mapsto 2\} \end{array}$$

The store $*x := 3$ will definitely modify the part of the heap that the pointer x refers to, by writing 3 to that location. So afterwards, $x \mapsto 3$ will definitely be true. Can we be sure that after we do $*x := 3$ that this won't affect the part of the heap that y refers to?

What if x and y alias, i.e. they both refer to the same location of the heap? In that case, it would not be true that afterwards $y \mapsto 2$ anymore since y refers to the same location of the heap that x does and so now it would be the case that $y \mapsto 3$ instead.

In general, the statement that $x \mapsto 2 \wedge y \mapsto 2$ gives us no information to say that x and y don't alias. As we saw earlier in this chapter, aliasing makes reasoning about pointer programs very difficult. Indeed, it is what invalidates the reasoning above. Separation logic's second weapon addresses this problem.

Separating Conjunction: $\star$

To deal with the problem of aliasing, separation logic introduces a new “and” operator, in addition to the usual $\wedge$ used above. The ordinary $\wedge$ is called *conjunction*. The new “and” operator is called *separating conjunction* and is written as $\star$. For predicates (e.g. pre- or postconditions) P and Q , we write

$$P \star Q$$

and this means that P and Q both hold. However it also says that the parts of the heap that P talks about, and the parts of the heap that Q talks about, must be *separate* from each other. Put another way, it means that P and Q both hold and, additionally, P and Q each talk about different (non-overlapping) parts of the heap.

The part of the heap that a predicate refers to is called the predicate's *footprint* or *domain*. For a predicate P we write $dom(P)$ to refer to P 's domain, which is just the set of heap locations that P talks about. The domain of the predicate $p \mapsto v$ is just the location p , i.e. $dom(p \mapsto v) = \{p\}$. The domain of a predicate $P \star Q$ (for some P and Q) is the union of the domains of P and Q respectively: $dom(P \star Q) = dom(P) \cup dom(Q)$. Then $P \star Q$ means that P and Q both hold and, additionally, that $dom(P) \cap dom(Q) = \{\}$.

Therefore, $x \mapsto 2 \star y \mapsto 2$ means not only that x and y both point to the value 2 but, additionally, that the two pointers *do not alias*, since we know that x and y must each refer to separate parts of the heap from each other.

Therefore, the following statement is true in separation logic:

$$\begin{aligned} &\{x \mapsto 2 \star y \mapsto 2\} \\ &\quad *x := 3 \\ &\{x \mapsto 3 \star y \mapsto 2\} \end{aligned}$$

8.3.3 The Meaning of Separation Logic Statements

What does a statement $\{P\} S \{Q\}$ mean in separation logic? Just like in Hoare logic, when $\{P\} S \{Q\}$ we know that if the program S begins execution in a state satisfying the precondition P then, when S terminates, Q will be true. However, in separation logic, we also know something else: S will not *fail*. By “fail” we mean that S won't try access an invalid pointer (e.g. a null pointer or a pointer that hasn't yet been initialised and so might not refer to a valid memory address).

For instance, suppose we changed the C program above so that it didn't check whether `malloc()` was successful before it dereferenced the pointer `p`. In that case, the program might fail if `malloc()` does

not succeed. When we prove something about a program in separation logic, we also prove that the program will never fail, assuming it begins execution from a state satisfying its precondition.

Therefore separation logic gives us guarantees that programs are memory safe, and so are free of null-pointer dereference vulnerabilities, buffer overflow vulnerabilities, use-after-free vulnerabilities and other memory errors.

8.3.4 The Rules of Separation Logic

Separation logic uses the same rules for skip, sequencing, if and while as in normal Hoare logic. It also has the consequence rule, just as in ordinary Hoare logic. (See chapter 7 if you need a refresher on those rules.)

It does have different rules for reasoning about assignments to local variables $x := E$, plus it of course has new rules to reason about load and store statements. However it also has a very special rule that normal Hoare logic does not, which makes separation logic more powerful than ordinary Hoare logic. This special rule is called the *frame rule*, and this rule harnesses the power of separating conjunction. We discuss the frame rule first, since it is so integral to separation logic, before discussing the other rules.

The Frame Rule

The frame rule allows one to reason *locally* about a statement, which means that we can reason about each statement just in terms of the parts of the heap that the statement interacts with while ignoring the rest of the heap. This works because the frame rule allows us to conclude that the parts of the heap that a statement does not touch cannot change, and therefore anything that was true about those parts of the heap before the statement executed must still be true afterwards.

The frame rule is written as follows:

$$\frac{\{P\} S \{Q\}, \text{modv}(S) \cap \text{vars}(R) = \{\}}{\{P \star R\} S \{Q \star R\}}$$

Here, S is any program statement. This rule says that if we can prove some statement $\{P\} S \{Q\}$ about S , then we know that any other parts of the heap that S does not interact with do not change. Therefore if we know *separately* that some property R holds before S , then R must still hold afterwards. The property R is called the *frame*.

The use of separating conjunction “ $\star$ ” in the pre- and post-conditions is what tells us that the part of the heap that R talks about is separate from the parts of the heap that P and Q talk about. If we can prove $\{P\} S \{Q\}$, then we know that S only interacts with parts of the heap that are described by P (otherwise S would fail and we wouldn’t have been able to prove $\{P\} S \{Q\}$). We also know that all of these parts of the heap described by P are separate from R (thanks to separating conjunction). Therefore, R isn’t affected by S .

Before discussing the side-condition $\text{modv}(S) \cap \text{vars}(R) = \{\}$ in the rule, and why it is needed, first note that the frame rule is what allows us conclude that the following separation logic statement is valid:

$$\begin{array}{c} \{x \mapsto 2 \star y \mapsto 2\} \\ *x := 3 \\ \{x \mapsto 3 \star y \mapsto 2\} \end{array}$$

In particular, $y \mapsto 2$ is the frame R . Since the statement $*x := 3$ does not interact at all with the part of the heap referred to by y , we know that it will not be affected by this statement. Hence, if $y \mapsto 2$ holds beforehand it will still hold afterwards.

The side condition $\text{modv}(S) \cap \text{vars}(R) = \{\}$ checks that the program doesn't change the meaning of the frame R . Suppose we changed the program above so that after doing $*x := 3$ it then did $y := x$. Would the reasoning above still be true?

$$\begin{array}{c} \{x \mapsto 2 * y \mapsto 2\} \\ *x := 3; \\ y := x \\ \{x \mapsto 3 * y \mapsto 2\} \end{array}$$

Clearly it would not be since now, after we run the program x and y no longer refer to separate parts of the heap (they both refer to the same part of the heap because we made y an alias for x), so $y \mapsto 2$ is no longer true and we can't even use separating conjunction anymore because x and y are longer separate!

The problem here is that assigning to y changes the meaning of the frame $y \mapsto 2$. This is why we need the side condition $\text{modv}(S) \cap \text{vars}(R) = \{\}$. It says that the set of local variables modified by statement S does not include any variables mentioned by the frame R . The set of variables modified by a statement is written $\text{modv}(S)$. The set of variables mentioned in an assertion R (pre or postcondition) is written $\text{vars}(R)$. When a statement S modifies some variable mentioned by R it can of course change what R means, as the example above demonstrates. So this side condition is needed to make sure that this doesn't happen, in order to conclude that the frame is not affected by S .

$\text{modv}(S)$ can be defined very straightforwardly for all statements. Assignment statements $x := E$ modify only the variable x , as do load statements $x := *E$. Store statements do not modify any local variables (they only modify the heap). For all other statements, the set of variables they modify is simply all variables modify by any sub-statement.

$$\begin{aligned} \text{modv}(\text{skip}) &= \{\} \\ \text{modv}(*E_p := E_v) &= \{\} \\ \text{modv}(x := E) &= \{x\} \\ \text{modv}(x := *E) &= \{x\} \\ \text{modv}(S_1; S_2) &= \text{modv}(S_1) \cup \text{modv}(S_2) \\ \text{modv}(\text{if } B \text{ then } S_1 \text{ else } S_2 \text{ endif}) &= \text{modv}(S_1) \cup \text{modv}(S_2) \\ \text{modv}(\text{while } B \text{ do } S \text{ done}) &= \text{modv}(S) \end{aligned}$$

The Load Rule

The rule for reasoning about load statements, also sometimes called the *heap read rule*, is as follows:

$$\frac{x \notin \text{vars}(E)}{\{E \mapsto v\} x := *E \{x = v * E \mapsto v\}}$$

This rule says that if we know that the address E refers in the heap to a valid memory location that holds some value v , then if we load from that location and put the value into the variable x (i.e. we execute $x := *E$), then afterwards $x = v$ must be true and $E \mapsto v$ will still be true.

The condition that $x \notin \text{vars}(E)$ is needed to ensure that modifying x does not change the meaning of E : suppose we did the load statement $x := *(x + 1)$ and, beforehand we knew that $x + 1 \mapsto 4$ was true.

Then afterwards, $x = 4$ would be true but (very likely) it would no longer be the case that $x + 1 \mapsto 4$ was true anymore (since that would be true only if $5 \mapsto 4$ were true).

This rule can be generalised to handle the case in which $x \in \text{vars}(E)$, but for this course, the rule above is sufficient.

The Store Rule

The store rule is as follows:

$$\frac{}{\{E_p \mapsto v\} * E_p := E_v \{E_p \mapsto E_v\}}$$

It says that after executing $*E_p := E_v$ we know that $E_p \mapsto E_v$ must be true. However, this is valid only if E_p already refers to a valid memory location; otherwise, if it didn't, the program might fail. So we need the precondition that there exists some value v for which $E_p \mapsto v$ is true. It doesn't matter what v is, but we do need to know that $E_p \mapsto$ something.

The Assignment Rule

Recall that in ordinary Hoare logic, we tend to reason *backwards* over programs. Backwards reasoning is also possible in separation logic. However it is more common in separation logic to reason forwards over programs instead. Therefore, the assignment rule in separation logic is different to the assignment rule in Hoare logic.

The assignment rule in separation logic is as follows:

$$\frac{x \notin \text{vars}(E)}{\{\mathbf{emp}\} x := E \{x = E\}}$$

Here **emp** is a bit like the assertion “true”: it talks about none of the heap (i.e. it talks about the *empty* heap—hence its name) and is true of every state. This rule says that after assigning E to x , $x = E$, as expected. Again we need the side condition that $x \notin \text{vars}(E)$ to make sure that the meaning of E is not changed. Hence, this rule cannot be used to reason about assignment statements like $x := x + 1$.

To reason about those kinds of statements, we need a more general rule:

$$\frac{}{\{x = n\} x := E \{x = (E[n/x])\}}$$

This rule says that after the assignment, $x = E$ except that we need to replace any occurrences of x in E with x 's value before the assignment, n . Using this rule, we can reason forwards to prove $\{x = 3\} x := x + 1 \{x = 4\}$, since $x = ((x + 1)[3/x]) = 3 + 1 = 4$.

8.3.5 An Example Proof

As an example proof in separation logic, consider the following statement:

$$\begin{aligned} &\{x \mapsto v_x * y \mapsto v_y\} \\ &\quad t := *x; \\ &\quad u := *y; \end{aligned}$$

$$\begin{aligned}
&*x := u; \\
&*y := t \\
&\{x \mapsto v_y \star y \mapsto v_x\}
\end{aligned}$$

This program starts with two valid pointers x and y . It then loads the values that x and y each point to into variables t and u respectively. Finally it stores u to x and t to y , so that finally x points to the value that y was originally pointing at, and vice versa.

To reason about this program we first apply the Hoare logic sequencing rule. In separation logic we reason forwards, so the first statement we reason about is $t := *x$ with precondition $\{x \mapsto v_x \star y \mapsto v_y\}$. So we need to find some intermediate assertion $?Q$ such that:

$$\begin{aligned}
&\{x \mapsto v_x \star y \mapsto v_y\} \\
&t := *x; \\
&\{?Q\}
\end{aligned}$$

and

$$\begin{aligned}
&\{?Q\} \\
&u := *y; \\
&*x := u; \\
&y := t \\
&\{x \mapsto v_y \star y \mapsto v_x\}
\end{aligned}$$

both hold. Working on the first goal we apply the frame rule, taking the frame R to be $y \mapsto v_y$. Hence $?Q = ?P \star y \mapsto v_y$ and we need to find $?P$ such that $\{x \mapsto v_x\} t := *x \{?P\}$. Using the load rule (since $t \notin \text{vars}(x)$), we have $\{x \mapsto v_x\} t = *x \{t = v_x \star x \mapsto v_x\}$, hence let $?P = t = v_x \star x \mapsto v_x$ and so we know that:

$$\begin{aligned}
&\{x \mapsto v_x \star y \mapsto v_y\} \\
&t := *x; \\
&\{(t = v_x \star x \mapsto v_x) \star y \mapsto v_y\}
\end{aligned}$$

This completes the reasoning about the first statement. We then have to reason about the three remaining statements. We of course use the Hoare sequencing rule to reason next about the statement $u := *y$; under the precondition $\{(t = v_x \star x \mapsto v_x) \star y \mapsto v_y\}$ that we just derived. This reasoning is analogous to the reasoning we did for the first load statement (using the frame and load rules). Afterwards we obtain

$$\begin{aligned}
&\{(t = v_x \star x \mapsto v_x) \star y \mapsto v_y\} \\
&u := *y; \\
&\{(t = v_x \star x \mapsto v_x) \star (u = v_y \star y \mapsto v_y)\}
\end{aligned}$$

and we have to prove

$$\begin{aligned}
&\{(t = v_x \star x \mapsto v_x) \star (u = v_y \star y \mapsto v_y)\} \\
&*x = u; \\
&*y = t; \\
&\{x \mapsto v_y \star y \mapsto v_x\}
\end{aligned}$$

Of course we apply the sequencing rule so we have to find some intermediate assertion $?S$ such that

$$\begin{aligned}
&\{(t = v_x \star x \mapsto v_x) \star (u = v_y \star y \mapsto v_y)\} \\
&*x = u; \\
&\{?S\} \\
&*y = t;
\end{aligned}$$

$$\{x \mapsto v_y \star y \mapsto v_x\}$$

We reason forwards over the first statement. As before we first need to apply the frame rule to the precondition to focus on just the part needed by the store rule to reason about $*x = u$. Since $\star$ is associative and commutative, we can rearrange the order and brackets in the different parts of the precondition as we like (technically, using the consequence rule). So we can take as the frame $t = v_x \star u = v_y \star y \mapsto v_y$, and via the store rule and the frame rule we can let $?S = x \mapsto u \star t = v_x \star u = v_y \star y \mapsto v_y$.

Then we can apply similar reasoning to solve the final goal:

$$\begin{array}{c} \{x \mapsto u \star t = v_x \star u = v_y \star y \mapsto v_y\} \\ *y = t; \\ \{x \mapsto v_y \star y \mapsto v_x\} \end{array}$$

First we use the consequence rule which says that this goal is true if we can find some final postcondition $?Z$ such that

$$\begin{array}{c} \{x \mapsto u \star t = v_x \star u = v_y \star y \mapsto v_y\} \\ *y = t; \\ \{?Z\} \end{array}$$

and $?Z \Rightarrow x \mapsto v_y \star y \mapsto v_x$. We apply the frame rule, taking the frame to be $x \mapsto u \star t = v_x \star u = v_y$ and then apply the store rule to obtain

$$\begin{array}{c} \{x \mapsto u \star t = v_x \star u = v_y \star y \mapsto v_y\} \\ *y = t; \\ \{x \mapsto u \star t = v_x \star u = v_y \star y \mapsto t\} \end{array}$$

All that remains to be shown is that $x \mapsto u \star t = v_x \star u = v_y \star y \mapsto t \Rightarrow x \mapsto v_y \star y \mapsto v_x$ which is obvious. This completes the proof.

8.4 Security: Information Flow

Content may be added in 2026 (please email toby.murray@unimelb.edu.au for more info if you are curious): To be added.

Bibliography

- [1] AdaCore and Altran UK Ltd. Spark 2014 reference manual, 2019. <http://docs.adacore.com/spark2014-docs/html/lrm/index.html>.

Chapter 9

Fault-tolerant design

In this chapter, we introduce the concept of *fault-tolerant design* – a design paradigm in which the system is designed to tolerate certain types of failure during operation. In this chapter, we discuss:

1. the notion of fault tolerance in systems, and its link to reliability and high-integrity systems; and
2. several design principles based on redundancy to design a fault-tolerant system.

9.1 Introduction to fault-tolerance

Recall from the subject *Software Engineering Methods* that the reliability of a system is expressed as a probability: it is the probability that a system may exhibit a failure over a defined interval. Reliability is often measured as *Mean Time To Failure* (MTTF).

Despite our best efforts, faults will still remain in software once it is in operation. *Fault-tolerant design* is a body of principles and methods for designing systems so that they can continue functioning correctly in the presence of faults.

Fault-tolerant designs are used to achieve high reliability. By making a system more tolerant of faults, we increase the mean time to failure. Such an increase is important in many high-integrity systems. The higher the MTTF, the less chance there is of failure, making the system safer, more secure, or more reliable.

Definitions

Before we start to discuss fault tolerance, we introduce some terms relevant to the topic, some of which you will already be familiar with; and some of which may be defined differently in other chapters in these notes (due to them being used in a different context/topic).

1. A *failure* is a point in the functioning of a system where the system behaviour deviates from its specification.
2. A *fault* is an incorrect step, process or data definition which will cause a system to fail for a given set of inputs.

3. An *error* is a manifestation of a fault.
4. A *hardware fault* is a physical defect that can cause a system or component to produce an error.
5. A *software fault* is a defect in the source of the software that can cause a system or component to produce an error.
6. The *reliability* of a system is the probability that the system will not exhibit a failure over a specified interval (e.g. time or set of inputs).

Software and hardware failures

It is important to note that *software and hardware fail in different ways*.

Hardware can fail as a result of wearing of parts, or parts that were manufactured faulty. To repair the system, replacing the part with an equivalent part is generally sufficient. Further to this, hardware generally fails *randomly*. In this context, “randomly” means that, given the same inputs, a hardware component can pass on these inputs one time, and then fail the next time. Hardware does not often fail as a result of a design fault.

Figure 9.1 shows a model failure curve for a hardware item. Hardware is likely to fail early due to a manufacturing error. If it does not, it tends to operate reliably for a period until the parts begin to wear out, at which times its likelihood of failure increases.

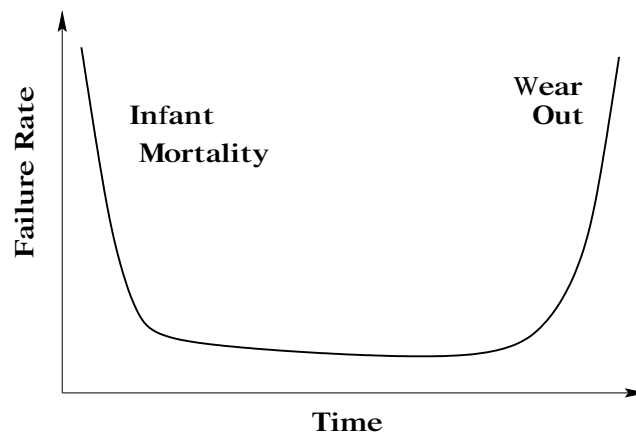


Figure 9.1: A model failure curve for hardware

Software does not wear out. Failures are generally the result of the design, so replacing software will typically not restore the system. Further, software generally fails *systematically*, meaning that in the same state, with the same inputs, it will fail every time, or succeed every time.

What is fault tolerance?

A system is said to be *fault tolerant* if it can continue to function according to its specification in the presence of a finite number of faults.

Thus, the field of fault tolerance accepts the fact that hardware and software will fail sometimes, but aims to produce a system that will be able to continue operating under a finite number of faults. In a fault tolerant system, the system itself must be able to detect that a fault has occurred, and work around it.

Classifying Failures

There are generally accepted to be three main causes of failures:

1. Faults in the specification or design of the system. It is extremely difficult to assure the correctness of a specification. If the specification contains faults then everything that stems from it may also contain faults.
2. Faults in the components making up the system. Components may be defective because of the manufacturing process or because of wear and tear.
3. Faults due to environmental effects. Devices can be subjected to a large number of environmental stresses, e.g. radiation for space vehicles, temperature, or high G-forces.

Classifying Faults

Faults may be classified using the following set of classes:

1. *Temporal behaviour classification*: There are essentially three types of faults:
 - (a) *Permanent* faults do not die away with time but remain until they are repaired or the faulty component replaced.
 - (b) *Intermittent* faults cycle between fault-active and fault-benign states.
 - (c) *Transient* faults die away after some time.
2. *Output behaviour*: There are two categories of output behaviour:
 - (a) A task generates *non-malicious* output if it is interpreted consistently by all processes receiving the output.
 - (b) A task generates *malicious* (or *Byzantine*) output if it cannot be interpreted consistently by all processes receiving the output.

Byzantine faults are much harder to neutralise in practice. We will discuss this in detail in Section 9.5.

3. *Independence and correlation*: Failures may be categorised as follows:
 - (a) *Independent* failures do not cause other failures either directly or indirectly.
 - (b) *Correlated* failures are sets of failures that are related in some way.

Redundancy

The main means of ensuring fault-tolerance is through redundancy. For a system to function despite the loss of some of its components means that there must have been spare capacity to begin with.

Fault-tolerant design techniques can be divided into four categories:

1. *Hardware Redundancy*: this involves multiple processors and duplicate computations.

2. *Software Redundancy*: this involves multiple developments of the software.
3. *Information Redundancy*: this involves adding additional bits for error detection and recovery.
4. *Time Redundancy*: this involves adding slacks to task schedules and rerunning tasks if necessary, which may required rolling back to correctly working states.

The first three methods are the most commonly used in modern high-integrity systems, although the fourth is applied successfully in modern database systems. In this section, we will focus on the first three: hardware, software, and information redundancy.

Hardware and software redundancy are closely related — the former inspiring the latter. However, due to the differences in how hardware and software can fail (as discussed above), techniques from hardware redundancy must be modified to be applied to software redundancy.

Throughout the rest of this chapter, we will look at some methods for achieve hardware, software, and information redundancy, and will analyse their effectiveness and limitations.

9.2 Hardware Redundancy

The analysis of hardware is important in high-integrity systems. As discussed in Chapter 2, software on its own cannot cause physical harm — it operates hardware that causes harm.

The use of hardware redundancy is common-place outside of high-integrity systems. For example, the Google search engine is run on multiple servers to decrease the likelihood of users being unable to connect. However, we are especially interested in not just switching servers when a server goes down, but *detecting and tolerating* specific faults in a system.

9.2.1 Redundancy

The use of redundancy for increase reliability is applied generally to electrical, mechanical, and software systems, however, the use of redundancy for reliability has been around for centuries:

“The most certain and effectual check upon errors which arise in the process of computation is to cause the same computations to be made by separate and independent computers; and this check is rendered still more decisive if their computations are carried out by different methods.” – D Lardner, 1824.

The above quote is from D. Lardner’s article in the *Edinburgh Review* from 1824. Lardner uses the term “computer” as it was 200 years ago: to refer to people who do manual calculations, rather than programs running on hardware.

Lardner’s idea still stands today. To achieve fault tolerance in high-integrity systems, multiple units (hardware and/or software) are used to do the same calculations.

In 1824, if 2-3 “computers” were to calculate a result and find a different answer, the calculation would be re-done by all three, with the hope that the error made would not be repeated. However, when using hardware/software to do calculations, repeating the calculation is unlikely to result a different answer. Instead, different techniques are required to mitigate these problems.

9.2.2 Static pairs

A simple way to implement redundancy of hardware is to hard-wire two processors together to form a *static pair*. The pair runs identical software using identical inputs and compares the output of each process. If the outputs are identical then the pair is functional; otherwise it is deemed faulty and the interface switches the pair off from the network. This is a straightforward way to detect faulty processors/components.

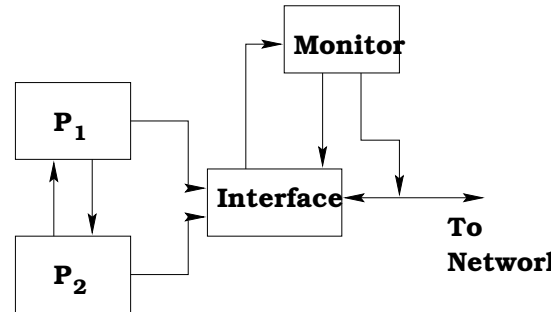


Figure 9.2: Static pairing with a monitor

There are two assumptions in this:

- (a) that the interface does not fail; and
- (b) that both processors do not fail identically and around the same time.

Assumption (a) is not reasonable but assumption (b) is reasonable for hardware failures; that is, given reliable components, it is unlikely that both will fail identically at the same time.

The interface is a critical (single) point of failure of the pair. One way to mitigate this problem is to introduce an interface *monitor* that checks the behaviour of the interface and conversely the interface can check the functioning of the monitor. Figure 9.2 presents a model of this. The monitor checks that if two different values are sent to the interface, the interface correctly switches the pair off the network. The interface checks the behaviour of the monitor as well. This introduces redundancy for the interface, so this cross-checking fails only if the monitor and interface fail at the same time.

9.2.3 Redundancy and voting

Static pairs can be used to detect a faulty component, however, they cannot continue operating reliably if one component fails, as there is no way to determine what the correct operating value should be. In the case of two redundant components, the best we can do when they disagree is to take some action such as shutting down to a fail-safe mode, because it is not straightforward (and often not possible) to determine which component has failed. As such, fault-tolerant designs tend to use *more than two* redundant components.

Once more than two components are used, it must be determined which components are correct, and which are faulty. To do this, we can use *voting*. The basic method is to use multiple units (hardware and/or software) and to vote on the result. The designer must decide whether to use *exact agreement* or *approximate agreement*.

Voting and consensus-based reliability

Consider three processors A , B and C such that A and B produce the value x , while C produces the value $x + \alpha$. The problem in voting-based solutions is:

What is the value of α which is acceptable?

If the same types of processors are executing the same code and receiving the same interrupts at the same time then it is easy to detect the faulty processor. If many components are in *exact agreement*, and one or two are not, then we can be confident that the two that are not in agreement are faulty. However, this is often not the case. *Approximate agreement* arises more often. For example, fluctuations in sensor readings or using different processors and algorithms to eliminate design faults may produce slightly different readings, with no way to determine which is correct.

To mitigate this problem, we require a *metric* $d(x_1, x_2)$ to be defined on outputs x_1 and x_2 . The metric is specific to the system being designed. Essentially, two values are said to be *sufficiently equal* (SE) if and only if $d(x_1, x_2)$ — the difference between the two outputs — is less than some specified error margin, ϵ . That is,

$$SE(x_1, x_2) \Leftrightarrow d(x_1, x_2) \leq \epsilon$$

However, the metric only provides us with a way to determine whether *two* values are sufficiently equal. In voting, we must instead determine that many values (more than two) agree to determine an output value of which we are confident.

Example 9.1

Consider a system with two sensors used to read the temperature of a chemical storage tank. The reading is used to determine what action to take, such as lowering cooling rods into the tank. The system designers decide on an error margin of 0.2 degrees Celsius. The sensors then produce the readings 35.6 degrees and 58.0 degrees. There is no way to decide which reading is accurate, so no action can be taken. However, if we have three sensors, readings producing 35.6, 35.7, and 58.0 degrees respectively, then we can be quite confident (but not certain) that the reading of 58.0 is incorrect, so we can discard this value.

□

In the case of more than two components, we need a *voting algorithm* to determine which readings are correct/incorrect. There are three common voting algorithms used in high-integrity systems:

1. *majority voting*;
2. *generalised k-plurality voting*; and
3. *generalised median voting*.

Each voting algorithm takes a set of values as input, which represent the outputs from the redundant components, and returns a single value based on those inputs, which can then be used by the system to make decisions.

Majority Voting

The concept behind the majority voting algorithm is quite straightforward: the output is the value on which more than half of the components agree. However, this is not as straightforward to decide as it initially seems. For example, given the set of five sensor readings $\{30.1, 30.2, 30.3, 30.4, 30.6\}$, with an error margin of 0.2, what should the output be? The first two elements do not agree with the final element, and the last two elements do not agree with the first. The algorithm needs to provide a single value: which should be chosen?

Assume that there are N outputs to be voted on and that N is odd. From an analysis of the system, we have chosen a metric and ϵ such that $d(x_1, x_2) \leq \epsilon$ if x_1 and x_2 are sufficiently equal for practical purposes (note that d is not transitive).

The majority voting algorithm then proceeds as follows:

1. From the set of inputs, construct classes of inputs, $P_1 \dots P_n$, such that:

- (a) $x, y \in P_i$ iff $d(x, y) \leq \epsilon$; and
- (b) P_i is *maximal*; that is, if $z \notin P_i$ there exists a $w \in P_i$ s.t. $d(w, z) > \epsilon$.

Each class of values should therefore be a set of values that agree with each other, and any input that agrees with that set should be in that set. Values can appear in more than once class.

2. Take the largest class, P_k , generated. This represents the largest maximal set of inputs that agree with each other. In the case that there are multiple largest classes for the same size, break the tie arbitrarily.
3. If P_k contains more than $\frac{N}{2}$ elements (that is, it forms a majority), choose any element in P_k as the output of the voting algorithm. Alternatively, one could take the average value.
4. If P_k contains less than $\frac{N}{2}$ elements, the voting algorithm reports a failure — no consensus could be reached. In such a case, the greater system must determine what to do, such as entering a fail-safe mode or raising an alert.

So, the majority algorithms finds the largest set of values that agree with each other. If this forms a majority, then any value in that set is representative of the majority, and can be used as the voting algorithm output.

Example 9.2

Consider an example of an array of five sensors measuring the temperature of a chemical storage tank. The readings of the five sensors, and therefore the input to the voting algorithm are:

30.1, 30.2, 30.3, 30.5, 30.7

The error margin is 0.2. None of the values can be immediately rejected, because each value agrees with at least one other value. Using the majority voting algorithm, we would allocate values to the following classes:

$$\begin{aligned} P_1 &= 30.1, 30.2, 30.3 \\ P_2 &= 30.3, 30.5 \\ P_3 &= 30.5, 30.7 \end{aligned}$$

The class P_1 is the largest class contains more than $\frac{5}{2}$ elements, so any value will suffice. We select the middle value, 30.2, as the output.

□

Generalised k -plurality voting

A generalised k -plurality voter is the same as a majority voter, except that the P_i that is chosen must contain at least k elements, where k is determined by the system designer, rather than simply $\frac{N}{2}$ elements.

A majority vote is simply a generalised k -plurality voter with $k = \frac{N}{2}$.

A k -plurality voter can be used to raise (increase k) or lower (decrease k) the risk tolerance of a system. That is, for a higher value of k , meaning that a larger set must agree, the system designers are specifying that they will be less tolerant of failure, and therefore, they will be more risk averse. In most real applications, k is specified to be less than $\frac{N}{2}$, meaning that a majority is not required.

Generalised median voting

The generalised median voter is a straightforward mechanism that simply chooses the median value by continually removing the elements n and m , such that for all $i, j \in P$, $d(n, m)$ is greater than $d(i, j)$, until one element remains. Thus, it requires inputs that with a total order relation between them, such as $\leq$.

An *averaging* voter is not sufficient because even a single faulty unit can skew the results terribly. For example, consider a faulty unit that returns 0 for all measurements, with two correctly functioning units. They may return 0, 100, 102. The average is 67.3*, while the correct value is more likely to be around 101.

Question: Couldn't the generalised median voting algorithm be even more influenced by faulty units: what if the median value is generated by a unit that is faulty?

Comparison of voters

Table 9.1 presents a comparison of the three voters for various situations. For example, if the outputs from all components are correct and sufficiently equal (the first row of the table), then all three voters will produce a correct output. However, if a majority of the outputs are incorrect and none are sufficiently equal (fifth row), then the majority and k -plurality voters will each produce no output, while the median voter will produce an incorrect output.

From this table, we can see some interesting aspects of the voters. Median voters always produce an output, so are useful in systems in which a decision must be made (e.g. there is no option to enter a fail-safe mode). Majority and k -plurality voters will only produce incorrect output in the unlikely event that there is a majority (or k -majority) that are all incorrect and sufficiently equal. Thus, these are used in systems in which taking the right decision is more important, and when the voter has no output, some alternative action (such as entering a fail-safe mode) is preferred.

Case	Majority voter	k -plurality voter	Median voter
All outputs correct and SE*	Correct	Correct	Correct
Majority correct and SE	Correct	Correct	Correct
$k^\dagger$ outputs correct and SE	No output	Correct	Possibly correct
All correct but none SE	No output	No output	Correct
All incorrect and none SE	No output	No output	Incorrect
$k^\dagger$ outputs incorrect and SE	No output	Incorrect	Possibly correct
Majority incorrect and SE	Incorrect	Incorrect	Incorrect
All incorrect and SE	Incorrect	Incorrect	Incorrect

* SE = Sufficiently equal

† where $k < \frac{N}{2}$

Table 9.1: Comparison of voter types.

9.2.4 N-Modular Redundancy (NMR)

The N-modular redundancy scheme is a method used for masking faulty components in system. It works by using multiple (N) redundant components (where N is usually odd) and voting on their output. There are two typical designs, outlined in Figure 9.3. This figure shows the popular *triplex* design. The first design has one voter, while the second design has three voters, which mitigates the risk of the voter failing.

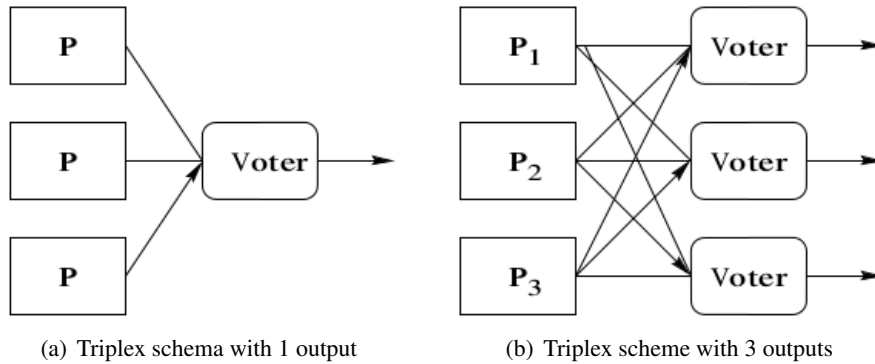


Figure 9.3: Structure of a N-modular sub-system

A system of $N = 2m + 1$ components can tolerate m failed units. The most popular is the *triplex* or *triple redundancy* scheme, which can tolerate the failure of a single unit. For example, if a failure occurs in a triplex then the faulty unit can be detected next time there is a vote and the unit isolated. This then forms a duplex that can detect a faulty unit, but it cannot mask it, because it is now similar to a static pair.

Exercise 9.1

With N units, how many failed units can N-modular redundancy tolerate to *detect* a faulty unit, rather than *mask* it? $\square$

Purging

When a faulty unit is detected, it should be *purged*, which means its results should no longer be considered, unless there is some reason to believe that the fault is transient. Purging can be done in two ways:

- *self-purging*, in which a unit compares its result to the voted output, and purges itself if it is sufficiently different; or
- sift-out redundancy, in which all combinations of the pairs of units are compared with each other (producing a total of $\binom{N}{2}$ outputs), and a controller can disconnect units that disagree with the majority.

While having multiple voters enables redundancy in the voters, there are other pragmatic reasons for having multiple voters. Many applications may require only one output, however for others, multiple outputs may be sensible. For example, a braking system with four sensors (one on each wheel) to determine the current speed must also send four values to the four brake callipers. If one of the voters sends an incorrect value, the other three are still able to produce enough force for the car to brake at close to the correct speed. However, if one value is sent, and it happens to be incorrect, then the force applied may not brake the car.

9.3 Software Redundancy

Fault tolerance for software is harder than for hardware. While we can run software on multiple units of hardware, simply installing multiple copies of the software on redundant hardware does not mitigate software failures, because in general, the N copies of the software will all fail at the same time, caused by the same design fault. These are called *common-mode* failures.

There are two general approaches used for software redundancy: *N-version programming* and *recovery blocks*. In this section, we present both of these in detail.

9.3.1 N-Version Programming

N-version programming is a software-based version of hardware redundancy. The catch is that we cannot simply create N copies of the same software, as these will all fail on the same input. Instead, N -version programming requires that N *independent* versions of the code are written. All N independent versions of the code are executed and the output is voted on. Voting algorithms used in hardware redundancy can be used (e.g. majority voting).

However, careful planning and thought is required for this:

- First, it will be considerably more expensive. For hardware items, the design is a big cost, but producing new hardware components is often relatively less expensive (compared to the design process).

For software however, the effort in create new copies is essentially zero: copying some bits is so cheap as to not worry about the cost. The cost in software is all in the design. Thus, the effort to produce two independent software components is approximately double producing one.

- Second, it will not be effective if the same design and coding faults are replicated. It is well documented that if multiple separate software teams design, implement, and test the same system, they are likely to make mistakes in the same places as each other. Thus, we see common-mode failures in independent implementations of the same specification.

Common-mode failures

Common-mode failures in engineering refer to those failures that are not statistically independent from one another. For example, the back two tires on a car are likely to fail (run out of tread) at around the same time because they will have driven the same distance.

In software, common mode failures typically result because programmers tend to make the same mistakes as each other; e.g. misinterpreting requirements in the same way, small boundary shifts, or using the same (faulty) algorithm from a text book.

Causes of common-mode failures in software

In software, common-mode failures can occur in multiple different implementations of the same system, irrelevant of operating environment or how long they have been in place. Common-mode failures can result in N-version systems due to the following:

1. Faults in the original system specification. If two versions are implemented from the same specification, which contains a fault, then the two versions are likely to contain the same fault. In software redundancy, versions are often written from the same specification.
2. Development environments. Similar languages and environments encourage similar styles and similarity in programs, using the same programming tools may result in the same faults.
3. The use of similar algorithms. For example, different teams implementing the same numerical algorithm may implement the same instabilities, especially if they both use well-known algorithms for solving the problem.
4. Similar training to the independent teams. This is common within organisations.

Eliminating common-mode failures

Eliminating common-mode failures in N-version programming is done via introducing *design diversity*. Essentially, this means implementing the same specification using diverse methods, tools, and teams.

In response to the four points in the previous section, common-mode failures can be eliminated using the following:

1. Eliminate as many faults as possible in the specification (easier said than done!).
2. Have a controlling individual who specifies the programming environments, languages, and tools that are used by the separate teams. That is, one person (or group of people) is responsible for ensuring that separate teams use diverse means for designing, implementing, and testing the software.

3. Keep development teams as separate as possible to avoid the use of common algorithms and data structures. This includes keeping them geographically separate to avoid them discussing their solutions with each other.
4. Use teams that have been educated/trained by different organisations. It is not uncommon to outsource to multiple organisations to introduce diversity.

Design diversity through independence?

Does using design diversity work? Unfortunately, statistics on these methods are hard to come by, because performing controlled experiments on large-scale systems is expensive.

One academic study performed by Knight and Leveson [1] attempts to assess the impact of independence on common-mode failures. This study compared common-mode failures of student assignments implemented by individuals that were geographically separated. Overall, 27 different versions from two different universities were produced from the same specification: 9 versions from the University of Virginia (UVA) and 18 from the University of California and Irvine (UCI). The system was a simple missile defence system that took radar data as input and had to decide whether the signals represented incoming missiles. Neither set of students (from the different universities) were told that the other university was participating in the experiment, so it is unlikely that there was communication between individuals at different universities.

All 27 versions were tested on the same 1 million tests, using the original version (produced by NASA) as the test oracle. Thus, the only faults that would not be noticed were those that failed on all 28 versions of the system.

Knight and Leveson compared the 9 UVA versions against the 18 UCI versions for common failures: that is, those that failed for the same test. The results are shown in Table 9.2. For example, versions 3 and 11 both failed on the same 58 inputs as each other (out of a total of 1 million inputs).

The results indicate that being part of independent teams is perhaps not enough to provide reliable results. For example, version 3 and 20 both failed on 325 of the 1 million outputs — too high for a missile defence system.

Of course, from the results we can see that versions 3 and 8 are both particularly error prone; as are versions 20 and 26. These versions are probably less reliable than what we would expect from a team at NASA implementing a missile defence system. In addition, all versions used the same programming language. Nonetheless, the results indicate that independence alone may not be sufficient to achieve fault tolerance.

9.3.2 Recovery blocks

The *recovery block* approach is similar to N-version programming in that it also uses N versions. However, only one version is executed at any one time. If the output does not pass an *acceptance test* then next version is tried, and so on. Figure 9.4 shows a schematic for this design pattern.

When an acceptance test fails and another version is invoked, all changes to the state of the system must be reversed.

Correlated failures between UCI and UVA versions

UCI versions	UVA versions								
	1	2	3	4	5	6	7	8	9
10	0	0	0	0	0	0	0	0	0
11	0	0	58	0	0	2	1	58	0
13	0	0	1	0	0	0	71	1	0
14	0	0	28	0	0	3	71	26	0
15	0	0	0	0	0	0	0	0	0
16	0	0	0	0	0	1	0	0	0
17	2	0	95	0	0	0	1	29	0
18	0	0	2	0	0	1	0	0	0
19	0	0	1	0	0	0	0	1	0
20	0	0	325	0	0	3	2	323	0
21	0	0	0	0	0	0	0	0	0
22	0	0	52	0	0	15	0	36	2
23	0	0	72	0	0	0	0	71	0
24	0	0	0	0	0	0	0	0	0
25	0	0	94	0	0	0	1	94	0
26	0	0	115	0	0	5	0	110	0
27	0	0	0	0	0	0	0	0	0

Table 9.2: Results of the Knight and Leveson study on correlated failures

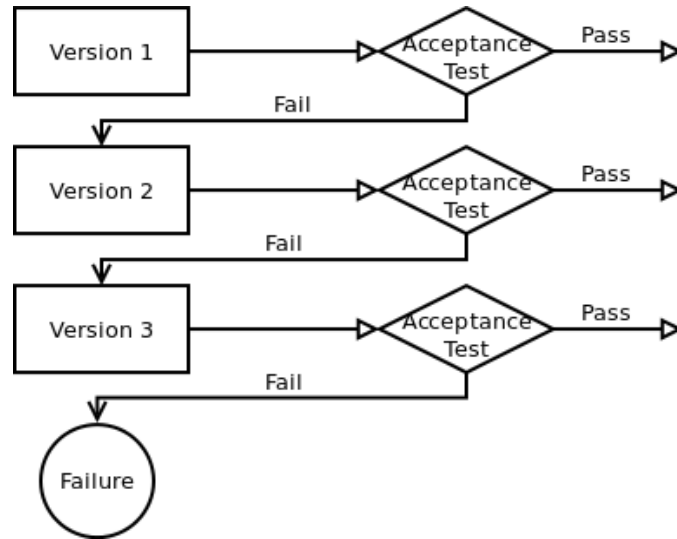


Figure 9.4: A schematic diagram for recovery blocks

Acceptance tests

Recovery blocks require *acceptance tests* to determine whether the output is correct or not. This is the same as the *oracle problem* in testing.

Some acceptance tests can be straightforward to define; e.g. for example, a program that solves values for variables in an equation can be checked by substituting in the answer. Others, are more difficult and require something similar in complexity to the original program. For some problems, the complexity of checking a solution is considerably less than solving that problem, however, in many other cases, a *sanity check* is the best we can do.

Example 9.3

As an example of an acceptance test, consider an algorithm that calculates the GPS coordinates of an unmanned vehicle to determine its location. Determining the position from GPS data is complex, however, estimating whether the location is correct based on other data can be reliable. For example, if the code performing the acceptance tests records the previous position from a few seconds prior, and knows the approximate speed and direction in which the vehicle is headed, then it can estimate the location using a simple “dead reckoning” algorithm.

Using such an algorithm, a small range of possible positions can be determined, and the output passes the acceptance test if and only if it falls inside that range.

□

The challenge with acceptance tests is to produce a tight but accurate range of possible values. If the designer attempts to be too accurate, this may result in a lot of false negatives — that is, labelling an output as incorrect when it is in fact correct. If the design sets the range to loosely, this may result in a low of false positives — that is, labelling an output as correct when it is incorrect.

The reliability of a recovery block is only as good as its acceptance test, so getting this right is of high importance. However, writing acceptance tests is more art than science.

Alternative versions in recovery blocks

As with N-version programming, recovery blocks require diverse designs to mitigate common-mode failures, and so implementing the different modes is expensive. The question is then: *why use recovery blocks instead of N-version programming?*

One reason is efficiency. The first version will generally be correct, so only one version will be executed. In addition, voting is not required.

However, a more pragmatic reason is development cost. Because the first version is likely to be correct, it will be executed the most. Similarly, if the first version fails, then the second version is likely to be correct as well, so the third version will hardly (if ever) be executed. As a result, one common strategy is to make each successive version more simple than the previous.

If the system can tolerate some outputs that are less accurate or are produced in a longer time, then an approximation algorithm or an algorithm with higher execution time may be much easier to develop. This has three benefits:

1. It reduces development costs, because each successive version should be easier and cheaper to implement than the last. For N-version programming, each version must be a “full” version.
2. It introduces more design diversity between the versions. If the different algorithms are actually expected to produce different results, and may even take different inputs, then the diversity between the algorithms will be greater.

3. The successive versions are more simple, which reduces the probability that there are faults. This increases the reliability of the entire block.

Example 9.4

Consider the GPS algorithm for the unmanned aerial vehicle discussed in Example 9.3. We could implement two versions in a recovery block, and one acceptance test.

The first block uses the GPS data to calculate the coordinates, and, as in Example 9.3, a “dead reckoning” algorithm is used to determine a range of values for the acceptance test.

If the acceptance test fails, the dead reckoning algorithm is used to determine the *most likely* location based on the previous location, the direction, and the speed. This is then used as the output.

Most GPS devices use such a recovery-block approach to determining GPS location when they cannot receive a signal. Just check out your device the next time that you are in a tunnel with no GPS signal — most devices will still attempt to track your current location.

□

9.4 Byzantine Failures

Recall from Section 9.1 that whenever a fault causes multiple users of a unit to see different behaviour from that unit, the units is said to be behaving *maliciously*.

The idea of malicious output is a strange one, so let’s illustrate it with an example. Consider a sensor sending out temperature readings to multiple components. If the communication mechanism between the sensor and the components is failing then, for each reading, each component may be receiving *different* readings from the sensor. This is classed as a malicious or *Byzantine* failure.

The term “Byzantine” comes from the *Byzantine Generals’ Problem*, proposed by Lamport et al. [3].

We will follow the description in Sections 17.1, 17.2 and 17.4 of dgc.ethz.ch/lectures/podc_allstars/lecture/podc.pdf.

9.5 Information Redundancy

Information redundancy is one method for increasing the fault tolerance of a system. The basic idea behind information redundancy is to store/send more information than is necessary, and to use that extra information to check for errors, and in some cases, to correct errors.

Coding schemes are typically used to implement information redundancy. Network communications research and microprocessor research have resulted in some efficient methods that use information redundancy to detect corrupted data.

9.5.1 Duplication

Duplication is the most basic form of information redundancy: all information is duplicated. Errors can be detected by simply comparing each bit and its duplicate.

However, this requires twice as much storage/communication, requiring double the resources, so it is too inefficient to be implemented in most systems, especially real-time systems, or resource constrained systems, such as remotely operated vehicles.

Duplication can detect errors in information, but if the information is duplicated only once, then it cannot correct errors, because we cannot know whether it is the original or duplicate that is in error. To correct information errors, *more than two* copies are required, which is a considerable overhead for many systems.

9.5.2 Parity coding

Error detection using parity coding

Parity coding is a simple method for detecting 1-bit errors in information; that is, where 1 bit is incorrect.

It works by counting whether there is an odd or even number of 1s in an n -bit word, and then appending 0 (odd) or 1 (even) to the end of the bit. When this information is read, a check can be done by counting the bits in the first n bits of the word, and comparing the result to the $n + 1$ th bit. If these are different, then an error has occurred.

Example 9.5

The 8-bit word 00001111 is transferred over a communication channel between two nodes. There are an even number of 1s, so 9 bits are transferred: 00001111; with the final 1 indicating that there is an even number of 1s in the first 8 bits.

If one of the bits is corrupted, for example, the first bit (10001111), then the number of 1s is now odd. The receiving node counts the number of 1s in the first 8 bits, notices that this is odd, and compares this with the final bit, thereby detecting the error.

□

Clearly, parity coding only works if an odd number of bits have been corrupted. If two bits are corrupted, then the number of 1s will be the same as before.

Despite this, the simplicity of the method and the small overhead make it popular for information redundancy.

Error correction using interlaced parity coding

Interlaced parity coding is a parity coding method that can *correct* as well as detect some bit errors. It works by dividing the word up into disjoint portions, and representing each portion with one bit — again using 0 or 1 to represent whether there are an even or odd number of bits.

Table 9.3, taken from Krishna and Shin [2], shows an example of this for an 8-bit word. Four parity bits are used, and depending on which parity bits are wrong, we can detect and correct one faulty bit, including if the parity bit is in error. In Table 9.3, the parity bits are associated with the bits using the following code:

- P_0 covers bits w_0, w_2, w_3, w_6, w_7 , and P_0
- P_1 covers bits w_0, w_1, w_3, w_5, w_6 , and P_1

- P_2 covers bits $w_0, w_1, w_2, w_4, w_5, w_7$, and P_2
- P_3 covers bits w_0, w_1, w_2, w_3, w_4 , and P_3

Example of overlapping parity with eight information bits

Bit error	Leads to error in parity bit			
	P_3	P_2	P_1	P_0
w_0	×	×	×	×
w_1	×	×	×	
w_2	×	×		×
w_3	×		×	×
w_4	×	×		
w_5		×	×	
w_6			×	×
w_7		×		×
P_0				×
P_1			×	
P_2		×		
P_3	×			

Table 9.3: Parity coding for an 8-bit word, taken from Krishna and Shin [2]

So, given an 8-bit word, the sender calculates the first parity bit (P_0) by counting whether there is an odd or even number of bits in the portion w_0, w_2, w_3, w_6 , and w_7 , and similarly for the other parity bits and their respective portions. It will then send a 12-bit word. The receiver can use this information to determine which (if any) bit is faulty.

Clearly there is an added overhead in this approach compared to the error detection technique discussed above, because it requires additional parity bits. As such, this scheme is generally only used in situations in which the original word cannot be recovered, such as storing information in memory or on disk. In cases where the word can be re-sent, error detection is generally sufficient.

Example 9.6

Consider an example of storing information in 8-bit words, with the additional 4 parity bits. One part of our program reads the stored word and parity bits: 00000000-1100. It must check whether the extended word is faulty.

It first calculates the expected value of the parity bits. The word is 00000000, so parity bits for this are straightforward to calculate: they are all 1 because there are an even number (zero!) of 1s in the string. As a result, the 12-bit word should be 00000000-1111.

There is no way of retrieving the original stored word, so the best we can do is try to correct this, hoping that a single bit is in error. Looking at the expected and actual parity bits, we see that bits P_2 and P_3 (the

last two) are incorrect. By looking at Table 9.3, we can see that bit w_4 must be the faulty bit (assuming exactly one bit has been corrupted), because this corresponds to the row in the table in which only P_2 and P_3 would be in error.

Because we know that this is the faulty bit, and because we know it can take only one other value (the opposite bit value), we can correct the word to 00001000-1100.

□

Example 9.7

Consider the stored bits 00000000-1101. Again, we know that the parity bits should all be 1, so the bit P_2 is incorrect. Looking at Table 9.3, we can see that the bit P_2 itself is the faulty bit. If we use the coding scheme shown in Table 9.3, then it is always the case that if exactly one parity bit is incorrect, then that bit is the faulty bit.

□

9.5.3 Checksums

The final information redundancy technique that we will look at is *checksums*. Most readers are probably already familiar with the use of checksums in network messaging. Checksums are a useful error detection mechanism, but cannot correct errors. Checksums are used to detect errors in blocks of data, especially when that data is being transferred between nodes.

The approach is simple. Given a block of data, break the data up into words; e.g. 32-bit words, and add these words together to get the checksum. The checksum is then appended to the data.

The receiver of the data strips the checksum upon receipt, re-calculates the checksum in the same way, and compares the re-calculated checksum to the stripped checksum. If they are different, the data has been corrupted.

There are a few different methods for adding together the words. The simplest is known as the *longitudinal parity check*, which simply computes the *exclusive or* of all words. To check the data, the sender simply computes the *exclusive or* of all words *including the checksum*. If the result contains a bit that is non-zero, then an error has been detected.

This can detect errors in multiple bits, but not if there are an even number of errors in the same bit of multiple words. The larger the set of blocks, the more likely it is to encounter an undetected error.

Checksums are popular because the overhead is small: for a block of 8-bit words, only an additional 8-bits are required.

9.6 Fault tolerance in practice

9.6.1 Airbus A310–A380

The Airbus series of aircraft designs rely heavily on hardware, software, and information redundancy to achieve fault tolerant behaviour of aircraft.

1. A310 (circa 1983) had ten separate digital flight control computers.

2. A320 (circa 1988) introduced fly by wire, in which four computers teamed in command/monitor pairs for redundancy, which became standard approach for subsequent Airbus flight control computers.
3. A340 (circa 1992) had one command/monitor pair forming a “fail fast” module, failing over to another command/monitor “hot spare” pair. Error detection was achieved through mismatched command, sequence checking, and self-test when aircraft energised.

A second command/monitor pair using a different microprocessor and different software provided design diversity to tolerate common mode design and manufacturing faults.

4. A380 (circa 2005) employs dual-redundant Ethernet for non-critical functions, electrical and hydraulic flight control diversity.

Design diversity in the A380 project was achieved via using dissimilar computers, physical separation of teams, multiple software bases, different software development tools, and data diversity.

9.6.2 Boeing 777

The Boeing 777 is another aircraft that relies heavily on hardware, software, and information redundancy.

1. Goal of Mean Time Between Actions to 25,000 operating hours, reduce probability of degrading below minimum capability to less than 10^{-10} .
2. Designed to tolerate Byzantine faults, object impact, component failure, power failure, electromagnetic interference, cloud environment.
3. Byzantine fault tolerance with data synchronisation and median voting.
4. Architecture of flight control computer has three independent channels each composed of three redundant computing lanes (command, monitor, standby). Standby allows dispatch of aircraft even with a lane or data channel failure.
5. Design diversity was achieved using different microprocessor hardware and different software compilers for a fault-intolerant single source code.

9.6.3 Boeing 737 MAX Flight Control Redesign

Content may be added in 2026 (please email toby.murray@unimelb.edu.au for more info if you are curious): In subsection 2.4.3 we discussed the case study of the Boeing 737 MAX aircraft. In this section we will talk about the testing regime that the Boeing 737 MAX underwent as part of its re-certification in 2019–2020, as widely reported, and the resulting changes to the architecture of the 737’s flight control system.

9.7 Additional reading

1. These notes were based mostly on Krishna and Shin's chapter "Fault-tolerance techniques" [2], which is Chapter 7 in their book *Real-time Systems*.

A PDF copy of Krishna and Shin's chapter is available on the LMS.

2. Moulding's chapter "Designing for high integrity: The software fault tolerance approach", which is Chapter 3 in C. T. Sennett, ed., *High-integrity Software*, Plenum Press, 1989 is a good resource of fault tolerance in high integrity systems.

The book is available in the library.

3. Laprie et al.'s article [4] titled "Definition and analysis of hardware- and software-fault-tolerant architectures" is an old but excellent overview of fault tolerance in systems engineering. It describes different techniques for achieving fault tolerance, including those outlined in these notes, and discusses their properties, such as cost, benefits, and limitations.

A PDF copy of the article is available on the LMS.

Bibliography

- [1] J. Knight and N. Leveson. An experimental evaluation of the assumption of independence in multi-version programming. *IEEE Transactions on Software Engineering*, SE-12(1):96–109, 1986.
- [2] C. Krishna and K. Shin. *Real-time Systems*, chapter 7. Addison-Wesley, 1999. (Available on the LMS).
- [3] L. Lamport, R. Shostak, and M. Pease. The Byzantine generals' problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401, 1982.
- [4] J-C. Laprie, J. Arlat, C. Beounes, and K. Kanoun. Definition and analysis of hardware-and software-fault-tolerant architectures. *Computer*, 23(7):39–51, 1990.